American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Project Name: University Admission Management System

Advance Database Management System

Mid-Term Project Report

Fall 2025-2026

Section: C

SL #	Student Name	ID	Contribution (%)	Topic Names
01	MD. Nazmus Sadat Numan	22-48497-3	25%	Normalization Table Creation Basic PL/SQL
02	Papia Sultana Prianka	22-48356-3	25%	ER Diagram Schema Diagram Conclusion
03	Syeda Fahmida Sharmin	22-49720-3	25%	Scenario Description User Interface Planning Advance PL/SQL
04	Barno Biswas	22-49203-3	25%	Introduction Project Proposal Data Insertion

Table of Contents

SL #	Contents	Page Number
01	Introduction	01 - 01
02	Project Proposal	01 - 03
03	User Interface Planning	04 - 10
04	Scenario Description	11 - 11
05	ER Diagram	12 - 12
06	Normalization	13 - 23
07	Schema Diagram	24 - 24
08	Table Creation Using SQL	25 - 43
09	Data Insertion Using SQL	43 - 52
10	Basic PL/SQL	52 - 60
11	Advance PL/SQL	61 - 75
12	Conclusion	75 - 76

Introduction:

The university admission process involves many activities like collecting applicant information, verifying academic records, handling payments, evaluating tests, assigning teachers, and finally enrolling selected applicants as students. When all these tasks are handled manually or across scattered files, mistakes become common, data gets lost, and managing thousands of applicants becomes extremely difficult.

To make this entire process smoother and more organized, we developed a digital system called the University Admission Management System. Instead of depending on paperwork or multiple spreadsheets, this system stores everything in one secure and organized place. From the moment an applicant submits their application until the day they officially become a student; every step is recorded clearly. This helps administrators, teachers, and applicants track the process easily and reduces human errors.

With this system, universities can manage applications, payments, evaluations, departments, courses, and enrolled students efficiently saving time, improving accuracy, and ensuring a better experience for everyone involved.

Project Proposal:

1. Problem Statement

Universities handle thousands of applicants every year. Managing the admission process manually creates several challenges:

- Applicant information may become inconsistent or misplaced.
- Tracking applications, payments, and evaluation results becomes time-consuming.
- Departments cannot easily monitor how many applicants applied or were accepted.
- Teachers must evaluate applicants, but without a proper system, records may be inaccurate.
- It is difficult to determine which applicants got admitted as students and maintain their academic information afterward.

Paper-based or spreadsheet-based management often leads to errors, duplicated data, slow processing, and poor data security. As the volume grows, universities require a centralized and reliable way to handle admissions from start to finish.

2. Proposed Solution

To solve these issues, we designed a normalized relational database system using Oracle. This database acts as a unified platform where all admission related information is stored securely and consistently. The system is built around major entities such as Applicant, Application, Payment, Evaluation, Admin, Teacher, Department, Program, Student, and clearly defines the relationships among them.

Key improvements the system provides:

- Automates application submission and tracking.
- Ensures secure and organized storage of personal, academic, and payment information.
- Links evaluations with teachers and applications to avoid confusion.
- Helps departments manage programs, teachers, and enrolled students.
- Supports the complete admission workflow from applying to becoming a student.

This structured approach eliminates redundancy, reduces errors, and guarantees smooth data flow throughout the admission lifecycle.

3. System Overview & Key Features

- **Applicant Management:** Stores detailed personal, academic, and contact information for every applicant, ensuring clean and complete data for evaluations.
- **Application Processing:** Tracks each application with its submission date, status, associated department, and evaluation results.
- **Payment Management:** Handles payment of per application, including amount, method, type, and date, allowing transparent financial tracking.
- **Admin Operations:** Admins can approve applications, publish evaluations, and oversee the overall admission process.

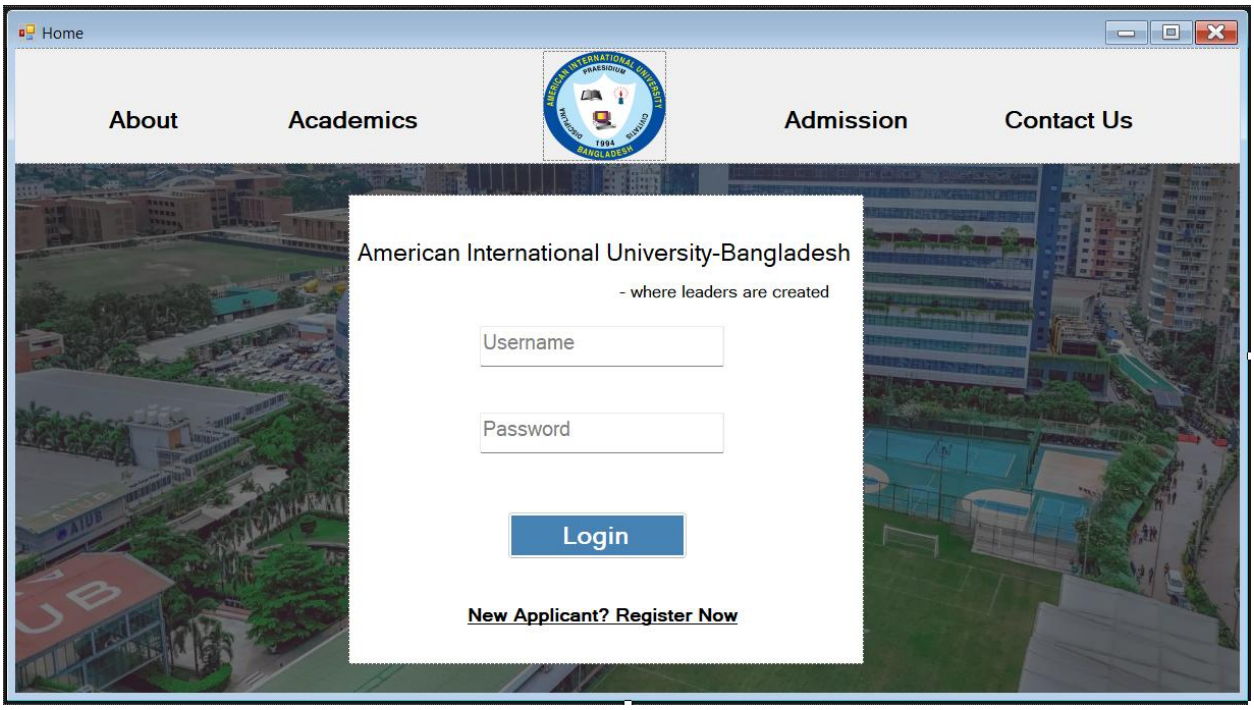
- **Evaluation Management:** Stores written, viva, and final marks while connecting each evaluation to both a teacher and an application for accuracy.
- **Teacher Assignment:** Departments can assign teachers, and teacher's evaluate applicants.
- **Student Enrollment:** Once selected, applicants become students. Their student details, department, admission date, and enrolled courses are stored effectively.
- **Program Offering:** Departments offer programs with specific credit and cost structures.

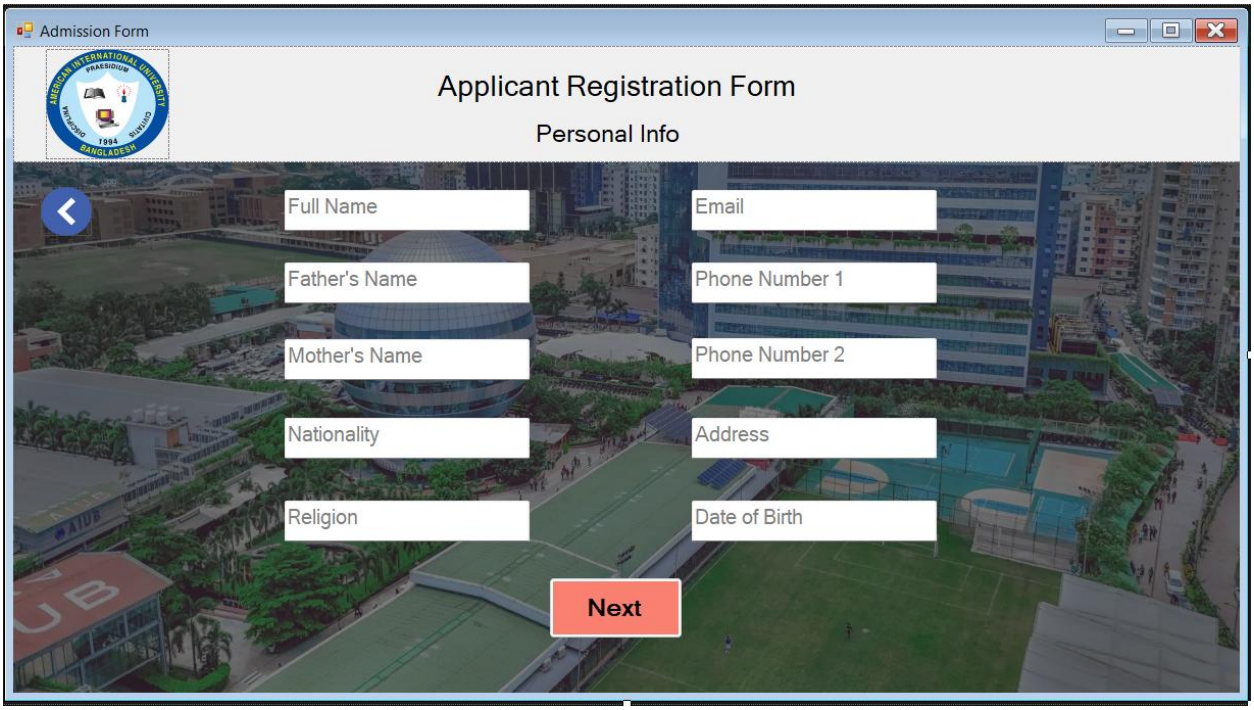
4. Expected Outcome

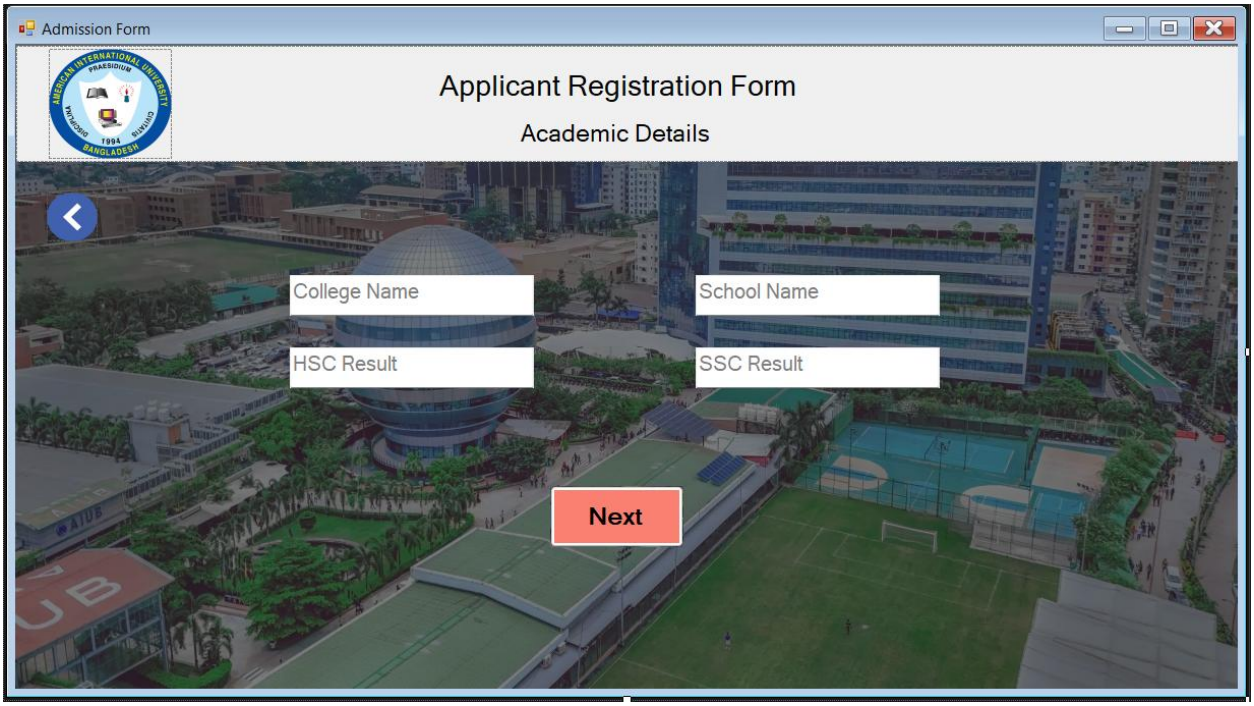
The completed system will result in:

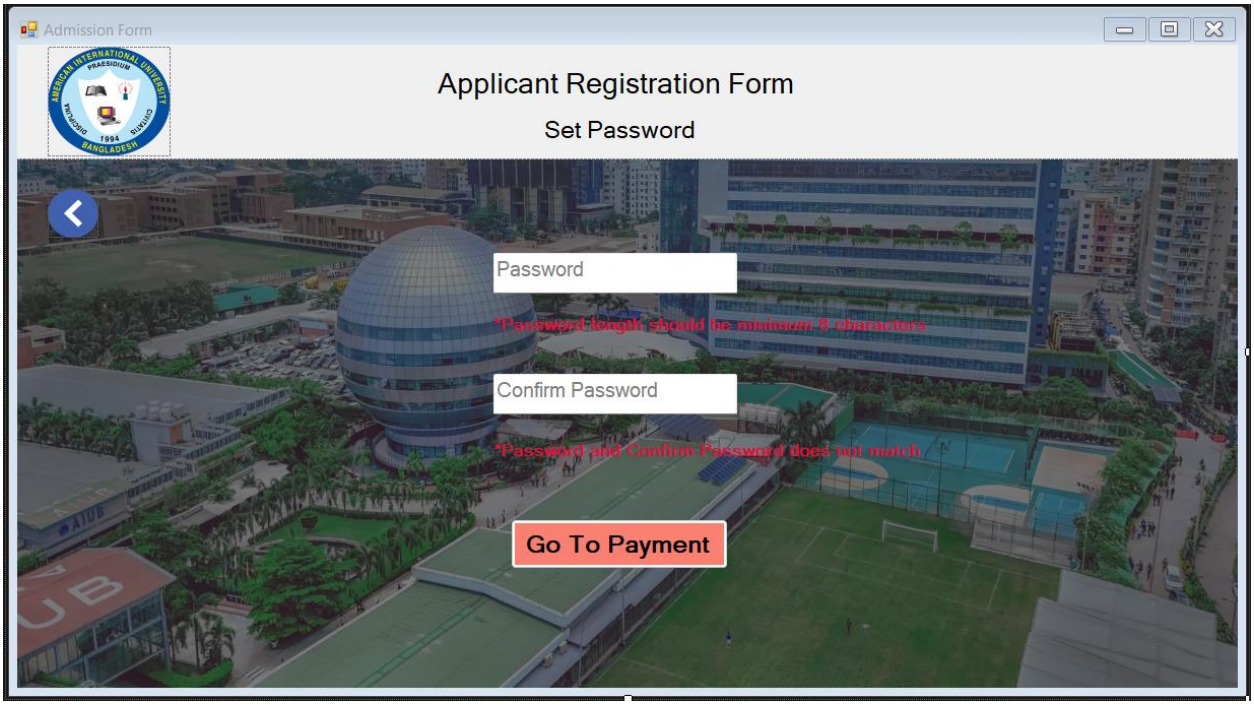
- A well-structured, fully normalized database for university admission requirements.
- Secure, consistent, and organized handling of large-scale applicant or student data.
- Faster admission processing with reduced human error.
- A real-world demonstration of database design principles such as ER modeling, normalization, and relational integrity.
- This database also provides a strong foundation for developing a full-featured admission portal or administrative dashboard in the future.

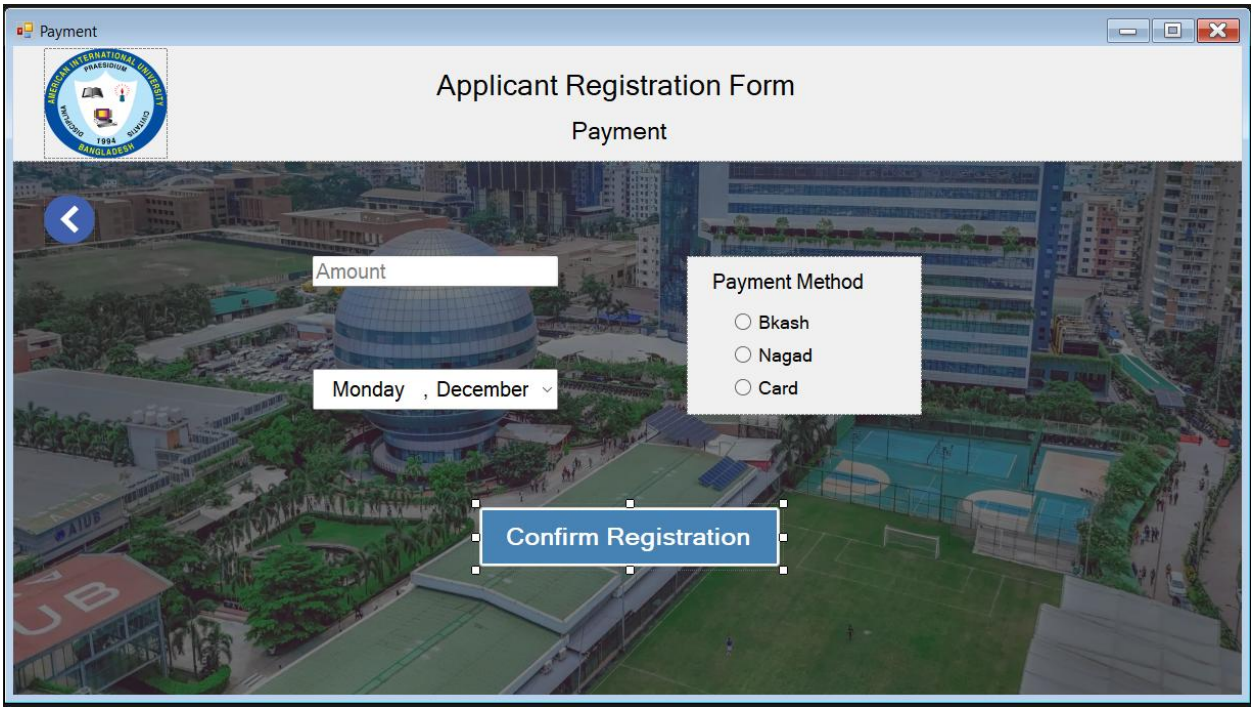
User Interface Planning:

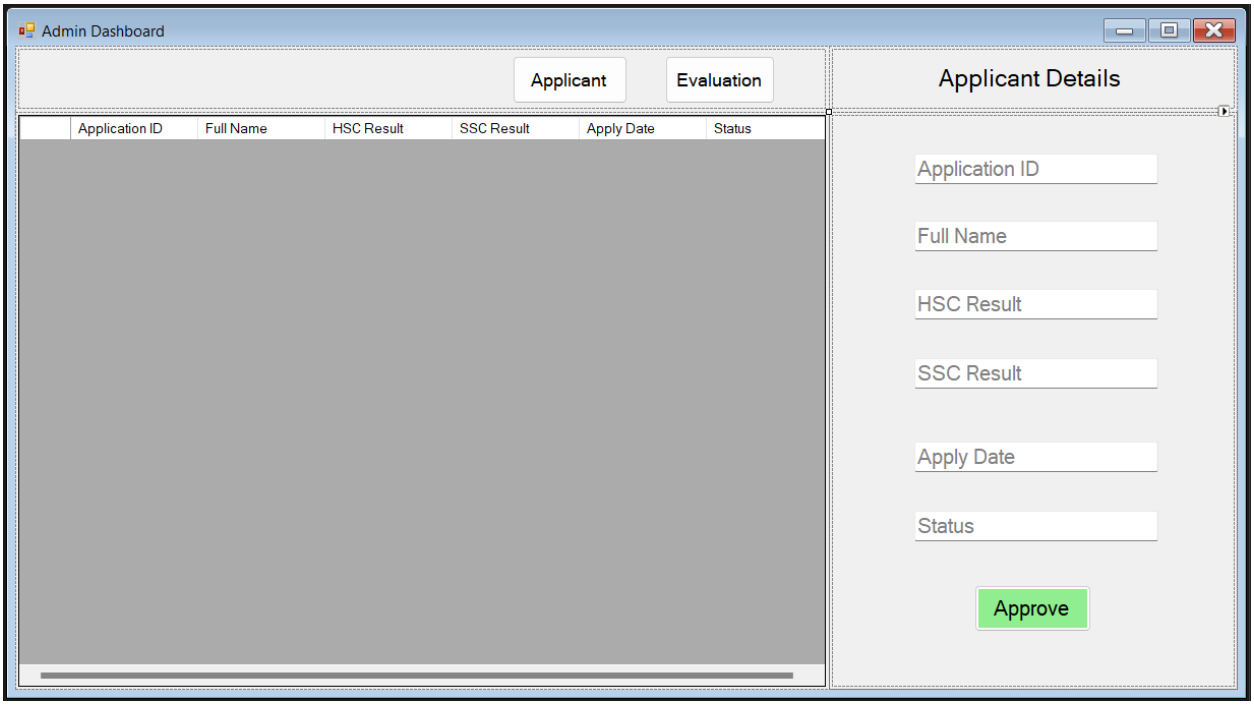


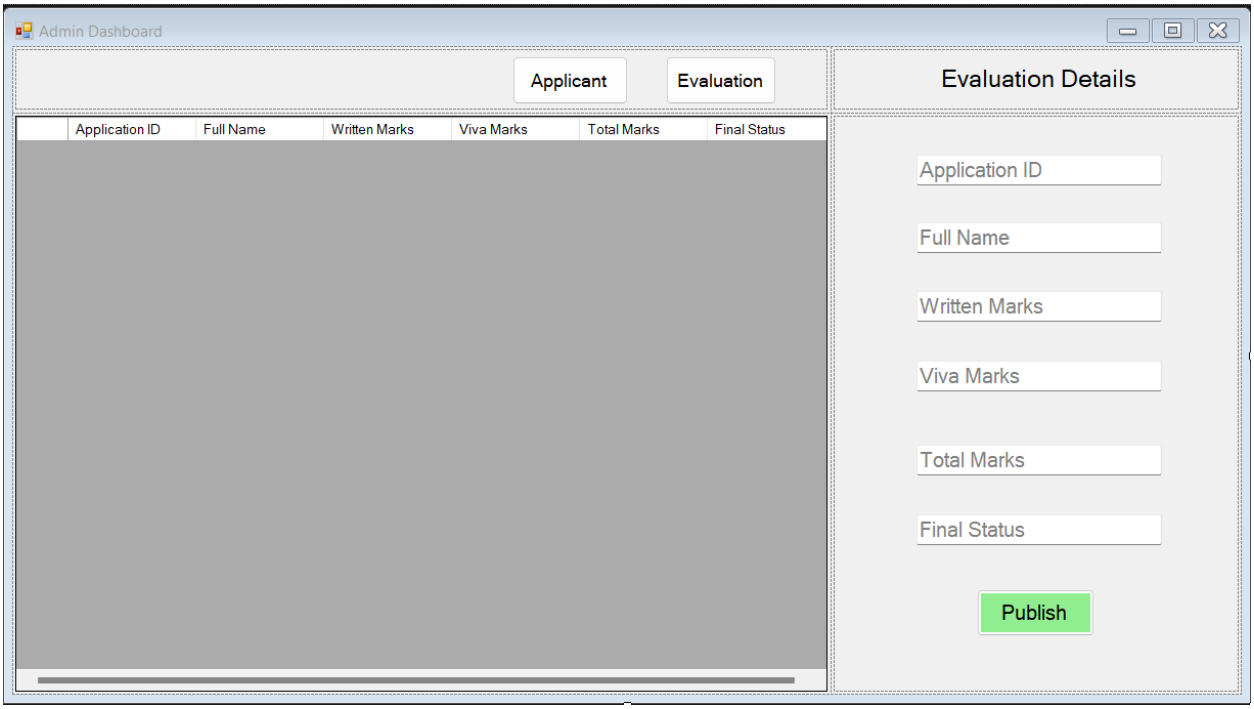


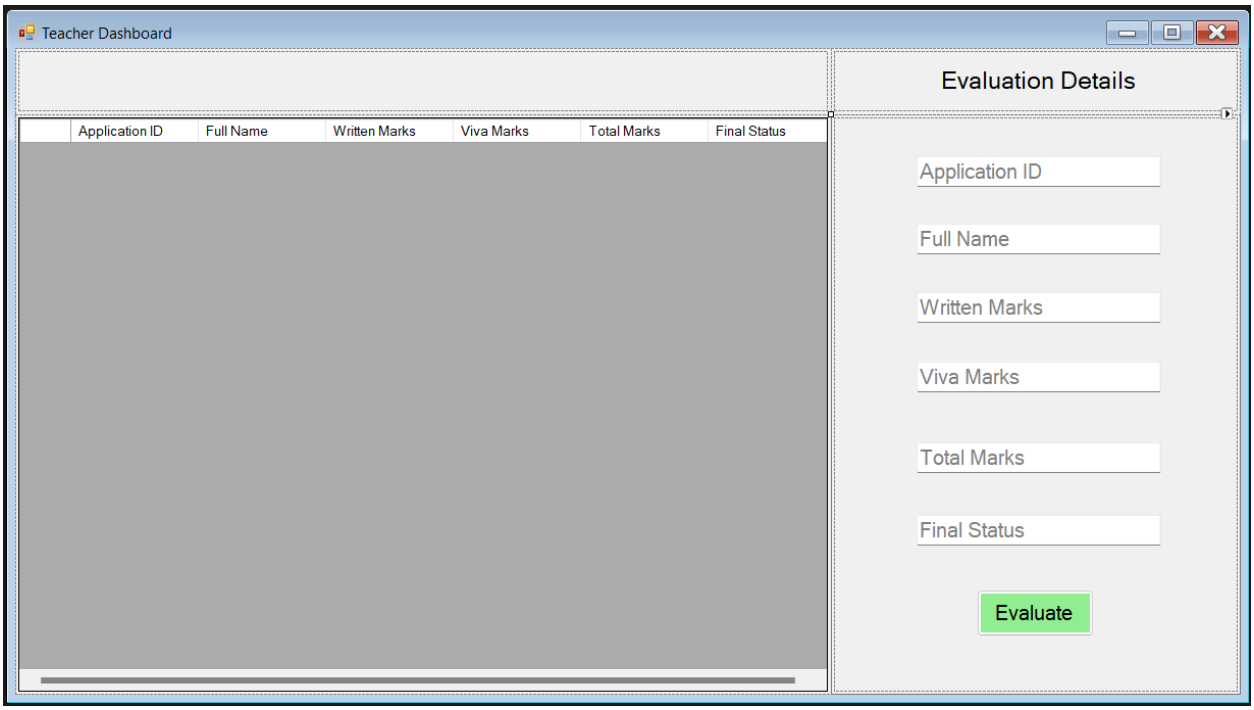

Admission Form
Applicant Registration Form
Academic Details
College Name
School Name
HSC Result
SSC Result
Next

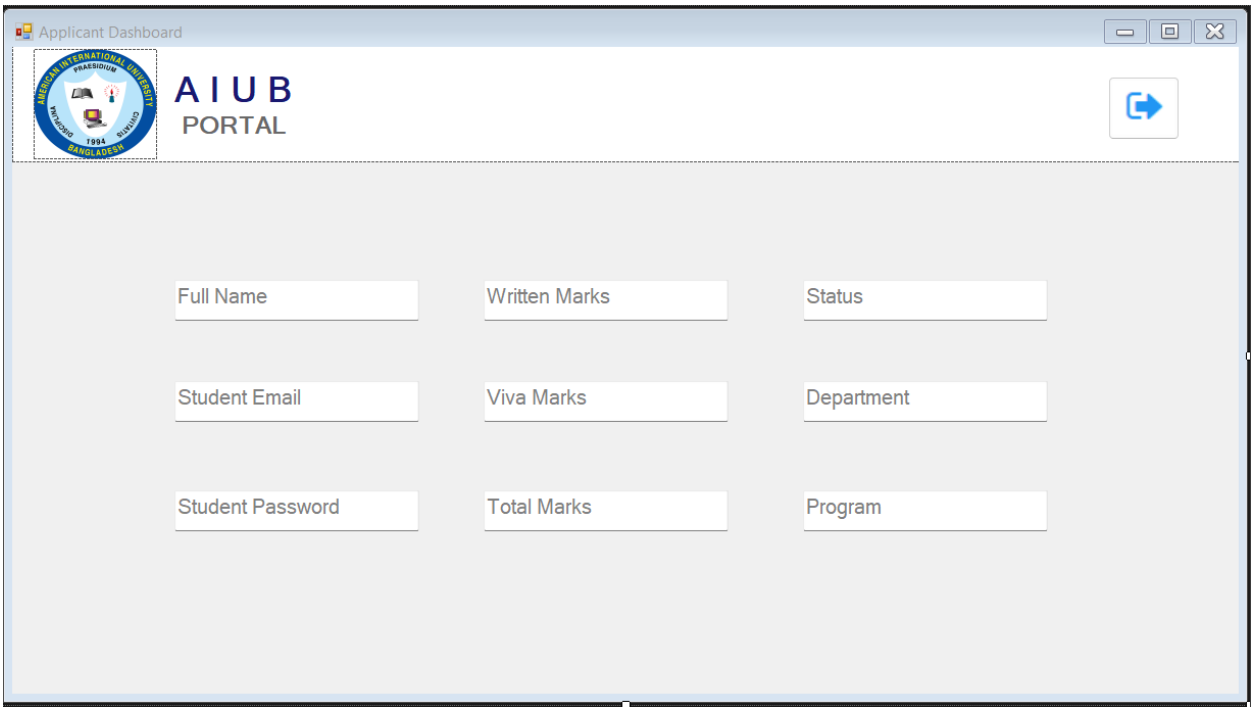

Admission Form
Applicant Registration Form
Set Password
Password
*Password length should be minimum # characters
Confirm Password
*Password and Confirm Password does not match
Go To Payment


Payment
Applicant Registration Form
Payment
Amount
Monday , December
Payment Method
Bkash
Nagad
Card
Confirm Registration

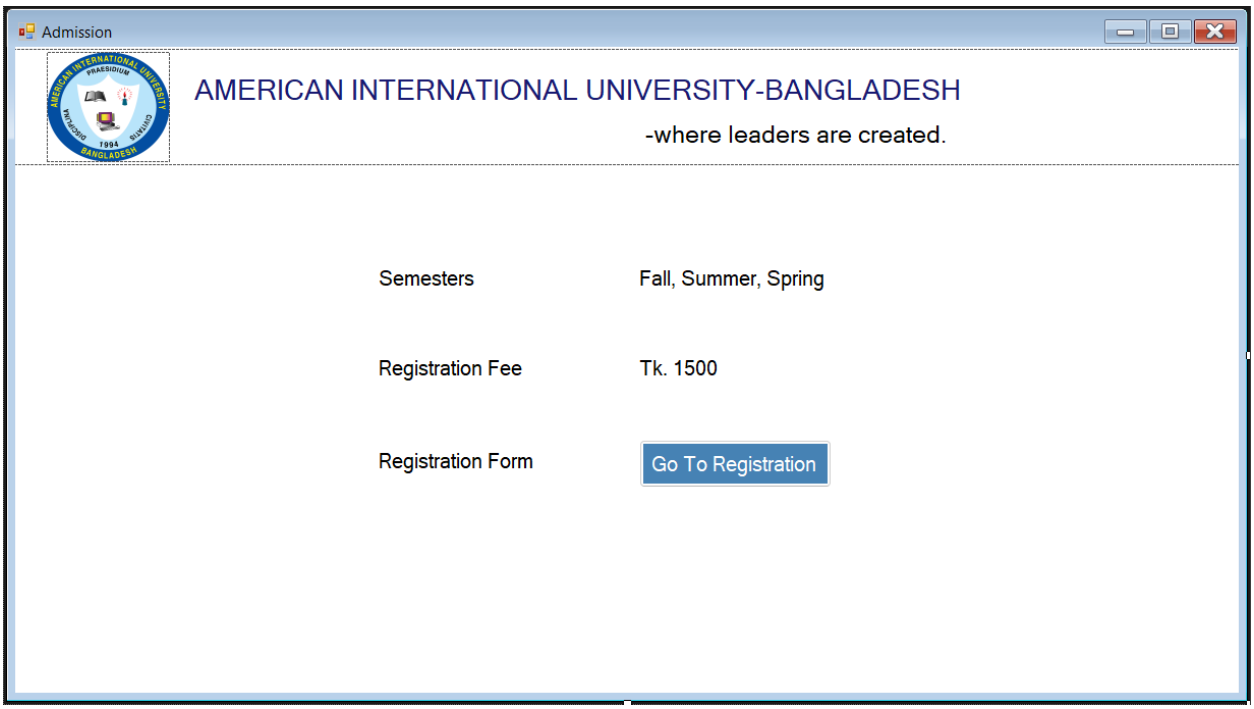

Admin Dashboard
Applicant
Evaluation
Application ID
Full Name
HSC Result
SSC Result
Apply Date
Status
Applicant Details
Application ID
Full Name
HSC Result
SSC Result
Apply Date
Status
Approve


Admin Dashboard
Applicant
Evaluation
Evaluation Details
Application ID
Full Name
Written Marks
Viva Marks
Total Marks
Final Status
Application ID
Full Name
Written Marks
Viva Marks
Total Marks
Final Status
Publish


Teacher Dashboard
Evaluation Details
Application ID
Full Name
Written Marks
Viva Marks
Total Marks
Final Status
Application ID
Full Name
Written Marks
Viva Marks
Total Marks
Final Status
Evaluate


Applicant Dashboard
AIUB
PORTAL
Full Name
Written Marks
Status
Student Email
Viva Marks
Department
Student Password
Total Marks
Program


Academics
AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH
-where leaders are created.
Departments
Programs
Computer Science
BSc in CSE
EEE
BSc in EEE
Business
BBA
Arts
BA in English
Architecture
BArch


Admission
AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH
-where leaders are created.
Semesters
Fall, Summer, Spring
Registration Fee
Tk. 1500
Registration Form
Go To Registration

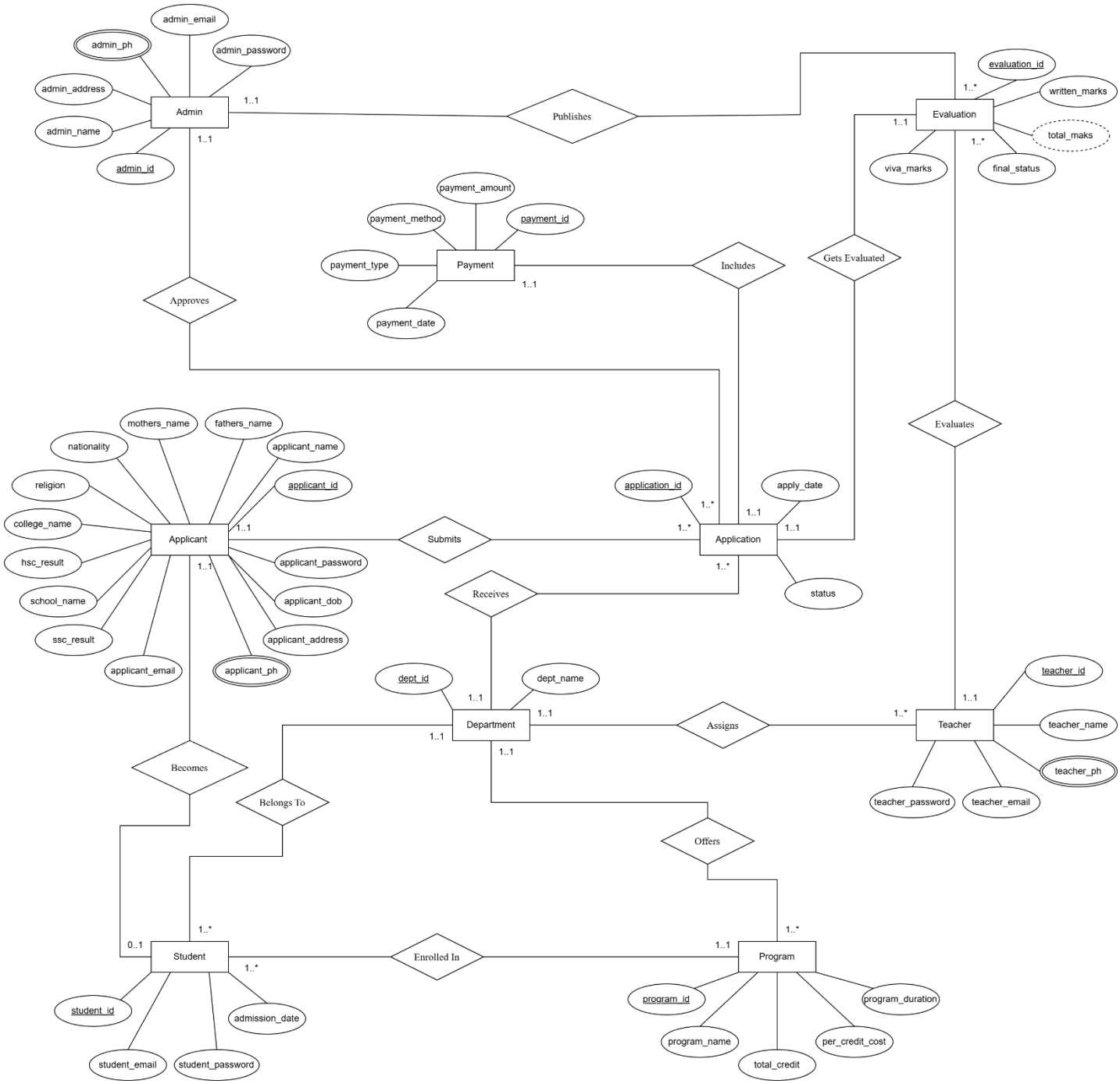

About
AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH
-where leaders are created.
Vision
American International University-Bangladesh continuously transforms the students to become innovative and globally competitive with excellent, state-of-the-art and academic knowledge and skills nurturing their full potentials not only as future leaders in their respective fields of endeavor, but also as unique contributors to the society.
Mission
American International University-Bangladesh is committed to provide quality and excellent academic programs responsive to the emerging global challenges. The university is dedicated to produce and foster competent world class graduates imbued with strong sense of ethical values ready to face the competitive world of business, science, technology, engineering and social sciences.


Contact Us
AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH
-where leaders are created.
Contact
E-mail Us
info@aiub.edu
Visit Us
408/1 (Old KA 66/1), Kuratoli, Khilkhet,Dhaka 1229, Bangladesh
Telephone
+88 02 841 4046-9; +88 02 841 4050

Scenario Description:

The University Admission Management System handles the entire process from application to student enrollment. In this system, one applicant may submit many applications. Many applications may be submitted by one applicant. An applicant is identified by an applicant id. The system also stores applicant name, father's name, mother's name, nationality, religion, college name, hsc result, school name, ssc result, email, phone number, password, date of birth and address. Each applicant can have multiple phone numbers. Each application has a unique identification number, apply date and status. One application may include one payment. One payment may be included by one application. Payment is identified by a payment id and the system store payment amount, method, payment type and date. After payment, one admin may approve many applications. Many applications may be approved by one admin. An admin is identified by an admin id. Admin name, address, phone number, email and password are also stored. Each admin can have multiple phone numbers. One department may receive many applications. Many applications may be received by one department. Each department has a unique identification number, department name and program. One department may offer many programs. Many programs may be offered by one department. Program has a unique id, name, total credit, per credit cost and duration. One application may get evaluated by one evaluation. One evaluation may evaluate one application. One department may assign many teachers. Many teachers may be assigned by one department. Many evaluations may be evaluated by one teacher. One teacher may evaluate many evaluations. To identify a teacher, the system stores a teacher id along with name, phone number, email and password. A teacher may have multiple phone numbers. One admin may publish many evaluations. An evaluation may be published by one admin. Evaluation has an unique id, written marks, viva marks, total marks and final status. One applicant may or may not become a student. A student must be an applicant. A student can be identified by a student id. Student email, password and admission date are also stored. One program may be enrolled by many students. Many students may enroll in one program. Many students may belong to one department. One department may have many students.

ER Diagram:


admin_email
admin_ph
admin_password
admin_address
Admin
admin_name
admin_id
1..1
1..1
Publishes
evaluation_id
written_marks
1..*
Evaluation
1..1
total_maks
1..*
viva_marks
final_status
payment_amount
payment_method
payment_id
payment_type
Payment
payment_date
1..1
Includes
Gets Evaluated
Approves
Evaluates
mothers_name
fathers_name
nationality
applicant_name
religion
applicant_id
college_name
Applicant
hsc_result
applicant_password
school_name
applicant_dob
ssc_result
applicant_address
applicant_email
applicant_ph
1..1
1..1
Submits
application_id
apply_date
1..*
1..1
1..*
Application
1..1
1..*
status
Receives
dept_id
dept_name
1..1
Department
1..1
1..1
Assigns
1..*
Teacher
1..1
teacher_id
teacher_name
teacher_ph
teacher_password
teacher_email
Becomes
Belongs To
Offers
1..*
0..1
Student
1..*
Enrolled In
1..1
Program
1..*
student_id
admission_date
student_email
student_password
program_id
program_duration
program_name
per_credit_cost
total_credit

Normalization:

Approves: (One to Many)

UNF:

Approves (admin_id, admin_name, admin_address, admin_ph, admin_email, admin_password, application_id, apply_date, status)

1NF: admin_ph is a multi-valued attribute.

1. admin_id, admin_name, admin_address, admin_ph, admin_email, admin_password, application_id, apply_date, status

2NF:

1. admin_id, admin_name, admin_address, admin_ph, admin_email, admin_password
2. application_id, apply_date, status

3NF: There is no transitive dependency.

1. admin_id, admin_name, admin_address, admin_ph, admin_email, admin_password
2. application_id, apply_date, status

Table creation:

1. admin_id, admin_name, admin_address, admin_ph, admin_email, admin_password
2. application_id, apply_date, status, **admin_id**

Publishes: (One to Many)

UNF:

(admin_id, admin_name, admin_address, admin_ph, admin_email, admin_password, evaluation_id, written_marks, viva_marks, total_marks, final_status)

1NF: admin_ph is a multi-valued attribute and total_marks is a derived attribute (written_marks + viva_marks).

1. admin_id, admin_name, admin_address, admin_ph, admin_email, admin_password, evaluation_id, written_marks, viva_marks, final_status

2NF:

1. admin_id, admin_name, admin_address, admin_ph, admin_email, admin_password
2. evaluation_id, written_marks, viva_marks, final_status

3NF: There is no transitive dependency.

1. admin_id, admin_name, admin_address, admin_ph, admin_email, admin_password
2. evaluation_id, written_marks, viva_marks, final_status

Table creation:

1. admin_id, admin_name, admin_address, admin_ph, admin_email, admin_password
2. evaluation_id, written_marks, viva_marks, final_status, **admin_id**

Gets Evaluated: (One to One)

UNF:

(application_id, apply_date, status, evaluation_id, written_marks, viva_marks, total_marks, final_status)

1NF: There is no multi valued attribute. total_marks is a derived attribute (written_marks + viva_marks).

1. application_id, apply_date, status, evaluation_id, written_marks, viva_marks, final_status

2NF:

1. application_id, apply_date, status
2. evaluation_id, written_marks, viva_marks, final_status

3NF: There is no transitive dependency.

1. application_id, apply_date, status
2. evaluation_id, written_marks, viva_marks, final_status

Table creation:

1. application_id, apply_date, status
2. evaluation_id, written_marks, viva_marks, final_status, **application_id**

Includes: (One to One)

UNF:

(application_id, apply_date, status, payment_id, payment_amount, payment_method, payment_type, payment_date)

1NF: There is no multi valued attribute.

1. application_id, apply_date, status, payment_id, payment_amount, payment_method, payment_type, payment_date

2NF:

1. application_id, apply_date, status
2. payment_id, payment_amount, payment_method, payment_type, payment_date

3NF: There is no transitive dependency.

1. application_id, apply_date, status
2. payment_id, payment_amount, payment_method, payment_type, payment_date

Table creation:

1. application_id, apply_date, status
2. payment_id, payment_amount, payment_method, payment_type, payment_date, **application_id**

Evaluates: (One to Many)

UNF:

(teacher_id, teacher_name, teacher_ph, teacher_email, teacher_password, evaluation_id, written_marks, viva_marks, total_marks, final_status)

1NF: teacher_ph is a multi-valued attribute and total_marks is a derived attribute (written_marks + viva_marks)

1. teacher_id, teacher_name, teacher_ph, teacher_email, teacher_password, evaluation_id, written_marks, viva_marks, final_status

2NF:

1. teacher_id, teacher_name, teacher_ph, teacher_email, teacher_password
2. evaluation_id, written_marks, viva_marks, final_status

3NF: There is no transitive dependency.

1. teacher_id, teacher_name, teacher_ph, teacher_email, teacher_password
2. evaluation_id, written_marks, viva_marks, final_status

Table creation:

1. teacher_id, teacher_name, teacher_ph, teacher_email, teacher_password
2. evaluation_id, written_marks, viva_marks, final_status, **teacher_id**

Submits: (One to Many)

UNF:

(applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph, applicant_password, applicant_dob, applicant_address, application_id, apply_date, status)

1NF: applicant_ph is a multi-valued attribute.

1. applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph, applicant_password, applicant_dob, applicant_address, application_id, apply_date, status

2NF:

1. applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph, applicant_password, applicant_dob, applicant_address
2. application_id, apply_date, status

3NF: There is no transitive dependency.

1. applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph, applicant_password, applicant_dob, applicant_address
2. application_id, apply_date, status

Table creation:

1. applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph, applicant_password, applicant_dob, applicant_address
2. application_id, apply_date, status, **applicant_id**

Becomes: (One to Zero/One)

UNF:

(applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph, applicant_password, applicant_dob, applicant_address, student_id, student_email, student_password, admission_date)

1NF: applicant_ph is a multi-valued attribute.

1. applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph, applicant_password, applicant_dob, applicant_address, student_id, student_email, student_password, admission_date

2NF:

1. applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph, applicant_password, applicant_dob, applicant_address
2. student_id, student_email, student_password, admission_date

3NF: There is no transitive dependency.

1. applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph, applicant_password, applicant_dob, applicant_address
2. student_id, student_email, student_password, admission_date

Table creation:

1. applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph, applicant_password, applicant_dob, applicant_address
2. student_id, student_email, student_password, admission_date, **applicant_id**

Belongs_To: (Many to One)

UNF:

(student_id, student_email, student_password, admission_date, dept_id, dept_name)

1NF: There is no multi valued attribute.

1. student_id, student_email, student_password, admission_date, dept_id, dept_name

2NF:

1. student_id, student_email, student_password, admission_date
2. dept_id, dept_name

3NF: There is no transitive dependency.

1. student_id, student_email, student_password, admission_date
2. dept_id, dept_name

Table creation:

1. student_id, student_email, student_password, admission_date, **dept_id**
2. dept_id, dept_name

Offers: (One to Many)

UNF:

(dept_id, dept_name, program_id, program_name, total_credit, per_credit_cost, program_duration)

1NF: There is no multi-valued attribute.

1. dept_id, dept_name, program_id, program_name, total_credit, per_credit_cost, program_duration

2NF:

1. dept_id, dept_name
2. program_id, program_name, total_credit, per_credit_cost, program_duration

3NF: There is no transitive dependency.

1. dept_id, dept_name
2. program_id, program_name, total_credit, per_credit_cost, program_duration

Table creation:

1. dept_id, dept_name
2. program_id, program_name, total_credit, per_credit_cost, program_duration, **dept_id**

Receives: (One to Many)

UNF:

(dept_id, dept_name, application_id, apply_date, status)

1NF: There is no multi valued attribute.

1. dept_id, dept_name, application_id, apply_date, status

2NF:

1. dept_id, dept_name
2. application_id, apply_date, status

3NF: There is no transitive dependency.

1. dept_id, dept_name
2. application_id, apply_date, status

Table creation:

1. dept_id, dept_name
2. application_id, apply_date, status, **dept_id**

Assigns: (One to Many)

UNF:

(dept_id, dept_name, teacher_id, teacher_name, teacher_ph, teacher_email, teacher_password)

1NF: teacher_ph is a multi-valued attribute.

1. dept_id, dept_name, teacher_id, teacher_name, teacher_ph, teacher_email, teacher_password

2NF:

1. dept_id, dept_name
2. teacher_id, teacher_name, teacher_ph, teacher_email, teacher_password

3NF: There is no transitive dependency.

1. dept_id, dept_name
2. teacher_id, teacher_name, teacher_ph, teacher_email, teacher_password

Table creation:

1. dept_id, dept_name
2. teacher_id, teacher_name, teacher_ph, teacher_email, teacher_password, **dept_id**

Enrolled In: (Many to One)

UNF:

(student_id, student_email, student_password, admission_date, program_id, program_name, total_credit, per_credit_cost, program_duration)

1NF: There is no multi-valued attribute.

1. student_id, student_email, student_password, admission_date, program_id, program_name, total_credit, per_credit_cost, program_duration

2NF:

1. student_id, student_email, student_password, admission_date
2. program_id, program_name, total_credit, per_credit_cost, program_duration

3NF: There is no transitive dependency

1. student_id, student_email, student_password, admission_date
2. program_id, program_name, total_credit, per_credit_cost, program_duration

Table creation:

1. student_id, student_email, student_password, admission_date, **program_id**
2. program_id, program_name, total_credit, per_credit_cost, program_duration

Temporary Tables:

1. admin_id, admin_name, admin_address, admin_ph, admin_email, admin_password
2. ~~admin_id, admin_name, admin_address, admin_ph, admin_email, admin_password~~
3. application_id, apply_date, status, **admin_id**
4. application_id, apply_date, status, **applicant_id**
5. application_id, apply_date, status, **dept_id**
6. ~~application_id, apply_date, status~~
7. ~~application_id, apply_date, status~~
8. evaluation_id, written_marks, viva_marks, final_status, **admin_id**

9. evaluation_id, written_marks, viva_marks, final_status, **teacher_id**
10. evaluation_id, written_marks, viva_marks, final_status, **application_id**
11. payment_id, payment_amount, payment_method, payment_type, payment_date, **application_id**
12. teacher_id, teacher_name, teacher_ph, teacher_email, teacher_password, **dept_id**
13. ~~teacher_id, teacher_name, teacher_ph, teacher_email, teacher_password~~
14. applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph, applicant_password, applicant_dob, applicant_address
15. ~~applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph, applicant_password, applicant_dob, applicant_address~~
16. student_id, student_email, student_password, admission_date, **applicant_id**
17. student_id, student_email, student_password, admission_date, **program_id**
18. student_id, student_email, student_password, admission_date, **dept_id**
19. dept_id, dept_name
20. ~~dept_id, dept_name~~
21. ~~dept_id, dept_name~~
22. ~~dept_id, dept_name~~
23. program_id, program_name, total_credit, per_credit_cost, program_duration, **dept_id**
24. ~~program_id, program_name, total_credit, per_credit_cost, program_duration~~

Final Tables:

1. admin_id, admin_name, admin_address, admin_ph1, admin_ph2, admin_email, admin_password
2. application_id, apply_date, status, **admin_id**, **applicant_id**, **dept_id**
3. evaluation_id, written_marks, viva_marks, final_status, **application_id**, **teacher_id**, **admin_id**
4. payment_id, payment_amount, payment_method, payment_type, payment_date, **application_id**

5. teacher_id, teacher_name, teacher_ph1, teacher_ph2, teacher_email, teacher_password, **dept_id**
6. applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph1, applicant_ph2, applicant_password, applicant_dob, applicant_address
7. student_id, student_email, student_password, admission_date, **applicant_id**, **dept_id**, **program_id**
8. dept_id, dept_name
9. program_id, program_name, total_credit, per_credit_cost, program_duration, **dept_id**

Name of Tables:

1. **Admin:** (admin_id, admin_name, admin_address, admin_ph1, admin_ph2, admin_email, admin_password)
2. **Applicant:** (applicant_id, applicant_name, fathers_name, mothers_name, nationality, religion, college_name, hsc_result, school_name, ssc_result, applicant_email, applicant_ph1, applicant_ph2, applicant_password, applicant_dob, applicant_address)
3. **Department:** (dept_id, dept_name)
4. **Application:** (application_id, apply_date, status, **admin_id**, **applicant_id**, **dept_id**)
5. **Program:** (program_id, program_name, total_credit, per_credit_cost, program_duration, **dept_id**)
6. **Teacher:** (teacher_id, teacher_name, teacher_ph1, teacher_ph2, teacher_email, teacher_password, **dept_id**)
7. **Student:** (student_id, student_email, student_password, admission_date, **applicant_id**, **dept_id**, **program_id**)
8. **Payment:** (payment_id, payment_amount, payment_method, payment_type, payment_date, **application_id**)
9. **Evaluation:** (evaluation_id, written_marks, viva_marks, final_status, **application_id**, **teacher_id**, **admin_id**)

Schema Diagram:

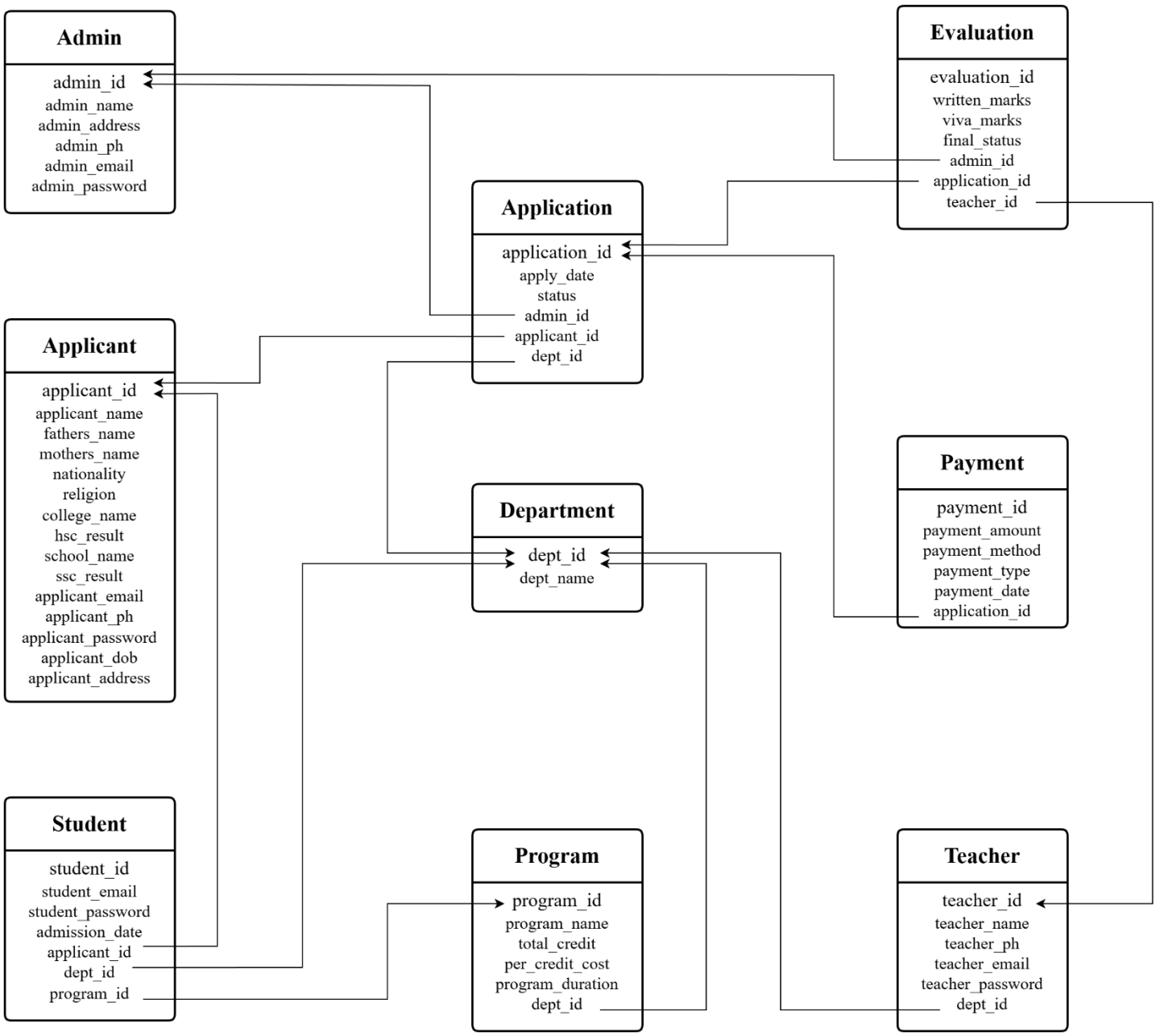
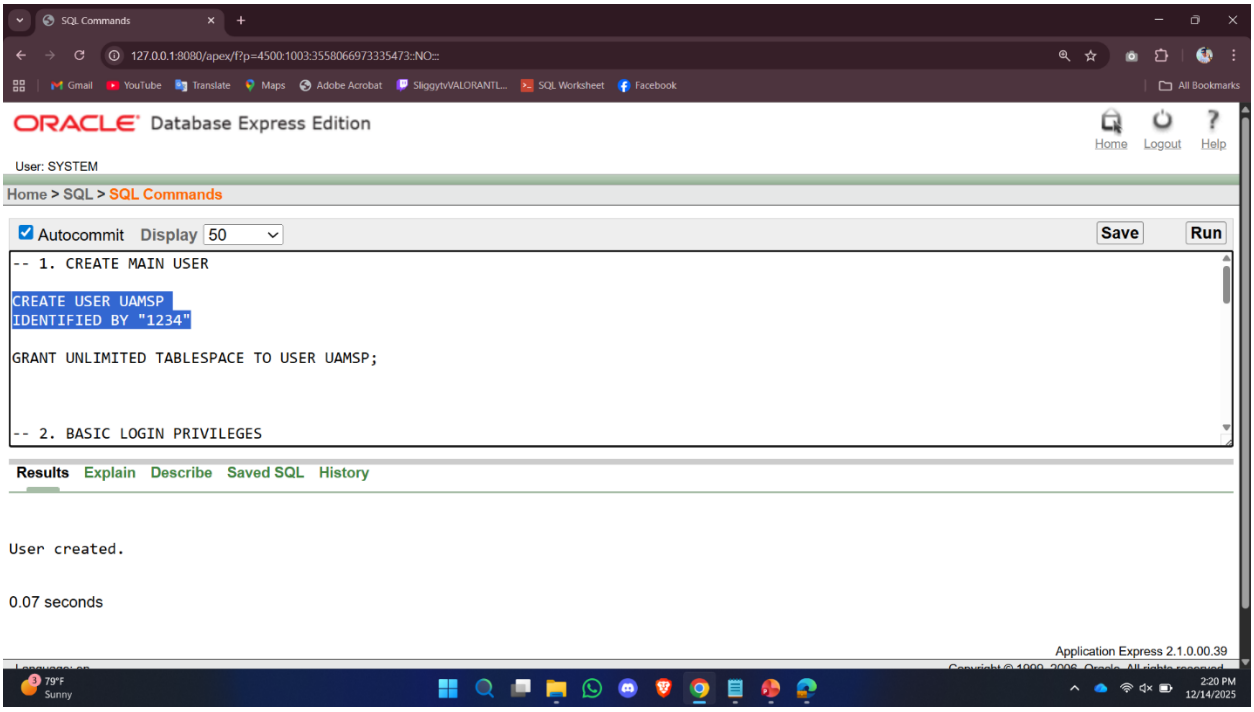


Table Creation Using SQL:

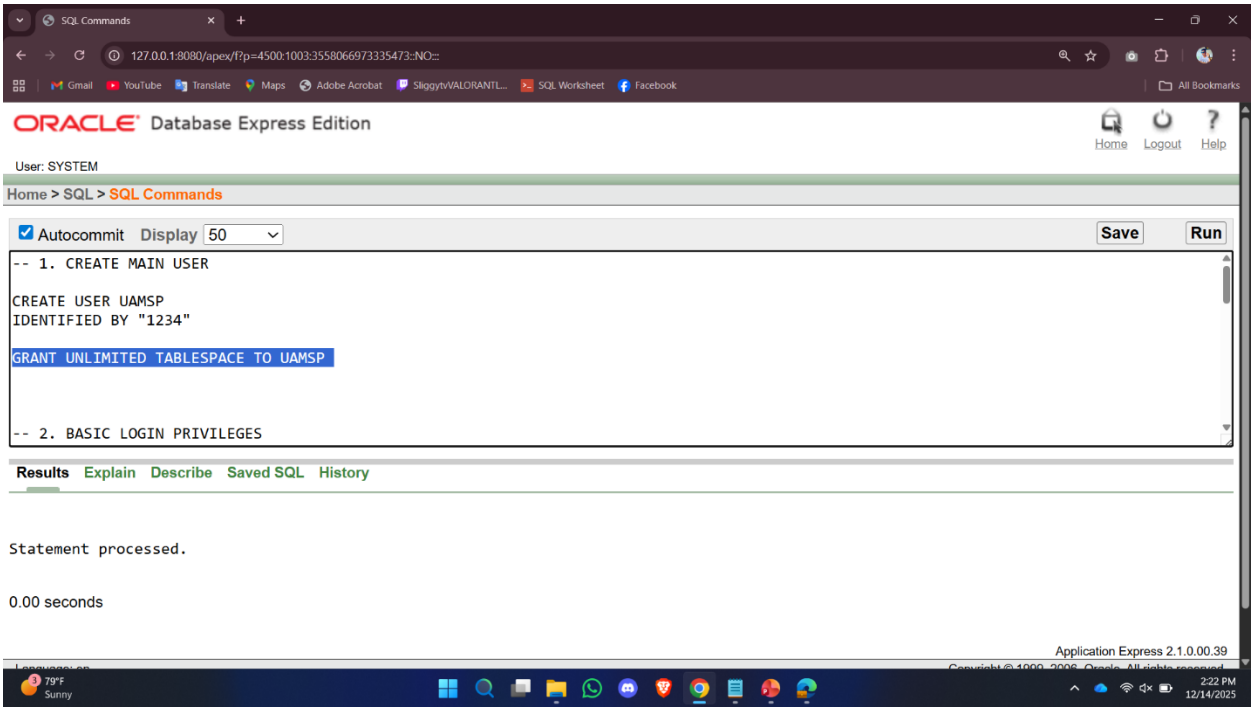
Create User:

CREATE USER UAMSP

IDENTIFIED BY "1234"

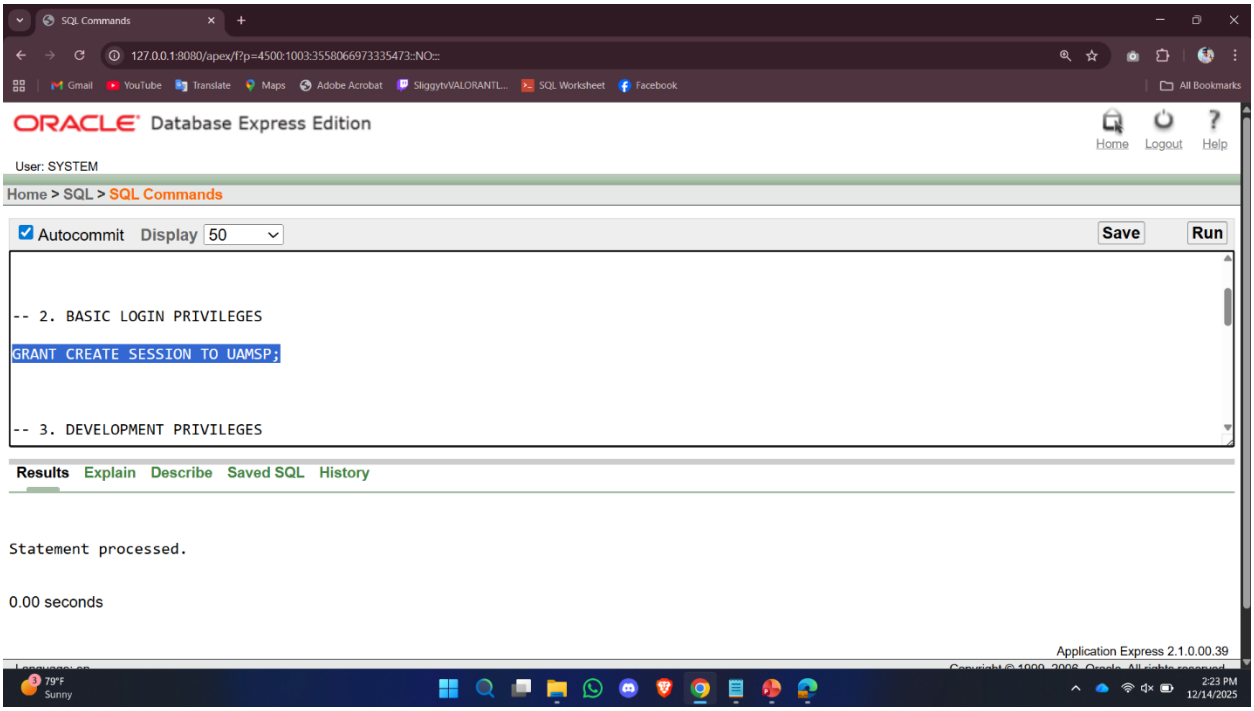


GRANT UNLIMITED TABLESPACE TO UAMSP;



Basic login Privileges:

GRANT CREATE SESSION TO UAMSP;



Privileges:

GRANT

CREATE TABLE,

CREATE VIEW,

CREATE SEQUENCE,

CREATE PROCEDURE,

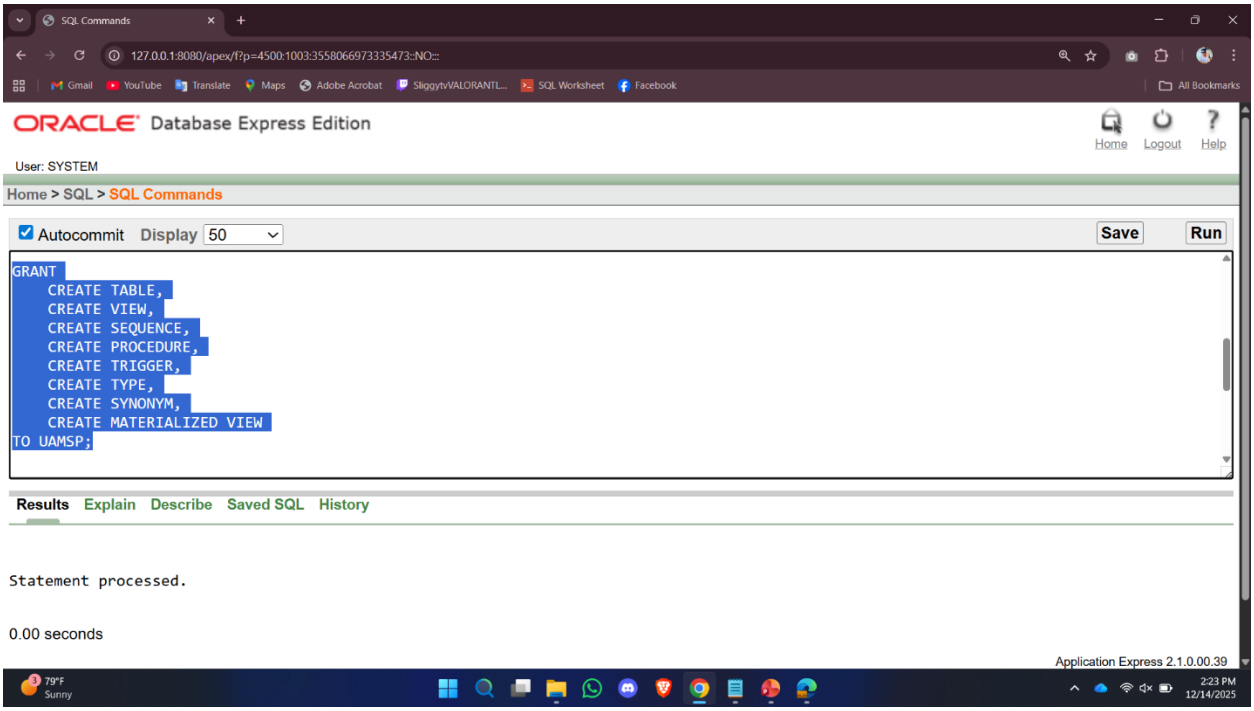
CREATE TRIGGER,

CREATE TYPE,

CREATE SYNONYM,

CREATE MATERIALIZED VIEW

TO UAMSP;

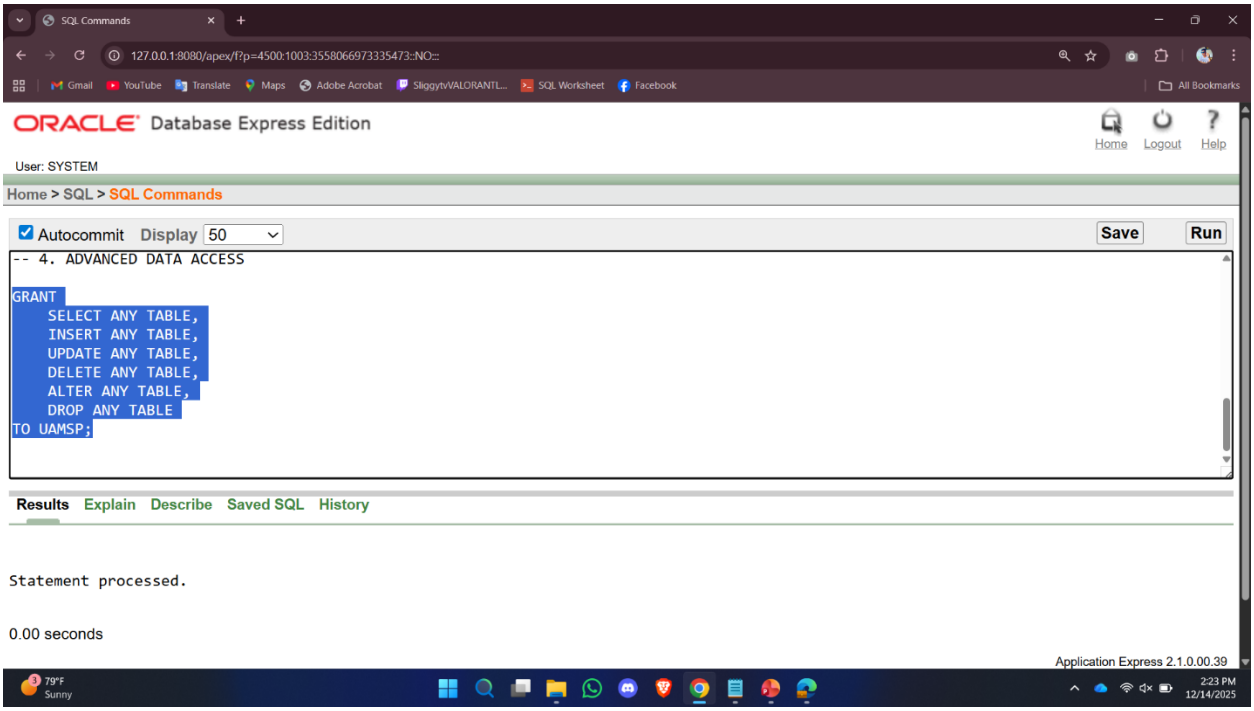


GRANT

SELECT ANY TABLE, INSERT ANY TABLE, UPDATE ANY TABLE, DELETE ANY TABLE, ALTER ANY TABLE,

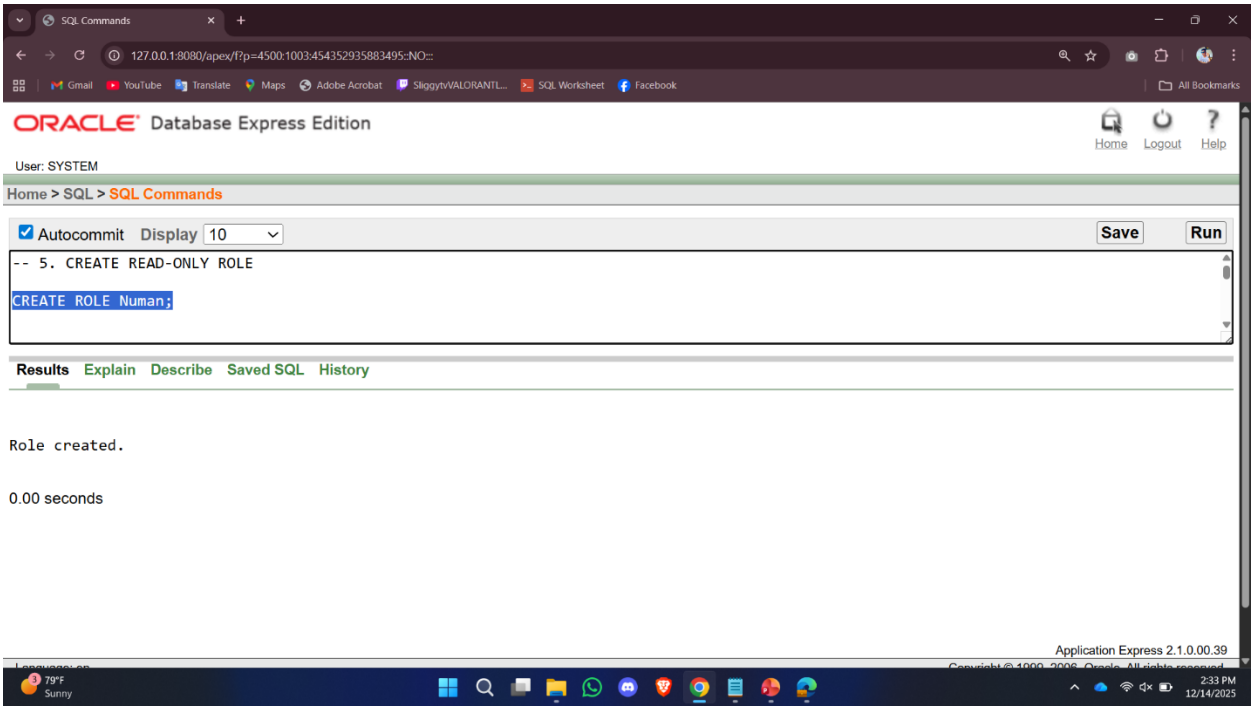
DROP ANY TABLE

TO UAMSP;



Create Role:

CREATE ROLE Numan;



Grant Select PRIVILEGES (UAMSP TABLES):

GRANT SELECT ON UAMSP.Admin TO Numan;

GRANT SELECT ON UAMSP.Applicant TO Numan;

GRANT SELECT ON UAMSP.Department TO Numan;

GRANT SELECT ON UAMSP.Program TO Numan;

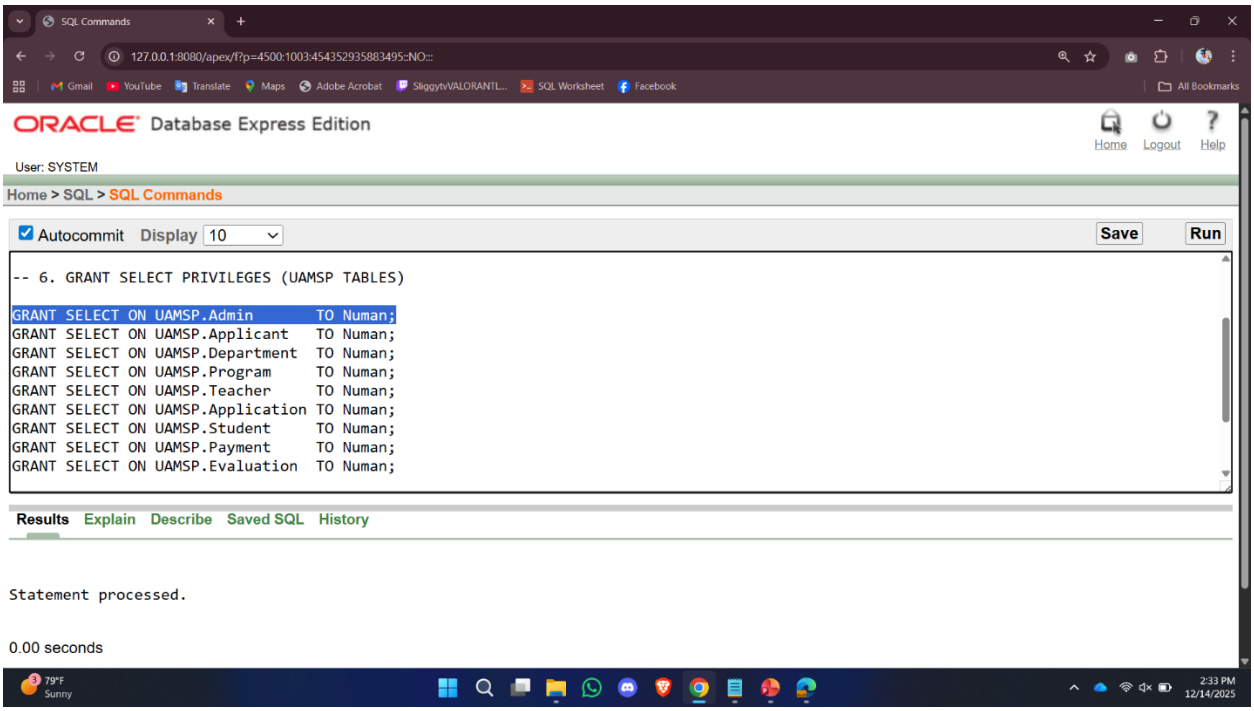
GRANT SELECT ON UAMSP.Teacher TO Numan;

GRANT SELECT ON UAMSP.Application TO Numan;

GRANT SELECT ON UAMSP.Student TO Numan;

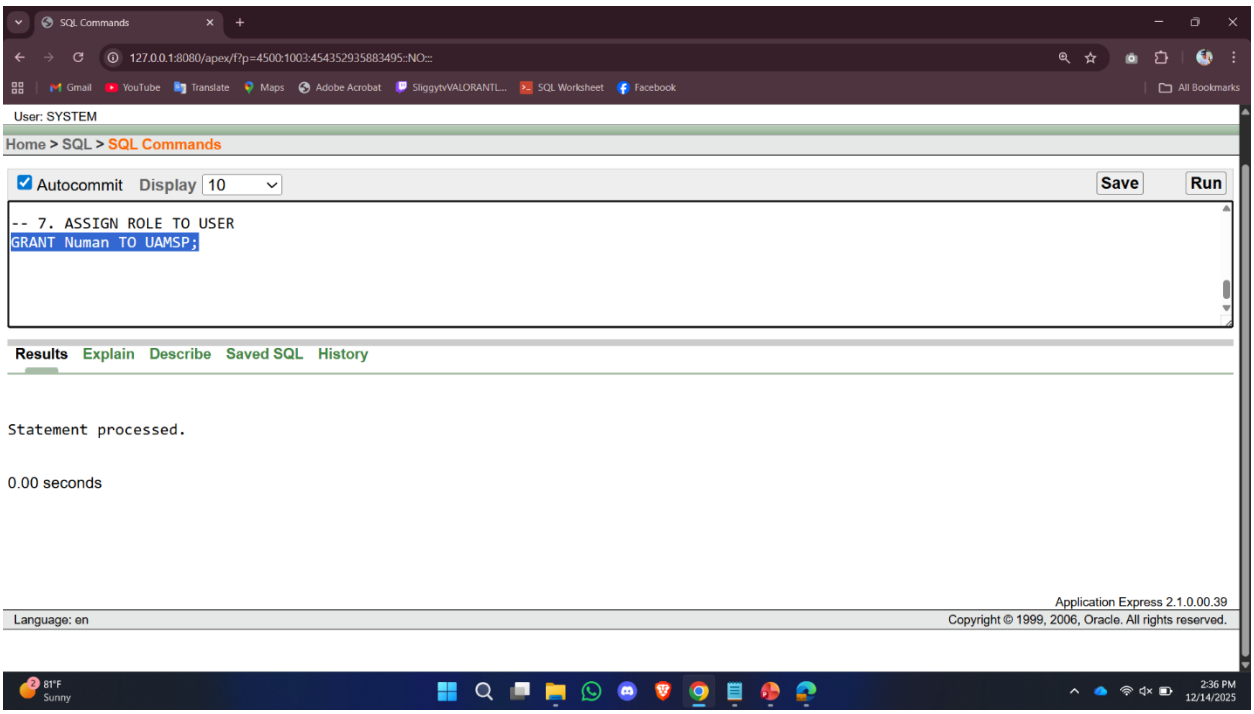
GRANT SELECT ON UAMSP.Payment TO Numan;

GRANT SELECT ON UAMSP.Evaluation TO Numan;



Assign Role TO User:

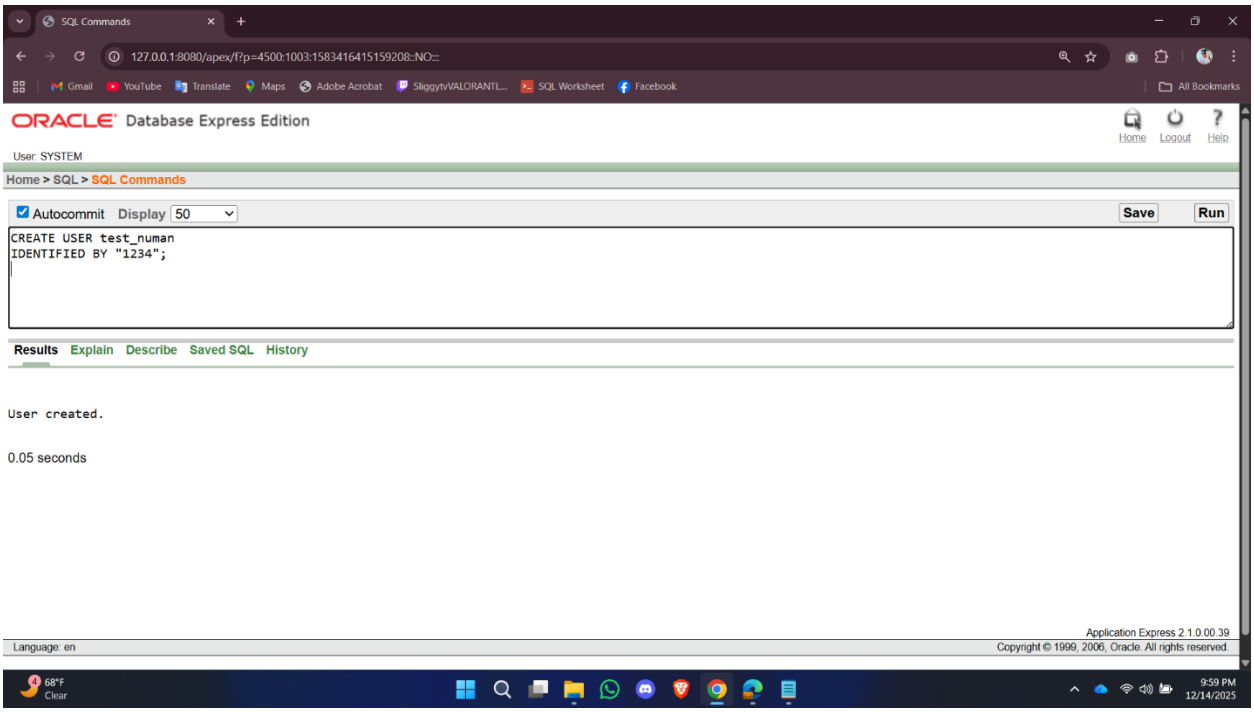
GRANT Numan TO UAMSP;



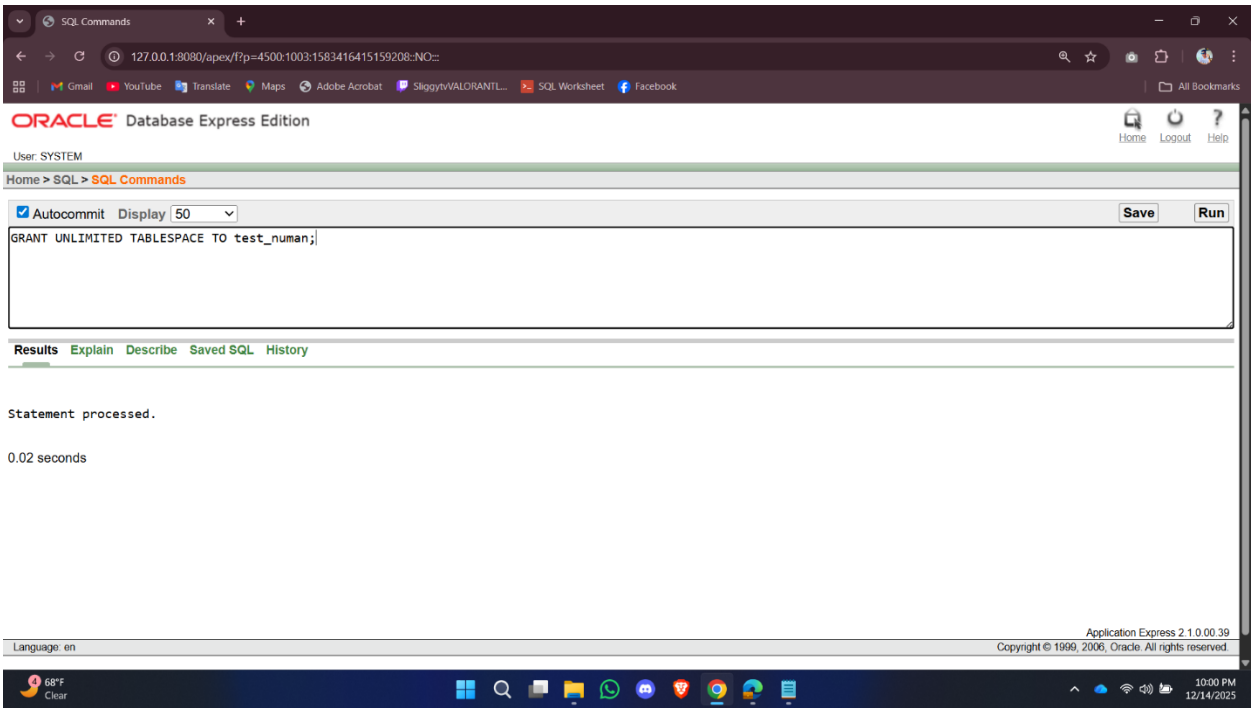
Check The Role:

CREATE USER test_numan

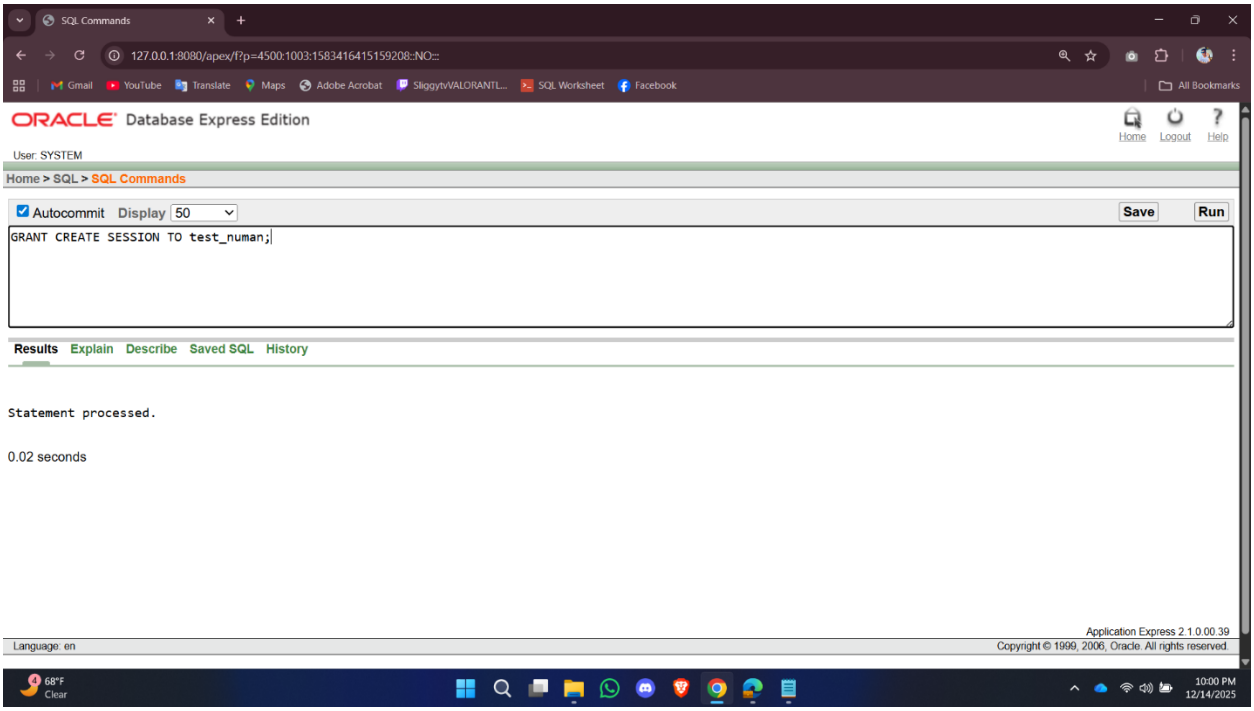
IDENTIFIED BY "1234";



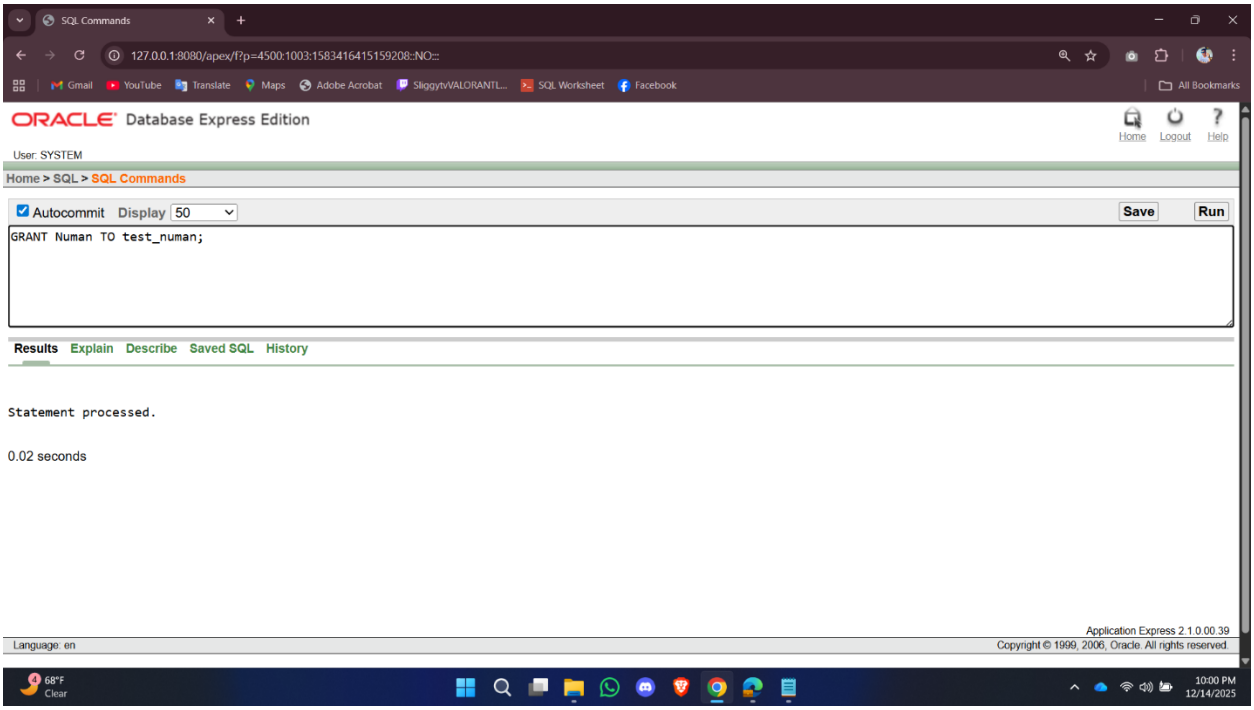
GRANT UNLIMITED TABLESPACE TO test_numan;



GRANT CREATE SESSION TO test_numan;



GRANT Numan TO test_numan;



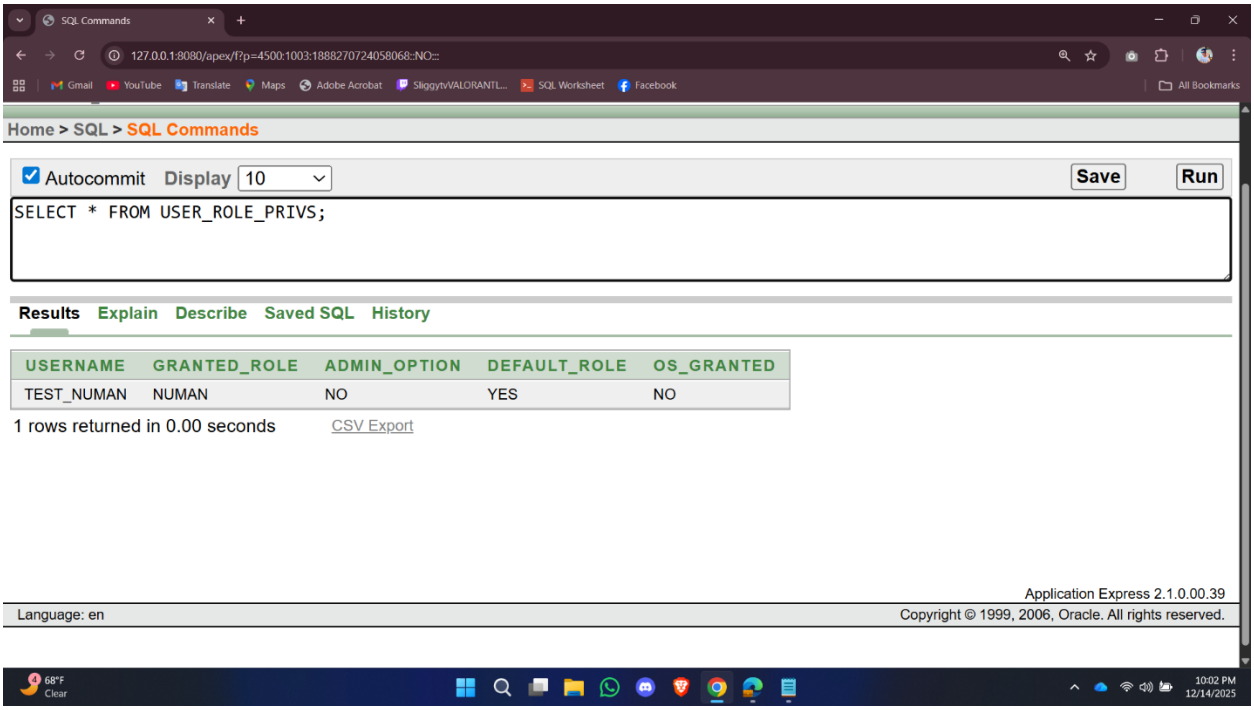
Then login as

Username: test_numan

Password: 1234

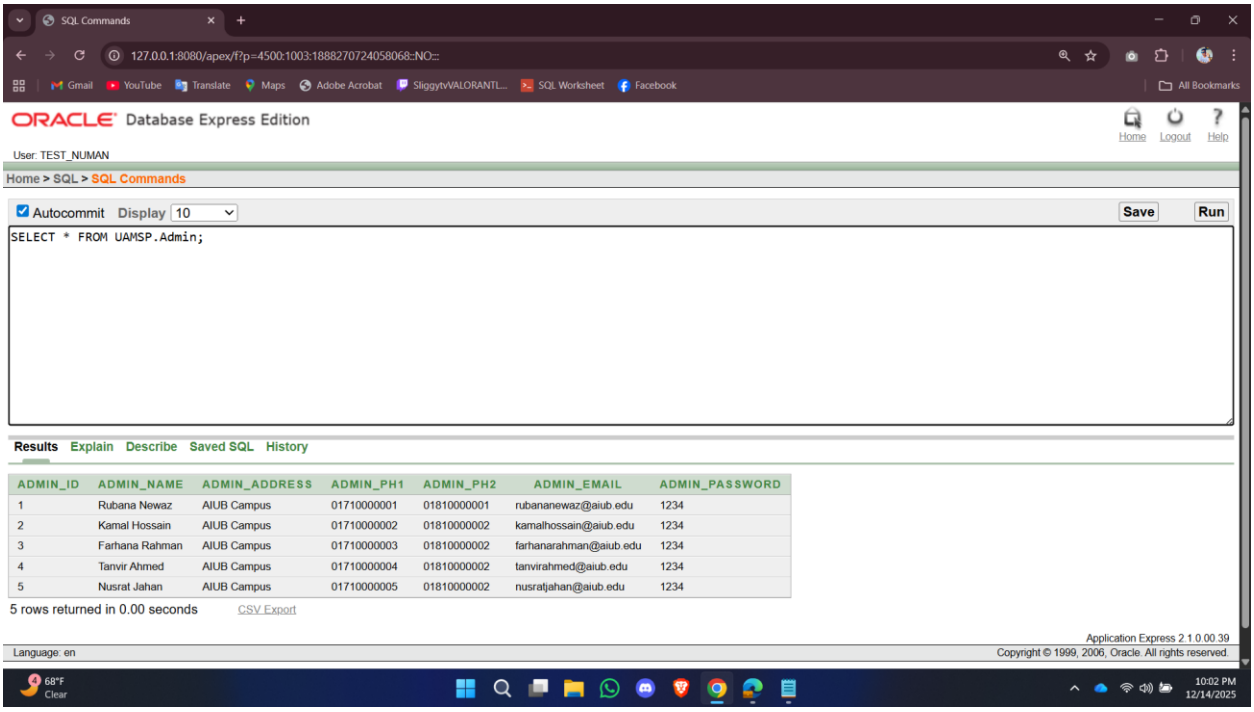
Check That the Role Is Enabled:

SELECT * FROM USER_ROLE_PRIVS;

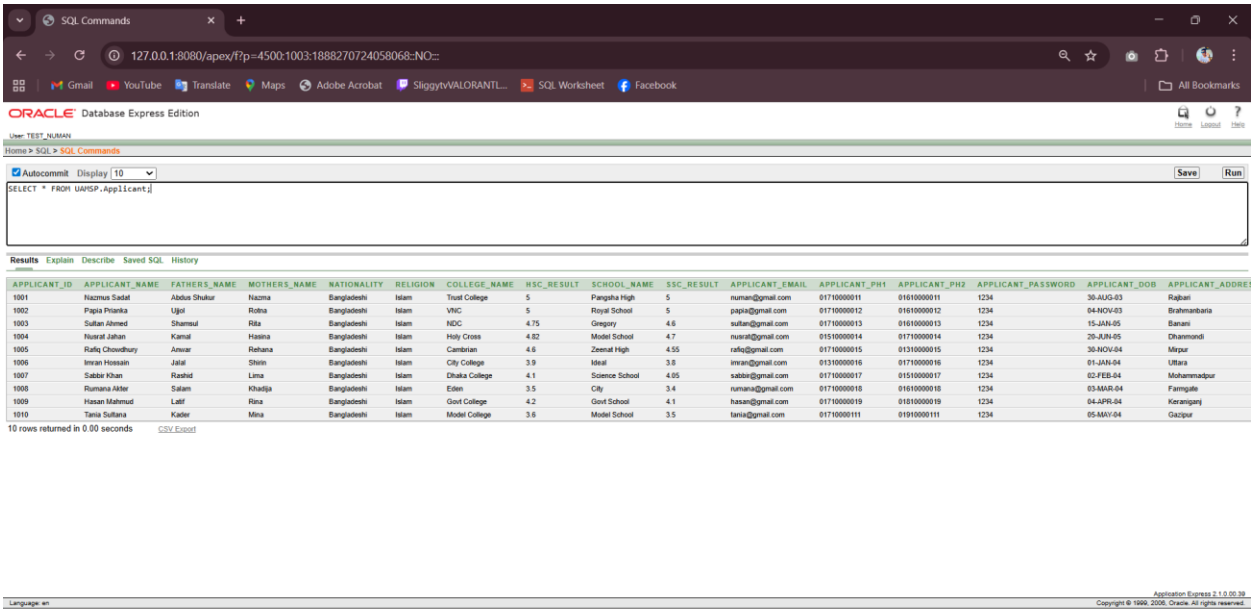


Test SELECT Access:

SELECT * FROM UAMSP.Admin;



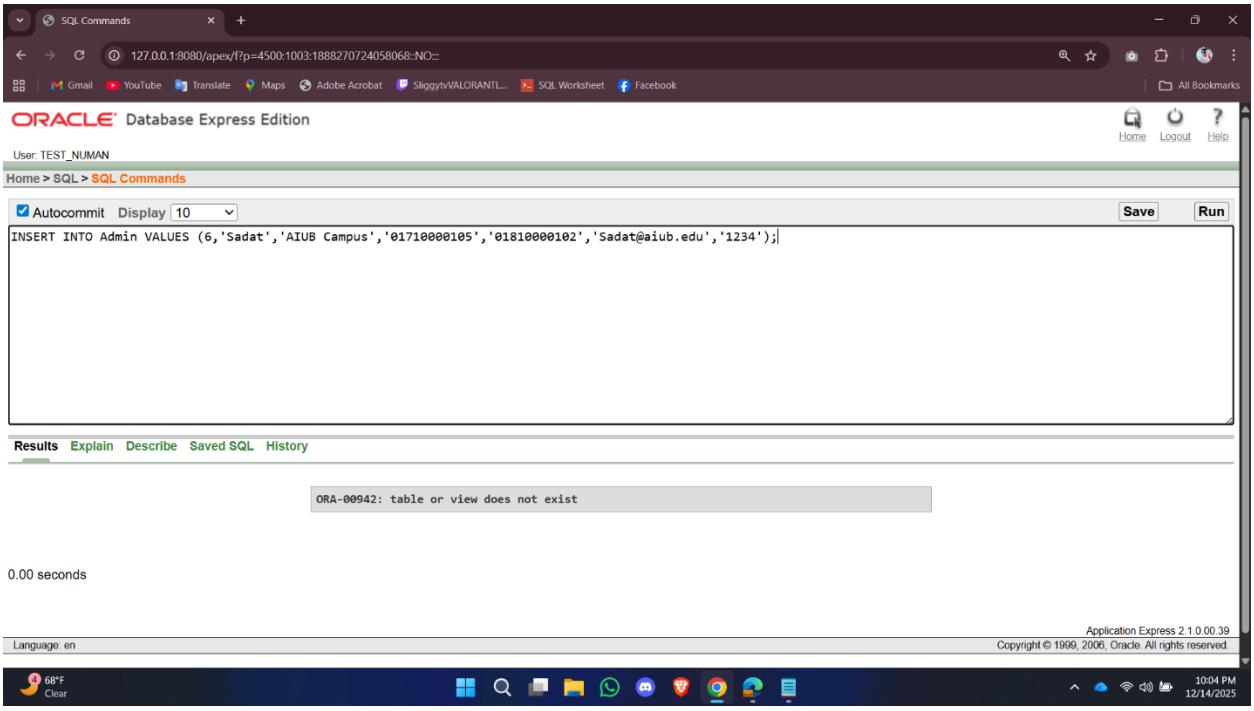
SELECT * FROM UAMSP.Applicant;





Test INSERT:

INSERT INTO Admin VALUES (6,'Sadat','AIUB Campus','01710000105','01810000102','Sadat@aiub.edu','1234');



SEQUENCES:

```
CREATE SEQUENCE admin_seq START WITH 1 INCREMENT BY 1 NOCACHE
NOCYCLE;

CREATE SEQUENCE applicant_seq START WITH 1001 INCREMENT BY 1 NOCACHE
NOCYCLE;

CREATE SEQUENCE dept_seq START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;

CREATE SEQUENCE application_seq START WITH 701 INCREMENT BY 1 NOCACHE
NOCYCLE;

CREATE SEQUENCE program_seq START WITH 2001 INCREMENT BY 1 NOCACHE
NOCYCLE;

CREATE SEQUENCE teacher_seq START WITH 1001 INCREMENT BY 1 NOCACHE
NOCYCLE;

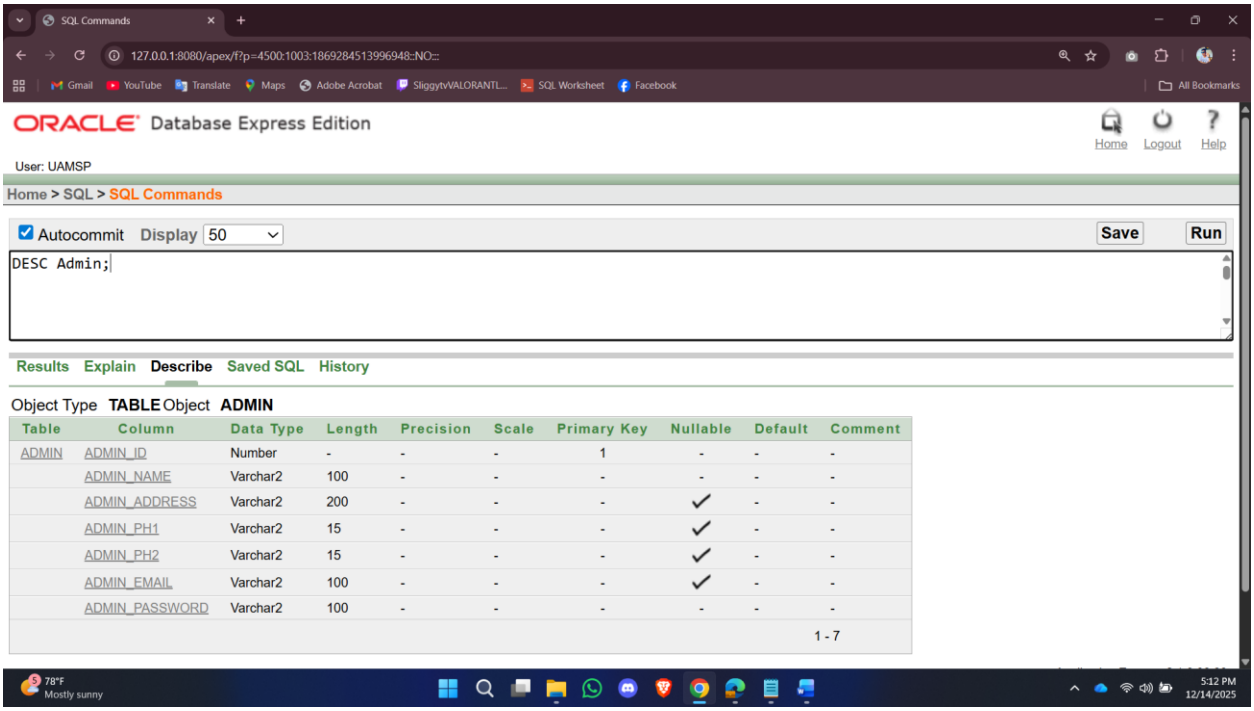
CREATE SEQUENCE student_seq START WITH 3001 INCREMENT BY 1 NOCACHE
NOCYCLE;

CREATE SEQUENCE payment_seq START WITH 9001 INCREMENT BY 1 NOCACHE
NOCYCLE;

CREATE SEQUENCE evaluation_seq START WITH 3301 INCREMENT BY 1 NOCACHE
NOCYCLE;
```

Admin Table:

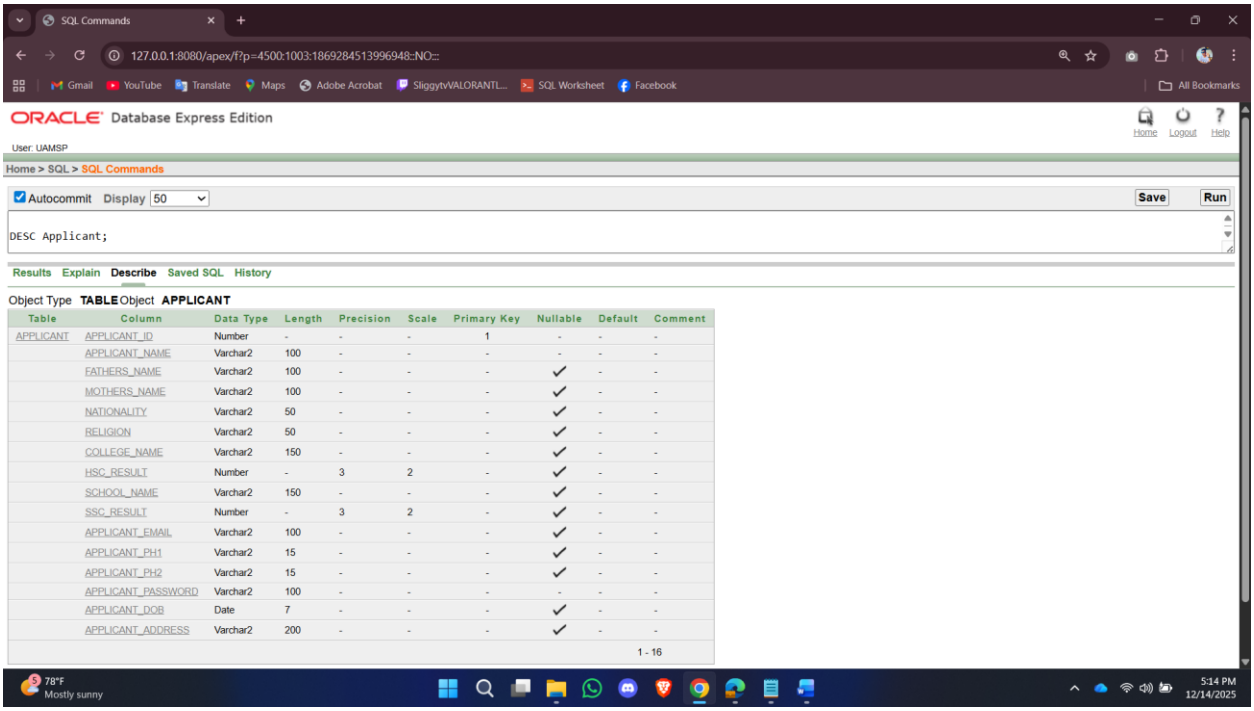
```
CREATE TABLE Admin (
    admin_id NUMBER PRIMARY KEY,
    admin_name VARCHAR2(100) NOT NULL,
    admin_address VARCHAR2(200),
    admin_ph1 VARCHAR2(15),
    admin_ph2 VARCHAR2(15),
    admin_email VARCHAR2(100) UNIQUE,
    admin_password VARCHAR2(100) NOT NULL
);
DESC Admin;
```



Applicant Table:

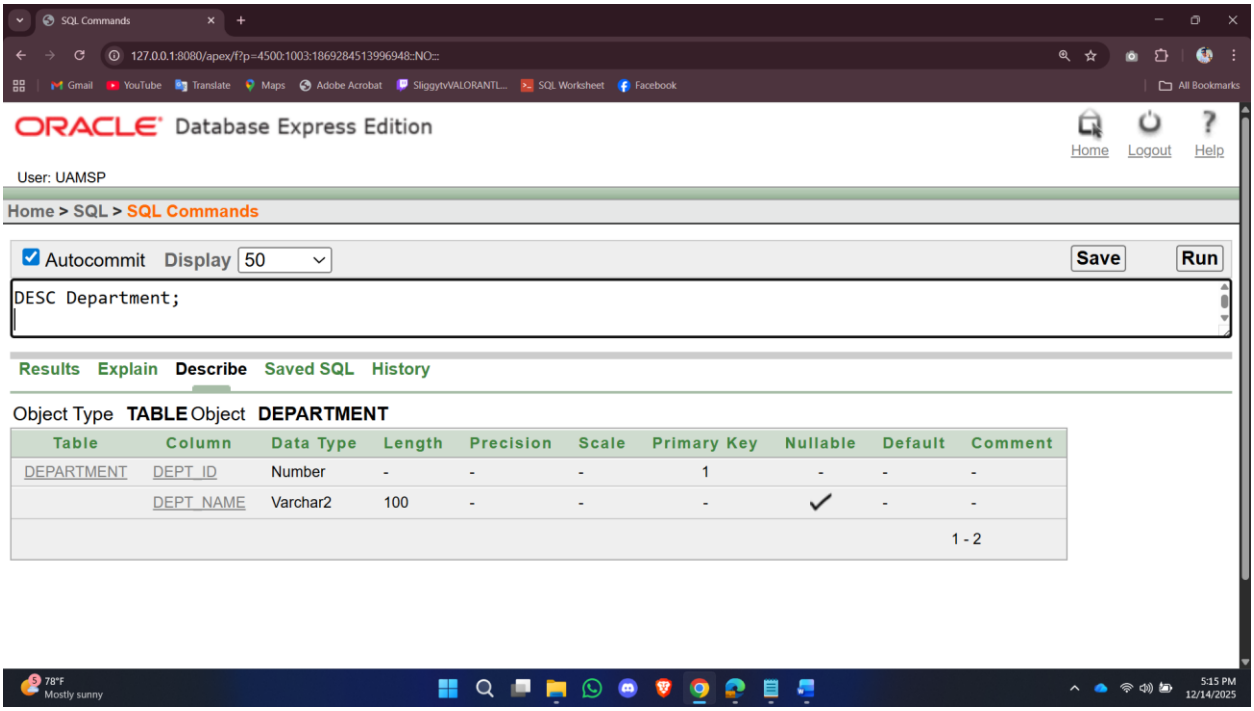
```
CREATE TABLE Applicant (
    applicant_id NUMBER PRIMARY KEY,
    applicant_name VARCHAR2(100) NOT NULL,
    fathers_name VARCHAR2(100),
    mothers_name VARCHAR2(100),
    nationality VARCHAR2(50),
    religion VARCHAR2(50),
    college_name VARCHAR2(150),
    hsc_result NUMBER(3,2),
    school_name VARCHAR2(150),
    ssc_result NUMBER(3,2),
    applicant_email VARCHAR2(100) UNIQUE,
    applicant_ph1 VARCHAR2(15),
    applicant_ph2 VARCHAR2(15),
    applicant_password VARCHAR2(100) NOT NULL,
```

```
    applicant_dob DATE,
    applicant_address VARCHAR2(200)
);
DESC Applicant;
```



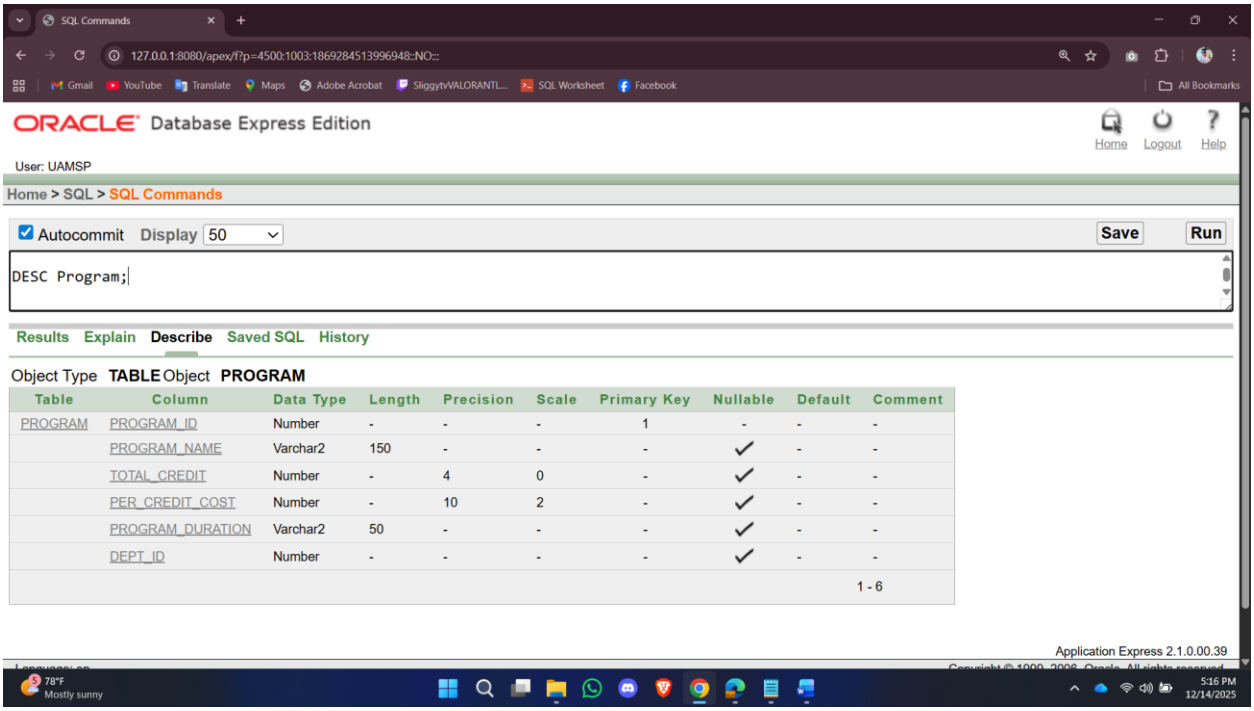
Department Table:

```
CREATE TABLE Department (
    dept_id   NUMBER PRIMARY KEY,
    dept_name VARCHAR2(100) UNIQUE
);
DESC Department;
```



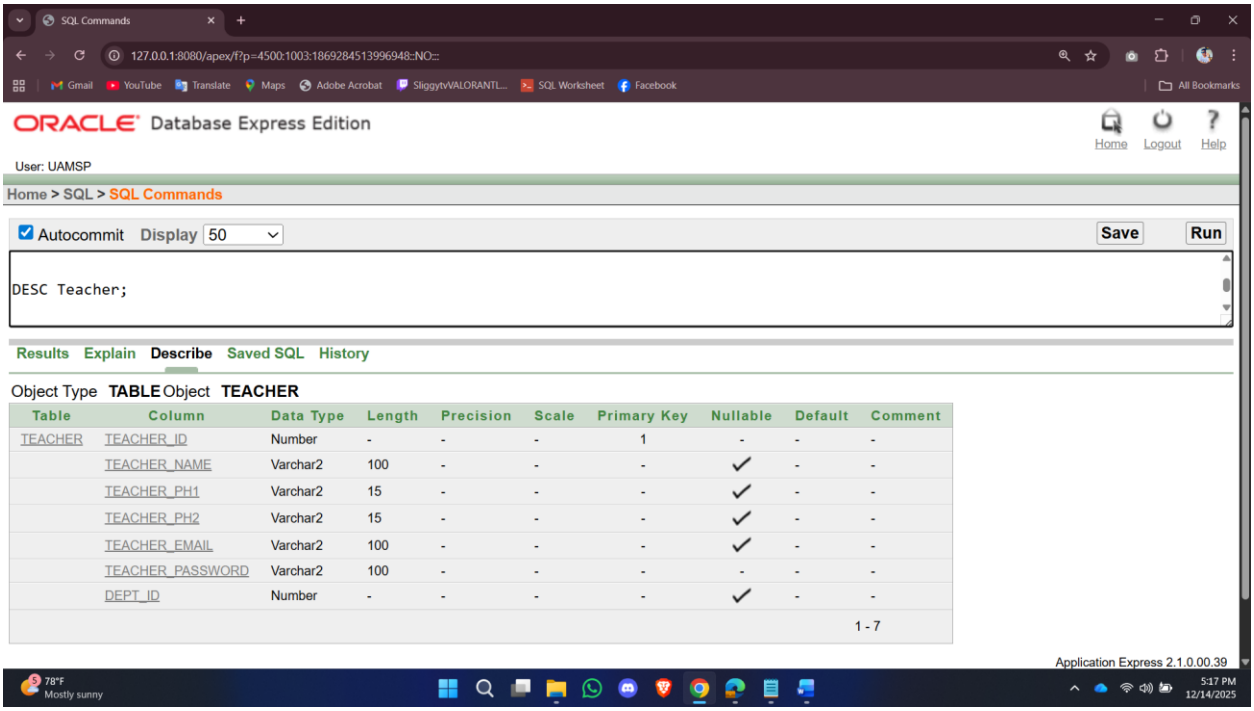
Program Table:

```
CREATE TABLE Program (
    program_id NUMBER PRIMARY KEY,
    program_name VARCHAR2(150),
    total_credit NUMBER(4),
    per_credit_cost NUMBER(10,2),
    program_duration VARCHAR2(50),
    dept_id NUMBER,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
DESC Program;
```



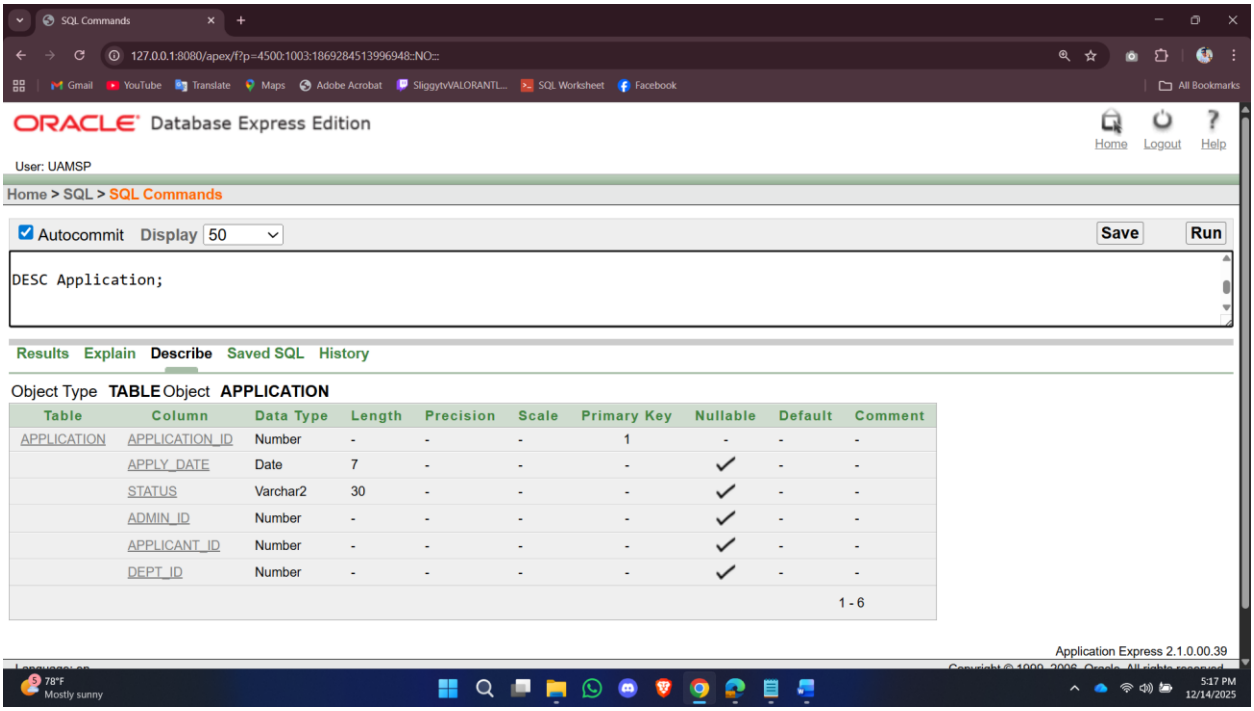
Teacher Table:

```
CREATE TABLE Teacher (
    teacher_id NUMBER PRIMARY KEY,
    teacher_name VARCHAR2(100),
    teacher_ph1 VARCHAR2(15),
    teacher_ph2 VARCHAR2(15),
    teacher_email VARCHAR2(100) UNIQUE,
    teacher_password VARCHAR2(100) NOT NULL,
    dept_id NUMBER,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
DESC Teacher;
```



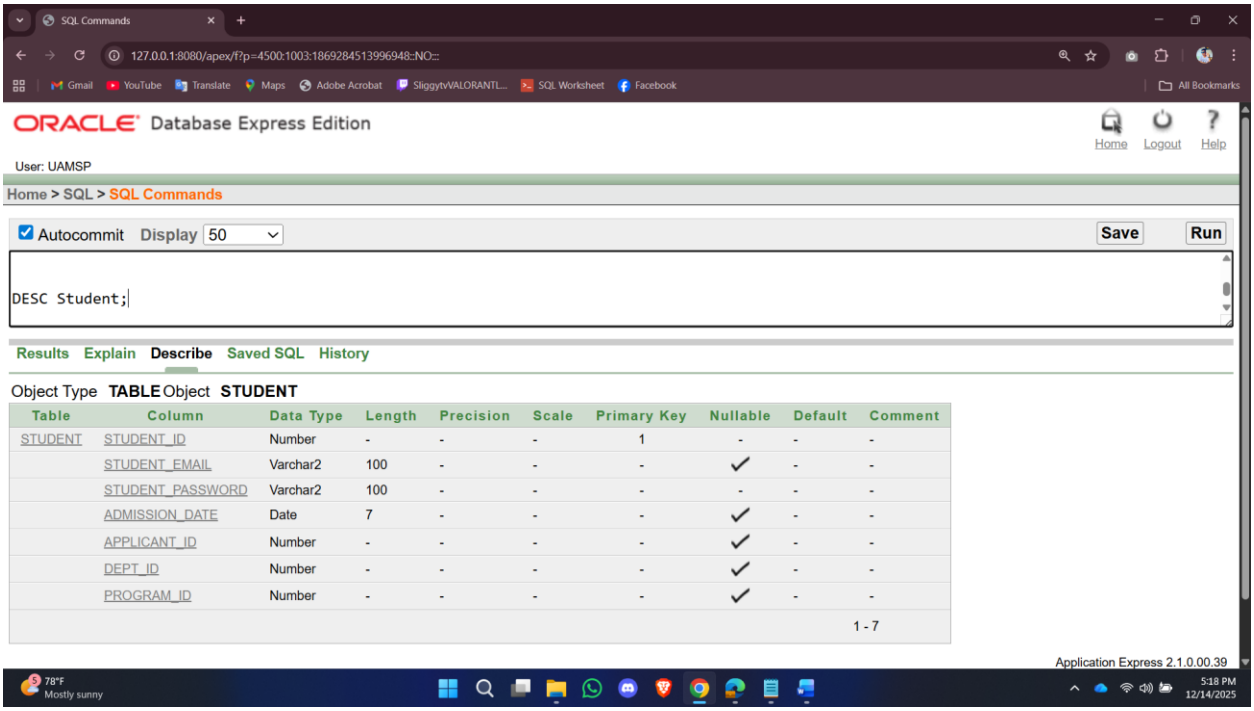
Application Table:

```
CREATE TABLE Application (
    application_id NUMBER PRIMARY KEY,
    apply_date DATE,
    status VARCHAR2(30),
    admin_id NUMBER,
    applicant_id NUMBER,
    dept_id NUMBER,
    FOREIGN KEY (admin_id) REFERENCES Admin(admin_id),
    FOREIGN KEY (applicant_id) REFERENCES Applicant(applicant_id),
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
DESC Application;
```



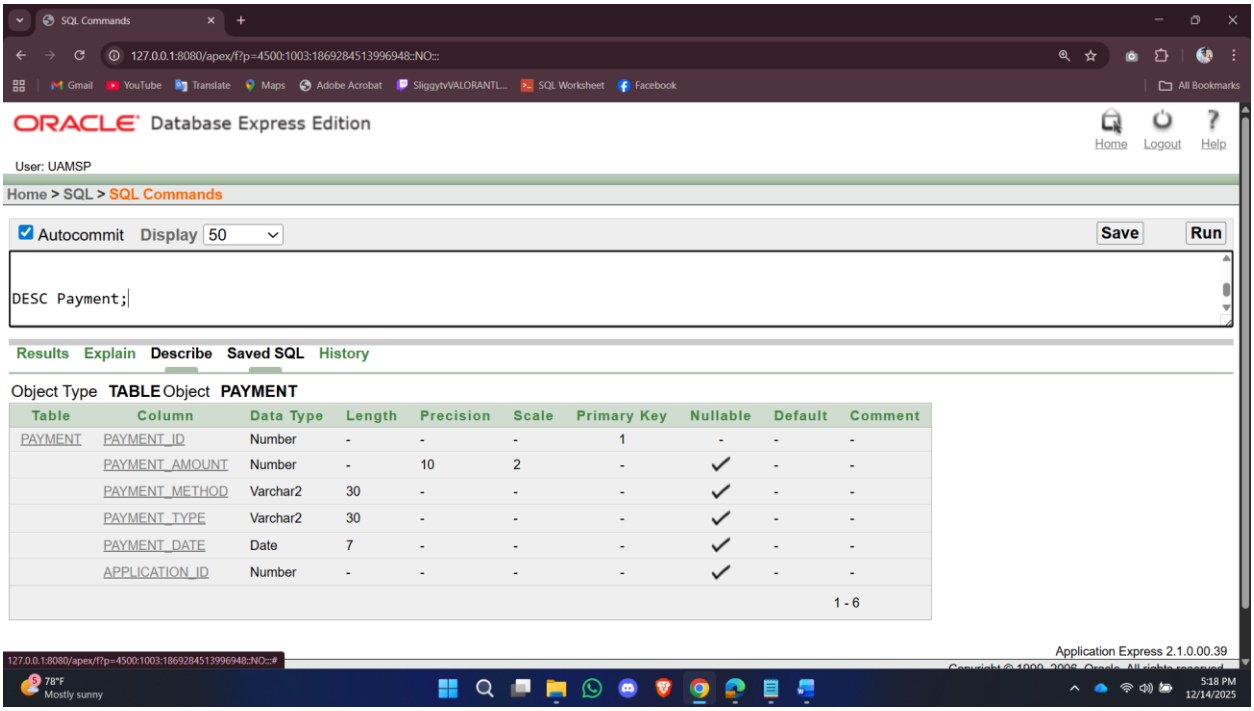
Student Table:

```
CREATE TABLE Student (
    student_id NUMBER PRIMARY KEY,
    student_email VARCHAR2(100) UNIQUE,
    student_password VARCHAR2(100) NOT NULL,
    admission_date DATE,
    applicant_id NUMBER,
    dept_id NUMBER,
    program_id NUMBER,
    FOREIGN KEY (applicant_id) REFERENCES Applicant(applicant_id),
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id),
    FOREIGN KEY (program_id) REFERENCES Program(program_id)
);
DESC Student;
```



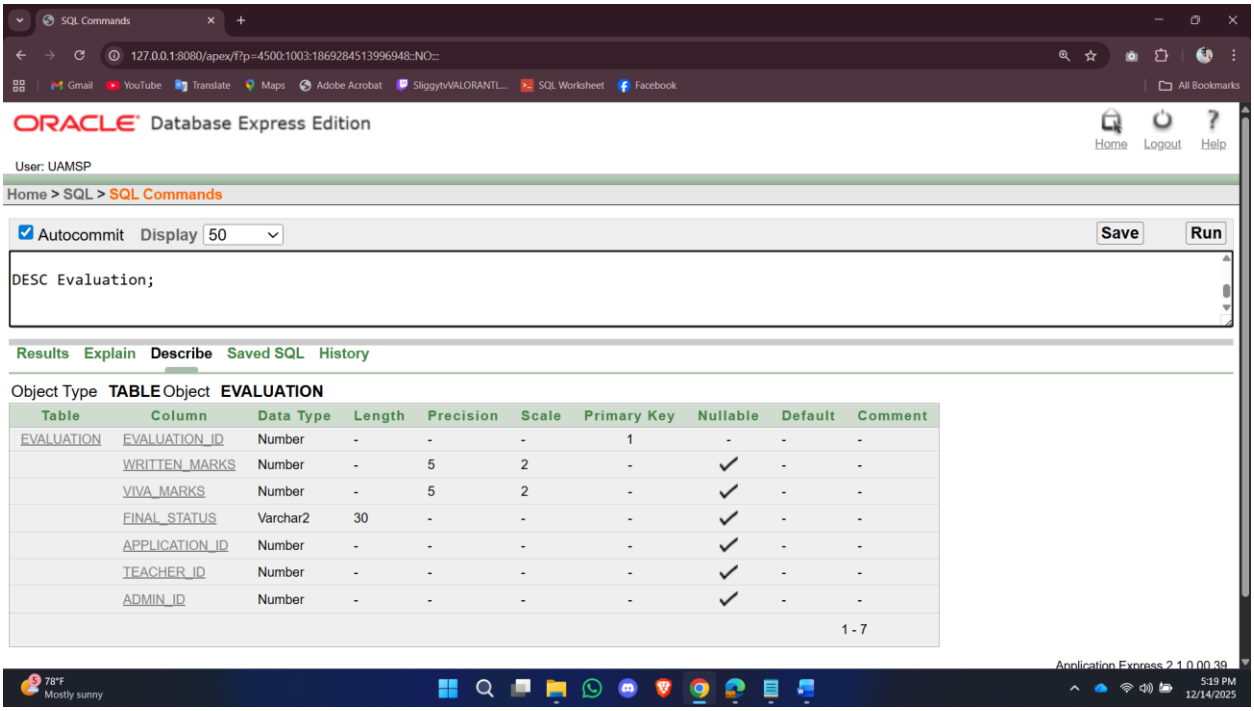
Payment Table:

```
CREATE TABLE Payment (
    payment_id NUMBER PRIMARY KEY,
    payment_amount NUMBER(10,2),
    payment_method VARCHAR2(30),
    payment_type VARCHAR2(30),
    payment_date DATE,
    application_id NUMBER,
    FOREIGN KEY (application_id) REFERENCES Application(application_id)
);
DESC Payment;
```



Evaluation Table:

```
CREATE TABLE Evaluation (
    evaluation_id NUMBER PRIMARY KEY,
    written_marks NUMBER(5,2),
    viva_marks NUMBER(5,2),
    final_status VARCHAR2(30),
    application_id NUMBER,
    teacher_id NUMBER,
    admin_id NUMBER,
    FOREIGN KEY (application_id) REFERENCES Application(application_id),
    FOREIGN KEY (teacher_id) REFERENCES Teacher(teacher_id),
    FOREIGN KEY (admin_id) REFERENCES Admin(admin_id)
);
DESC Evaluation;
```



Data Insertion Using SQL:

ADMIN:

INSERT INTO Admin VALUES (1,'Rubana Newaz','AIUB Campus','01710000001','01810000001','rubananewaz@aiub.edu','1234');

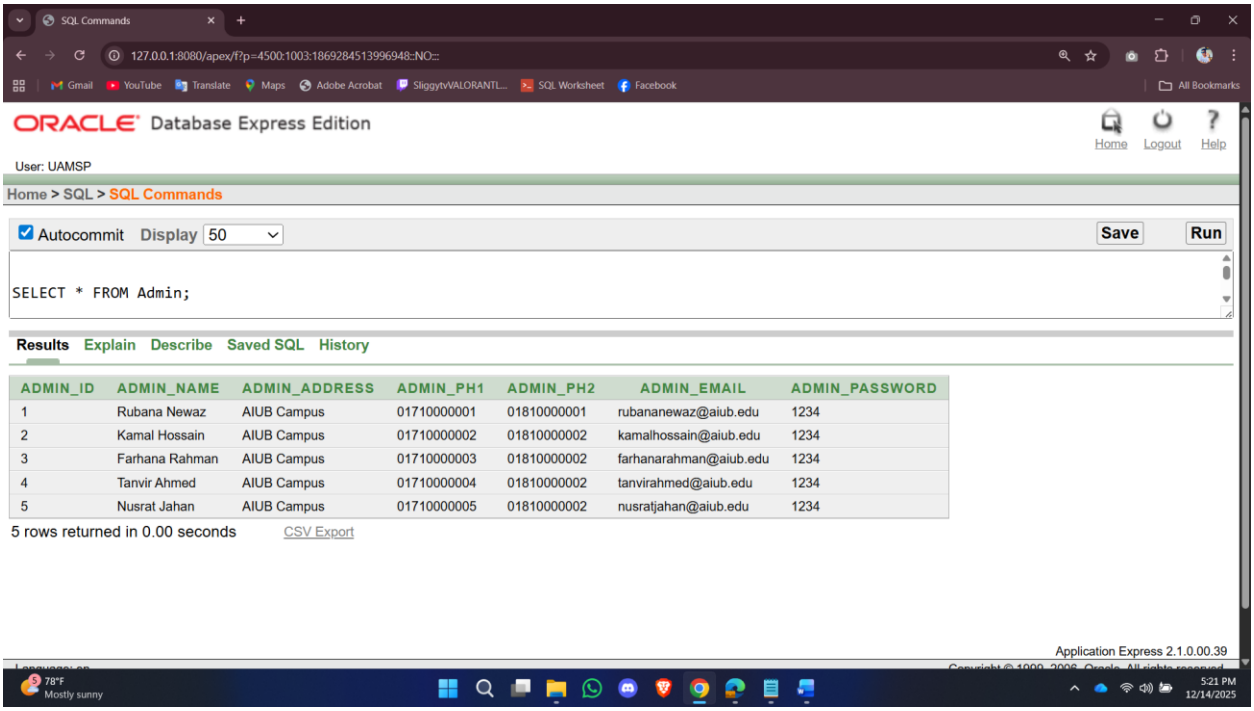
INSERT INTO Admin VALUES (2,'Kamal Hossain','AIUB Campus','01710000002','01810000002','kamalhossain@aiub.edu','1234');

INSERT INTO Admin VALUES (3,'Farhana Rahman','AIUB Campus','01710000003','01810000002','farhanarahman@aiub.edu','1234');

INSERT INTO Admin VALUES (4,'Tanvir Ahmed','AIUB Campus','01710000004','01810000002','tanvirahmed@aiub.edu','1234');

INSERT INTO Admin VALUES (5,'Nusrat Jahan','AIUB Campus','01710000005','01810000002','nusratjahan@aiub.edu','1234');

SELECT * FROM Admin;



DEPARTMENT:

INSERT INTO Department VALUES (1,'Computer Science');

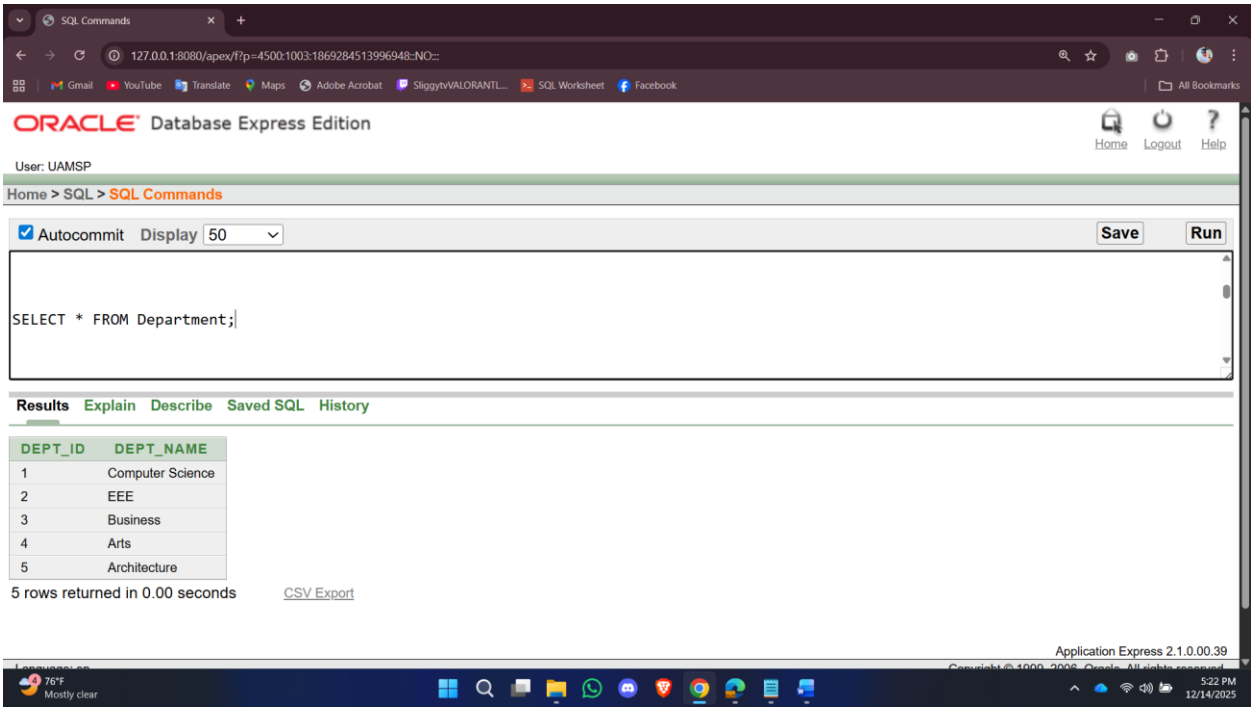
INSERT INTO Department VALUES (2,'EEE');

INSERT INTO Department VALUES (3,'Business');

INSERT INTO Department VALUES (4,'Arts');

INSERT INTO Department VALUES (5,'Architecture');

SELECT * FROM Department;



PROGRAM:

INSERT INTO Program VALUES (2001,'BSc in CSE',148,6500,'4 Years',1);

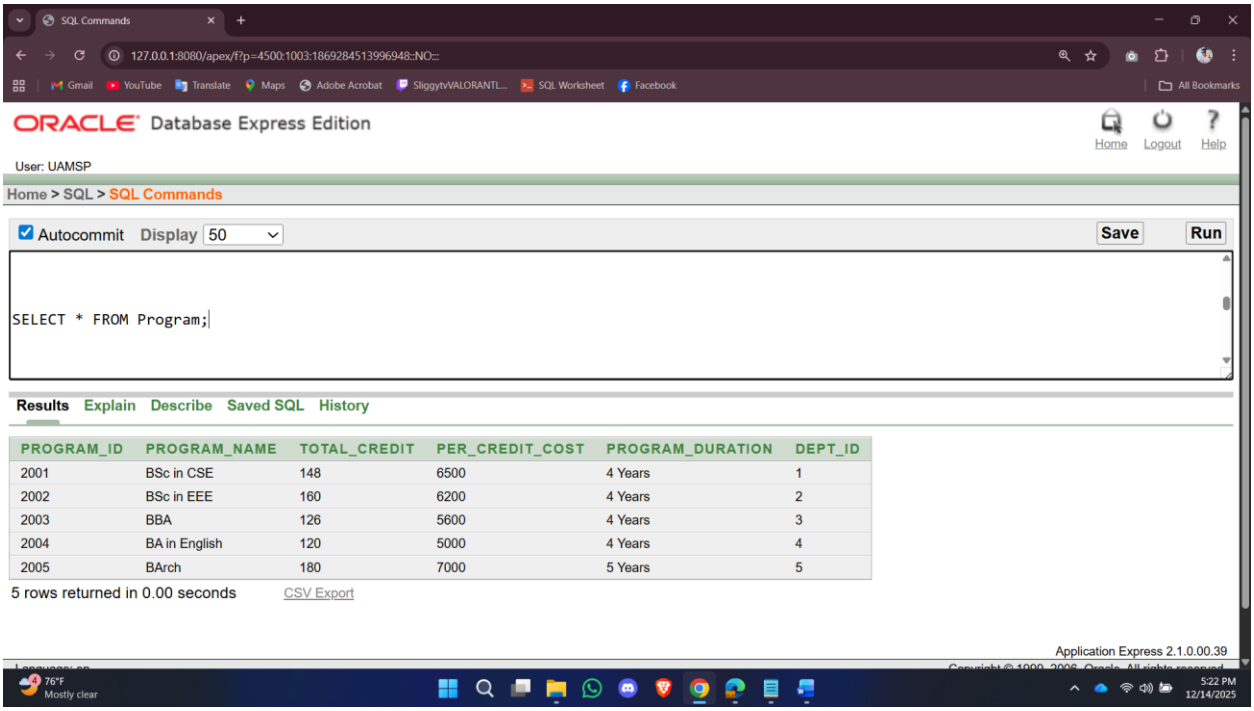
INSERT INTO Program VALUES (2002,'BSc in EEE',160,6200,'4 Years',2);

INSERT INTO Program VALUES (2003,'BBA',126,5600,'4 Years',3);

INSERT INTO Program VALUES (2004,'BA in English',120,5000,'4 Years',4);

INSERT INTO Program VALUES (2005,'BArch',180,7000,'5 Years',5);

SELECT * FROM Program;



TEACHER:

INSERT INTO Teacher VALUES (1001,'Juena Ahmed','01760000001','01760000001','juena@aiub.edu','1234',1);

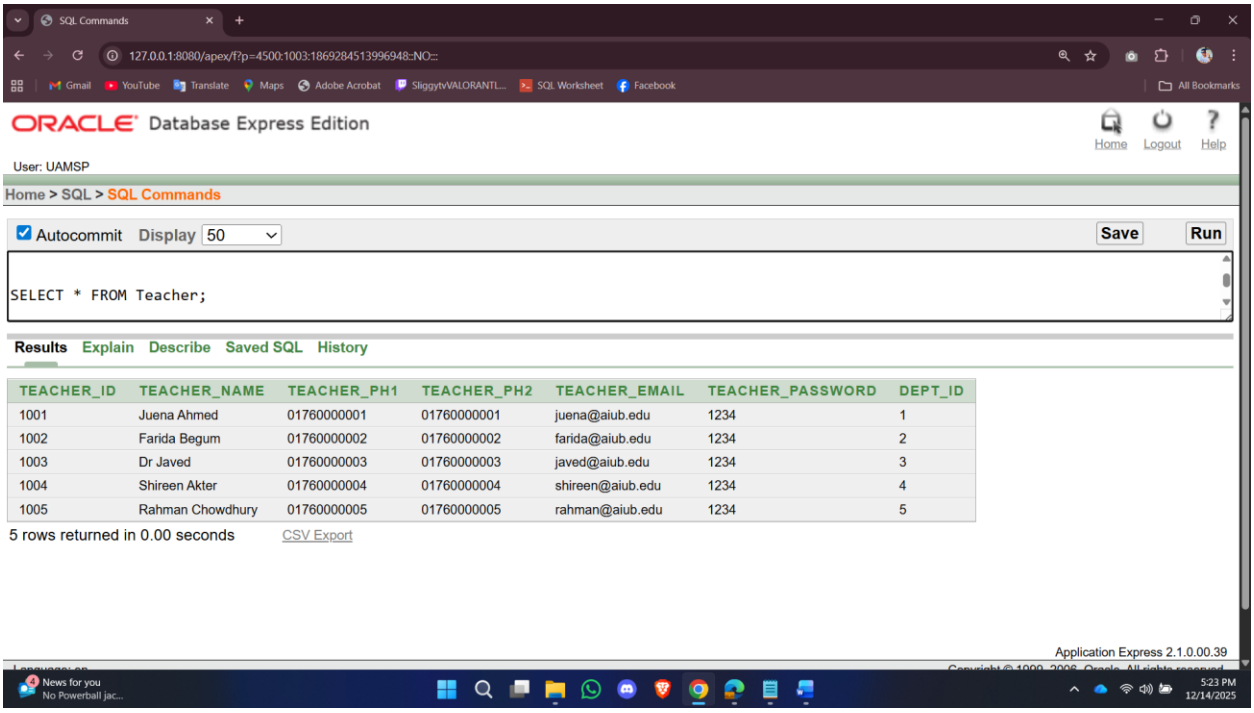
INSERT INTO Teacher VALUES (1002,'Farida Begum','01760000002','01760000002','farida@aiub.edu','1234',2);

INSERT INTO Teacher VALUES (1003,'Dr Javed','01760000003','01760000003','javed@aiub.edu','1234',3);

INSERT INTO Teacher VALUES (1004,'Shireen Akter','01760000004','01760000004','shireen@aiub.edu','1234',4);

INSERT INTO Teacher VALUES (1005,'Rahman Chowdhury','01760000005','01760000005','rahman@aiub.edu','1234',5);

SELECT * FROM Teacher;



APPLICANT:

INSERT INTO Applicant VALUES (1001,'Nazmus Sadat','Abdus Shukur','Nazma','Bangladeshi','Islam','Trust College',5.00,'Pangsha High',5.00,'numan@gmail.com','01710000011','01610000011','1234',DATE '2003-08-30','Rajbari');

INSERT INTO Applicant VALUES (1002,'Papia Prianka','Ujjol','Rotna','Bangladeshi','Islam','VNC',5.00,'Royal School',5.00,'papia@gmail.com','01710000012','01610000012','1234',DATE '2003-11-04','Brahmanbaria');

INSERT INTO Applicant VALUES (1003,'Sultan Ahmed','Shamsul','Rita','Bangladeshi','Islam','NDC',4.75,'Gregory',4.60,'sultan@gmail.com','01710000013','01610000013','1234',DATE '2005-01-15','Banani');

INSERT INTO Applicant VALUES (1004,'Nusrat Jahan','Kamal','Hasina','Bangladeshi','Islam','Holy Cross',4.82,'Model School',4.70,'nusrat@gmail.com','01510000014','01710000014','1234',DATE '2005-06-20','Dhanmondi');

INSERT INTO Applicant VALUES (1005,'Rafiq Chowdhury','Anwar','Rehana','Bangladeshi','Islam','Cambrian',4.60,'Zeenat High',4.55,'rafiq@gmail.com','01710000015','01310000015','1234',DATE '2004-11-30','Mirpur');

INSERT INTO Applicant VALUES (1006,'Imran Hossain','Jalal','Shirin','Bangladeshi','Islam','City

College',3.90,'Ideal',3.80,'imran@gmail.com','01310000016','01710000016','1234',DATE '2004-01-01','Uttara');

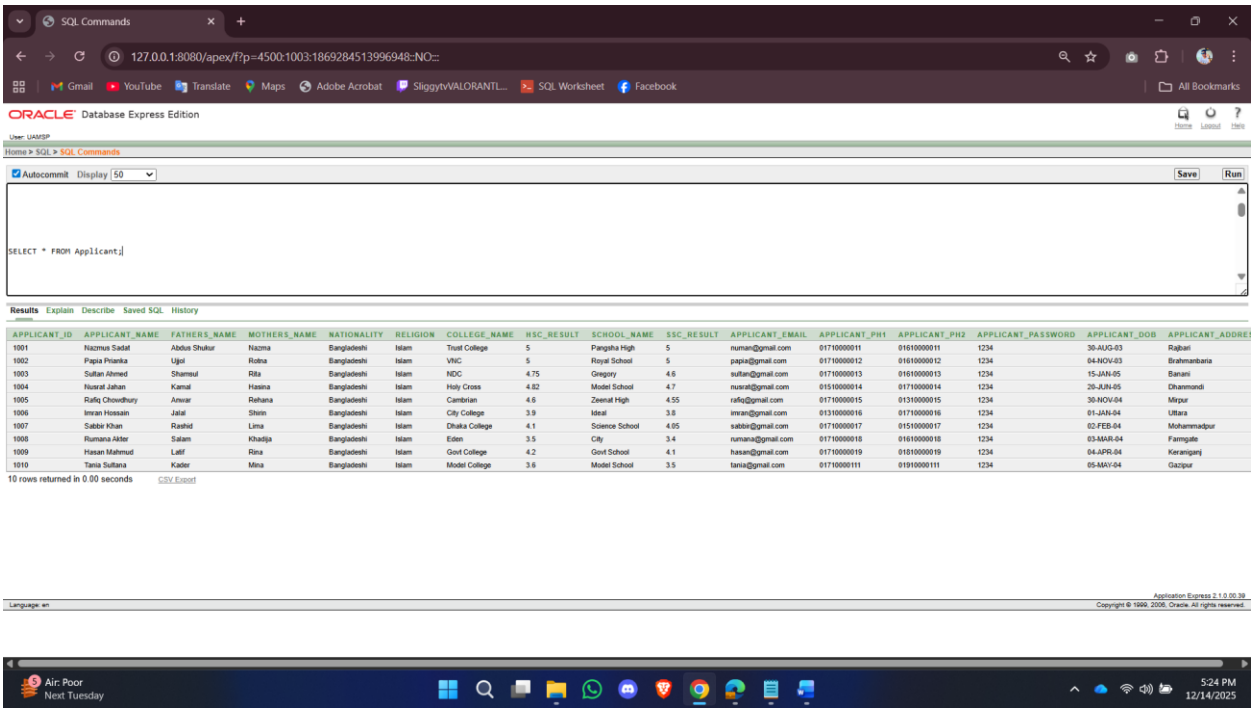
INSERT INTO Applicant VALUES (1007,'Sabbir Khan','Rashid','Lima','Bangladeshi','Islam','Dhaka College',4.10,'Science School',4.05,'sabbir@gmail.com','01710000017','01510000017','1234',DATE '2004-02-02','Mohammadpur');

INSERT INTO Applicant VALUES (1008,'Rumana Akter','Salam','Khadija','Bangladeshi','Islam','Eden',3.50,'City',3.40,'rumana@gmail.com','01710000018','01610000018','1234',DATE '2004-03-03','Farmgate');

INSERT INTO Applicant VALUES (1009,'Hasan Mahmud','Latif','Rina','Bangladeshi','Islam','Govt College',4.20,'Govt School',4.10,'hasan@gmail.com','01710000019','01810000019','1234',DATE '2004-04-04','Keraniganj');

INSERT INTO Applicant VALUES (1010,'Tania Sultana','Kader','Mina','Bangladeshi','Islam','Model College',3.60,'Model School',3.50,'tania@gmail.com','01710000111','01910000111','1234',DATE '2004-05-05','Gazipur');

SELECT * FROM Applicant;



APPLICATION:

INSERT INTO Application VALUES (701,DATE '2025-02-01','Accept',1,1001,1);

INSERT INTO Application VALUES (702,DATE '2025-02-02','Accept',1,1002,3);

INSERT INTO Application VALUES (703,DATE '2025-02-03','Accept',2,1003,1);

INSERT INTO Application VALUES (704,DATE '2025-02-04','Accept',2,1004,5);

INSERT INTO Application VALUES (705,DATE '2025-02-05','Accept',3,1005,2);

INSERT INTO Application VALUES (706,DATE '2025-02-06','Accept',3,1007,1);

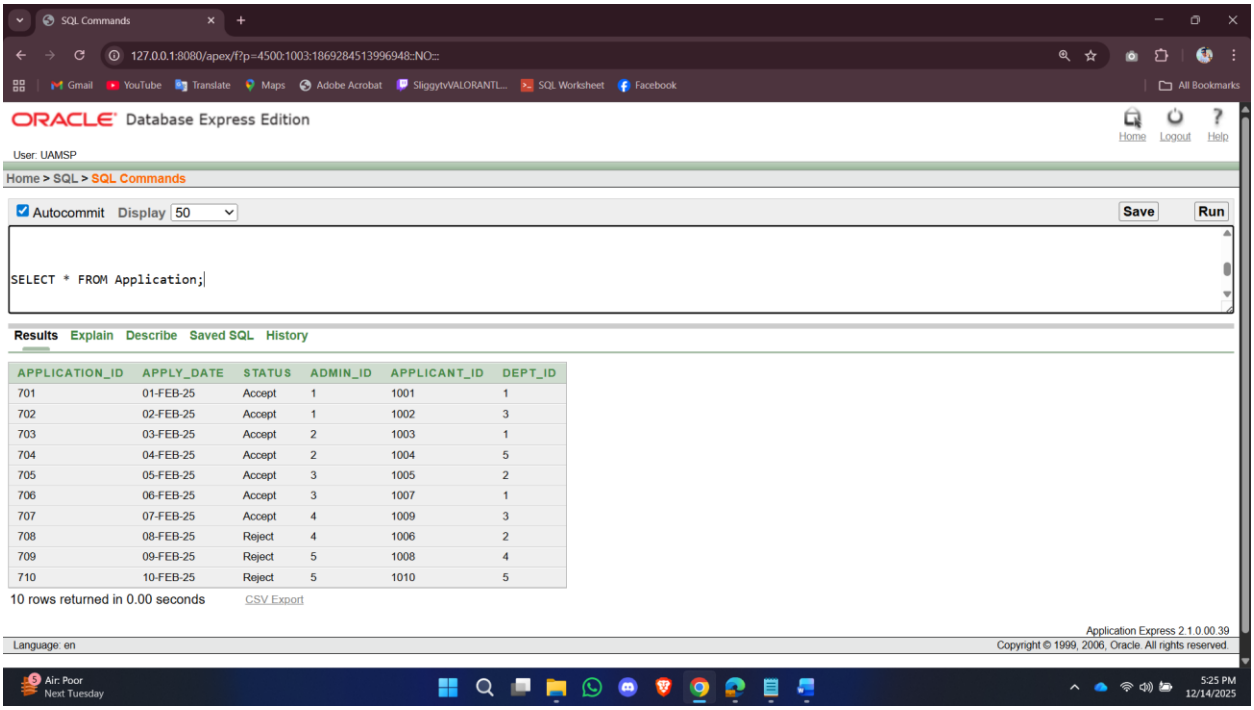
INSERT INTO Application VALUES (707,DATE '2025-02-07','Accept',4,1009,3);

INSERT INTO Application VALUES (708,DATE '2025-02-08','Reject',4,1006,2);

INSERT INTO Application VALUES (709,DATE '2025-02-09','Reject',5,1008,4);

INSERT INTO Application VALUES (710,DATE '2025-02-10','Reject',5,1010,5);

SELECT * FROM Application;



PAYMENT:

INSERT INTO Payment VALUES (9001,1500,'Bkash','Application Fee',DATE '2025-02-01',701);

INSERT INTO Payment VALUES (9002,1500,'Nagad','Application Fee',DATE '2025-02-02',702);

INSERT INTO Payment VALUES (9003,1500,'Card','Application Fee',DATE '2025-02-03',703);

INSERT INTO Payment VALUES (9004,1500,'Bkash','Application Fee',DATE '2025-02-04',704);

INSERT INTO Payment VALUES (9005,1500,'Card','Application Fee',DATE '2025-02-05',705);

INSERT INTO Payment VALUES (9006,1500,'Nagad','Application Fee',DATE '2025-02-06',706);

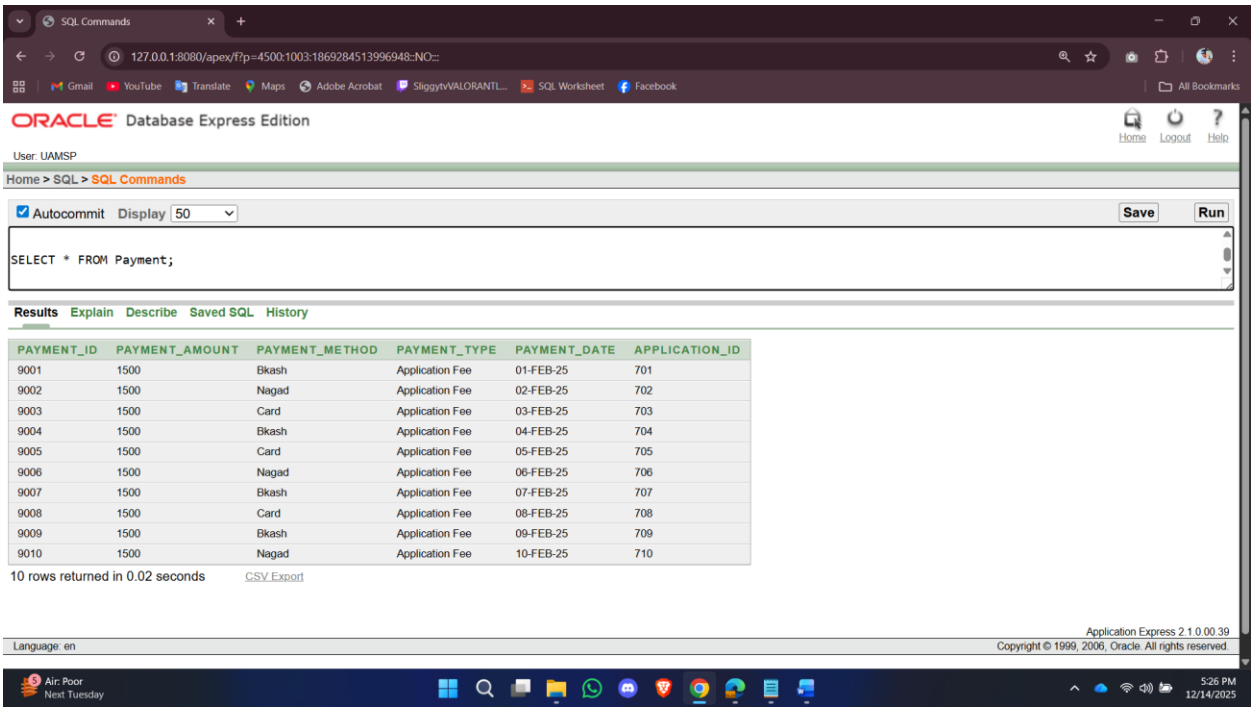
INSERT INTO Payment VALUES (9007,1500,'Bkash','Application Fee',DATE '2025-02-07',707);

INSERT INTO Payment VALUES (9008,1500,'Card','Application Fee',DATE '2025-02-08',708);

INSERT INTO Payment VALUES (9009,1500,'Bkash','Application Fee',DATE '2025-02-09',709);

INSERT INTO Payment VALUES (9010,1500,'Nagad','Application Fee',DATE '2025-02-10',710);

SELECT * FROM Payment;



EVALUATION:

INSERT INTO Evaluation VALUES (3301,75,15,'Passed',701,1001,1);

INSERT INTO Evaluation VALUES (3302,70,14,'Passed',702,1003,1);

INSERT INTO Evaluation VALUES (3303,68,13,'Passed',703,1001,2);

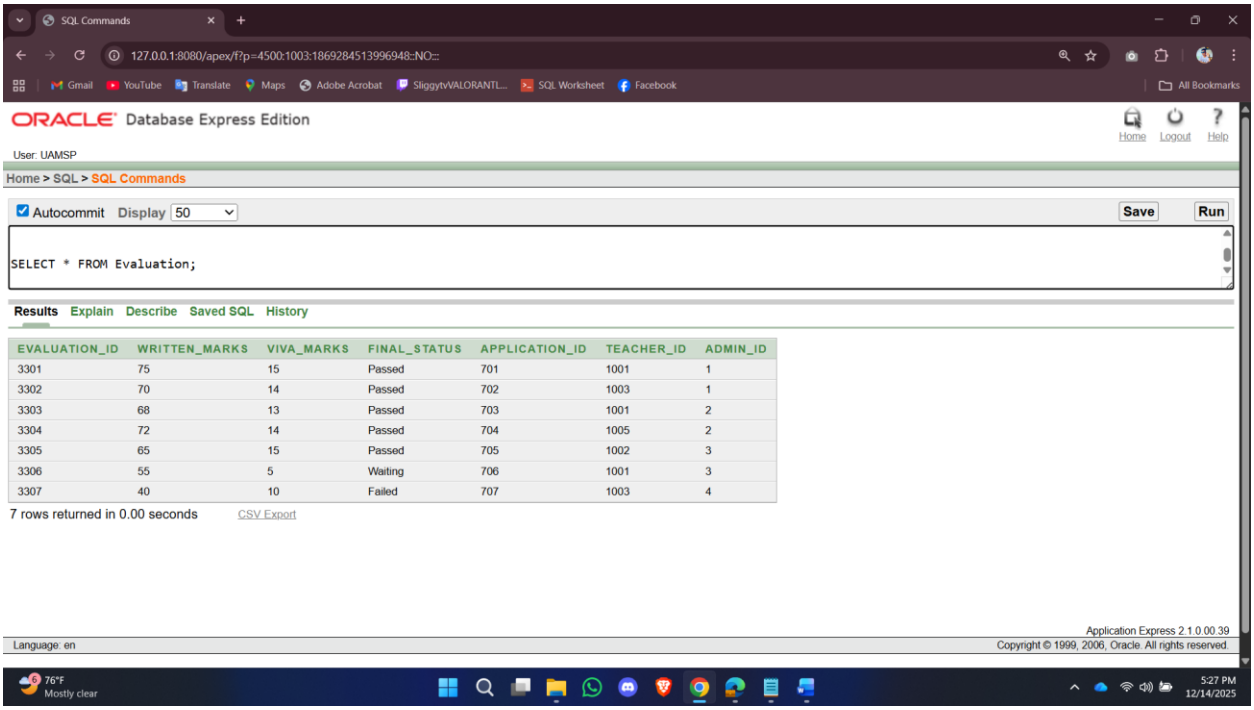
INSERT INTO Evaluation VALUES (3304,72,14,'Passed',704,1005,2);

INSERT INTO Evaluation VALUES (3305,65,15,'Passed',705,1002,3);

INSERT INTO Evaluation VALUES (3306,55,5,'Waiting',706,1001,3);

INSERT INTO Evaluation VALUES (3307,40,10,'Failed',707,1003,4);

SELECT * FROM Evaluation;



STUDENT:

INSERT INTO Student VALUES (3001,'3001@student.aiub.edu','1234',DATE '2025-02-20',1001,1,2001);

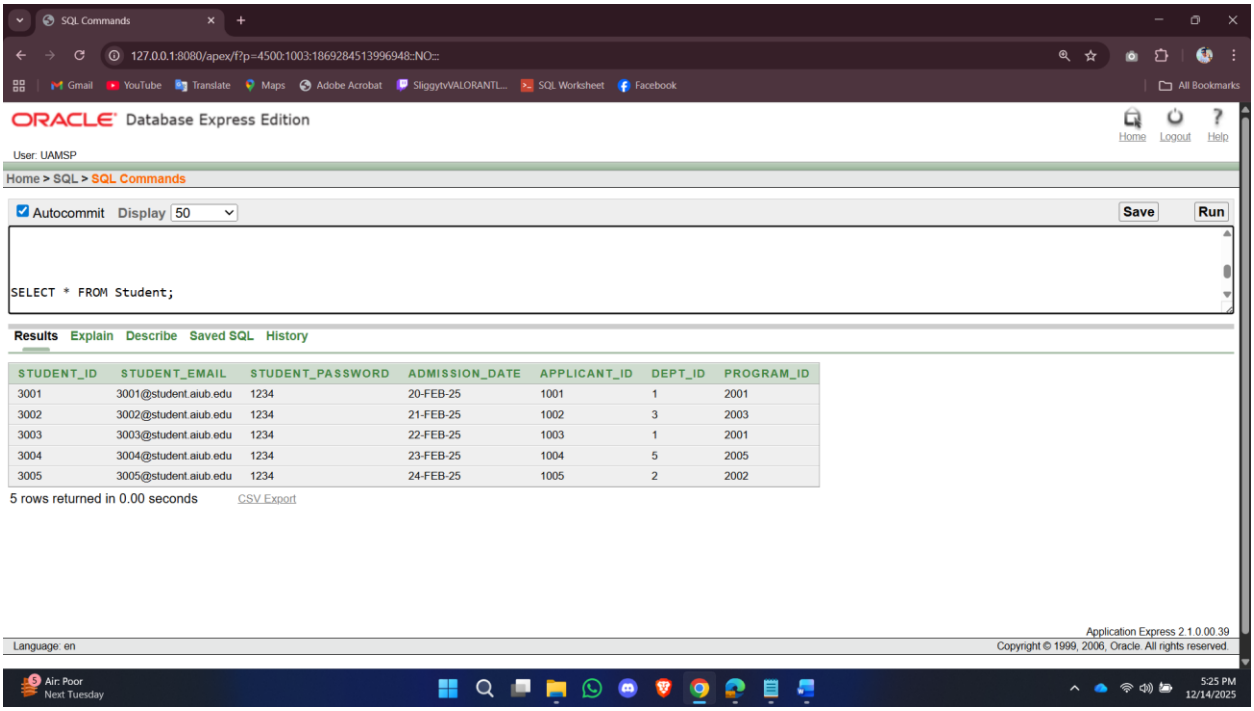
INSERT INTO Student VALUES (3002,'3002@student.aiub.edu','1234',DATE '2025-02-21',1002,3,2003);

INSERT INTO Student VALUES (3003,'3003@student.aiub.edu','1234',DATE '2025-02-22',1003,1,2001);

INSERT INTO Student VALUES (3004,'3004@student.aiub.edu','1234',DATE '2025-02-23',1004,5,2005);

INSERT INTO Student VALUES (3005,'3005@student.aiub.edu','1234',DATE '2025-02-24',1005,2,2002);

SELECT * FROM Student;



Basic PL/SQL:

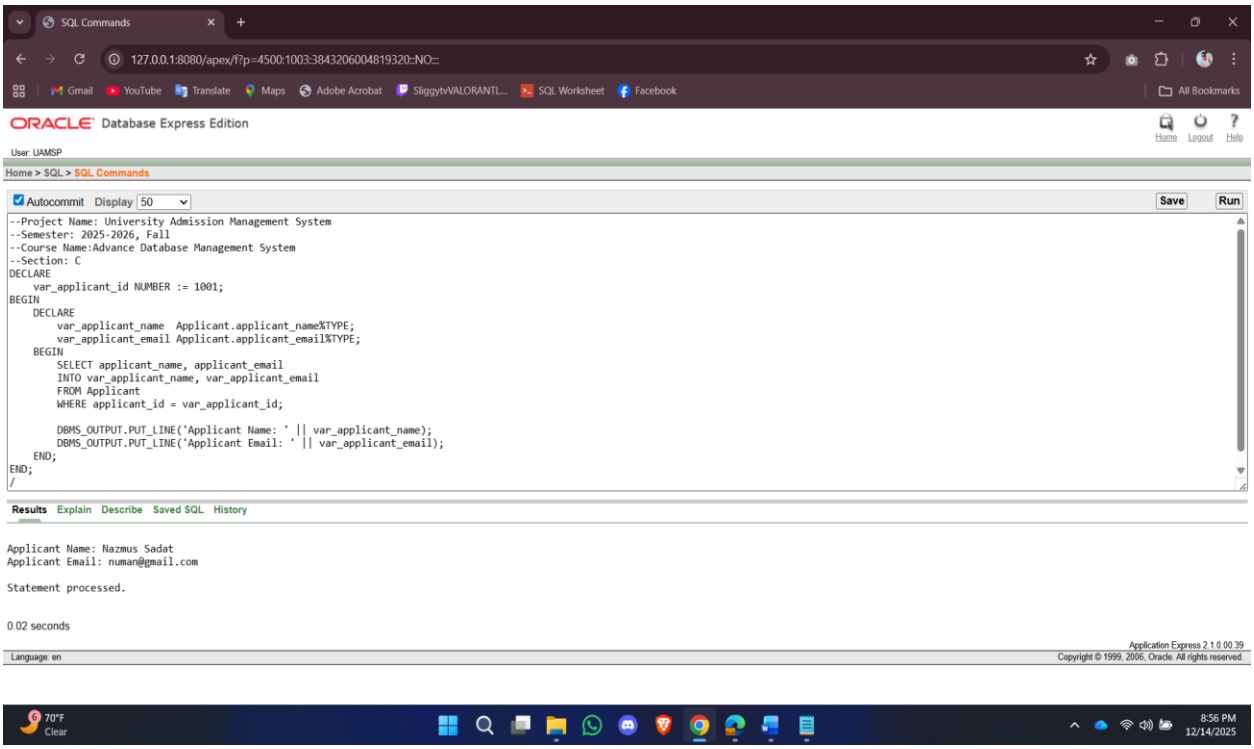
Variables:

1. Write a PL/SQL block using local variables to display an applicant's name and email for applicant ID 1001.

```
DECLARE
   var_applicant_id NUMBER := 1001;
BEGIN
   DECLARE
      var_applicant_name  Applicant.applicant_name%TYPE;
      var_applicant_email Applicant.applicant_email%TYPE;
   BEGIN
      SELECT applicant_name, applicant_email
      INTO var_applicant_name, var_applicant_email
      FROM Applicant
      WHERE applicant_id = var_applicant_id;

      DBMS_OUTPUT.PUT_LINE('Applicant Name: ' || var_applicant_name);
      DBMS_OUTPUT.PUT_LINE('Applicant Email: ' || var_applicant_email);
```

```
    END;
END;
/
```

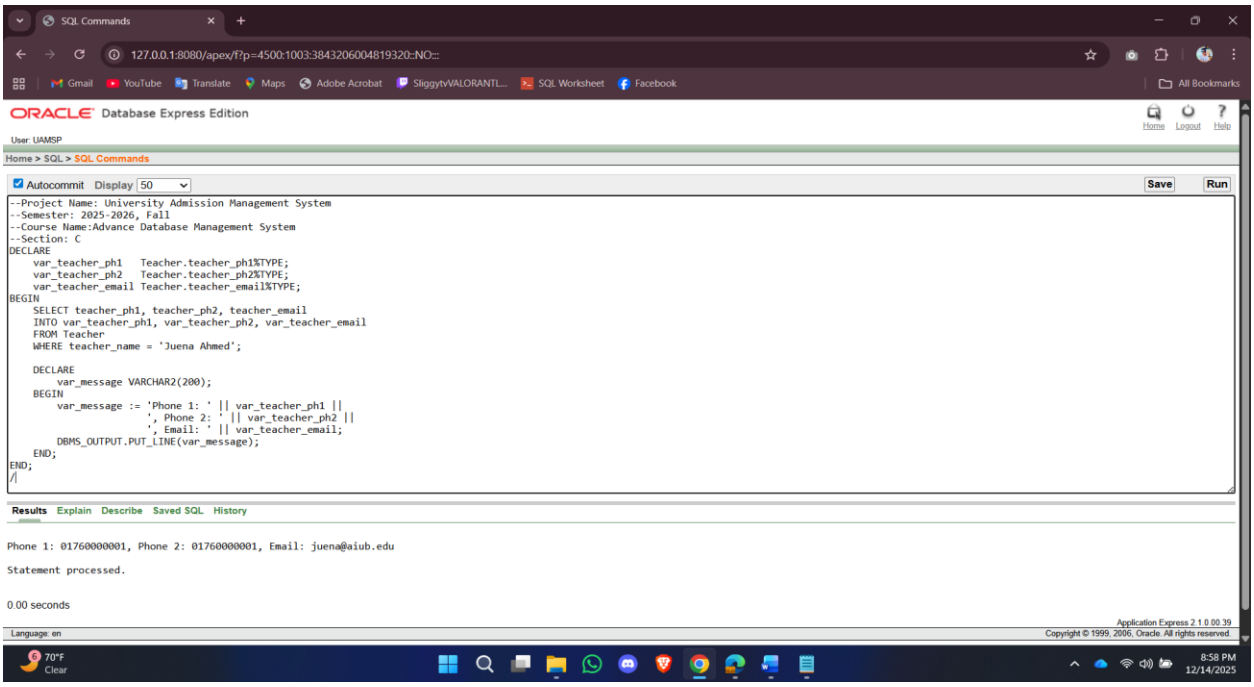


2. Write a PL/SQL block using global variables to display a teacher's phone numbers and email for teacher name Juena Ahmed.

```
DECLARE
   var_teacher_ph1    Teacher.teacher_ph1%TYPE;
   var_teacher_ph2    Teacher.teacher_ph2%TYPE;
   var_teacher_email Teacher.teacher_email%TYPE;
BEGIN
   SELECT teacher_ph1, teacher_ph2, teacher_email
   INTO var_teacher_ph1, var_teacher_ph2, var_teacher_email
   FROM Teacher
   WHERE teacher_name = 'Juena Ahmed';

   DECLARE
      var_message VARCHAR2(200);
```

```
    BEGIN
        var_message := 'Phone 1: ' || var_teacher_ph1 ||
                       ', Phone 2: ' || var_teacher_ph2 ||
                       ', Email: ' || var_teacher_email;
        DBMS_OUTPUT.PUT_LINE(var_message);
    END;
END;
/
```



Operators:

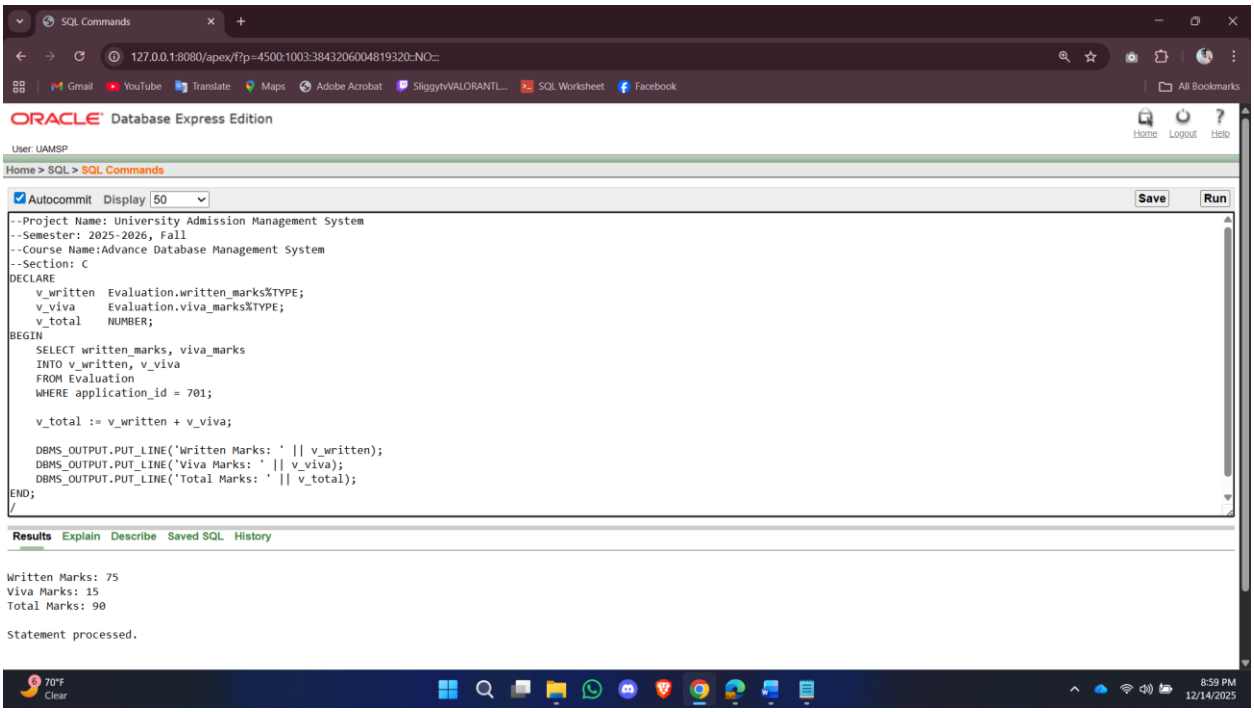
3. Write a PL/SQL block to calculate total marks by adding written marks and viva marks for application id 701.

```
DECLARE
    v_written  Evaluation.written_marks%TYPE;
    v_viva     Evaluation.viva_marks%TYPE;
    v_total    NUMBER;
BEGIN
    SELECT written_marks, viva_marks
    INTO v_written, v_viva
```

```
    FROM Evaluation
    WHERE application_id = 701;

    v_total := v_written + v_viva;

    DBMS_OUTPUT.PUT_LINE('Written Marks: ' || v_written);
    DBMS_OUTPUT.PUT_LINE('Viva Marks: ' || v_viva);
    DBMS_OUTPUT.PUT_LINE('Total Marks: ' || v_total);
END;
/
```



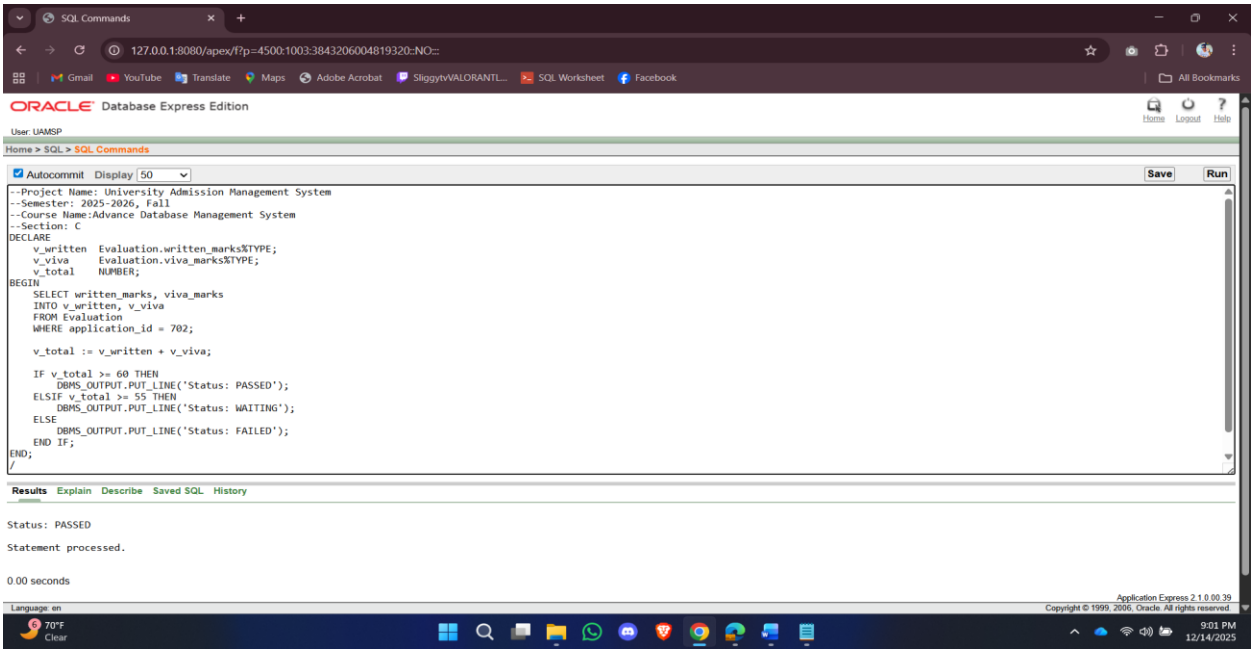
4. Write a PL/SQL block to check whether an applicant passed, waiting, or failed based on total marks using relational operators for application ID = 702.

```
DECLARE
    v_written  Evaluation.written_marks%TYPE;
    v_viva     Evaluation.viva_marks%TYPE;
    v_total    NUMBER;
BEGIN
    SELECT written_marks, viva_marks
```

```
    INTO v_written, v_viva
    FROM Evaluation
    WHERE application_id = 702;

    v_total := v_written + v_viva;

    IF v_total >= 60 THEN
        DBMS_OUTPUT.PUT_LINE('Status: PASSED');
    ELSIF v_total >= 55 THEN
        DBMS_OUTPUT.PUT_LINE('Status: WAITING');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Status: FAILED');
    END IF;
END;
/
```



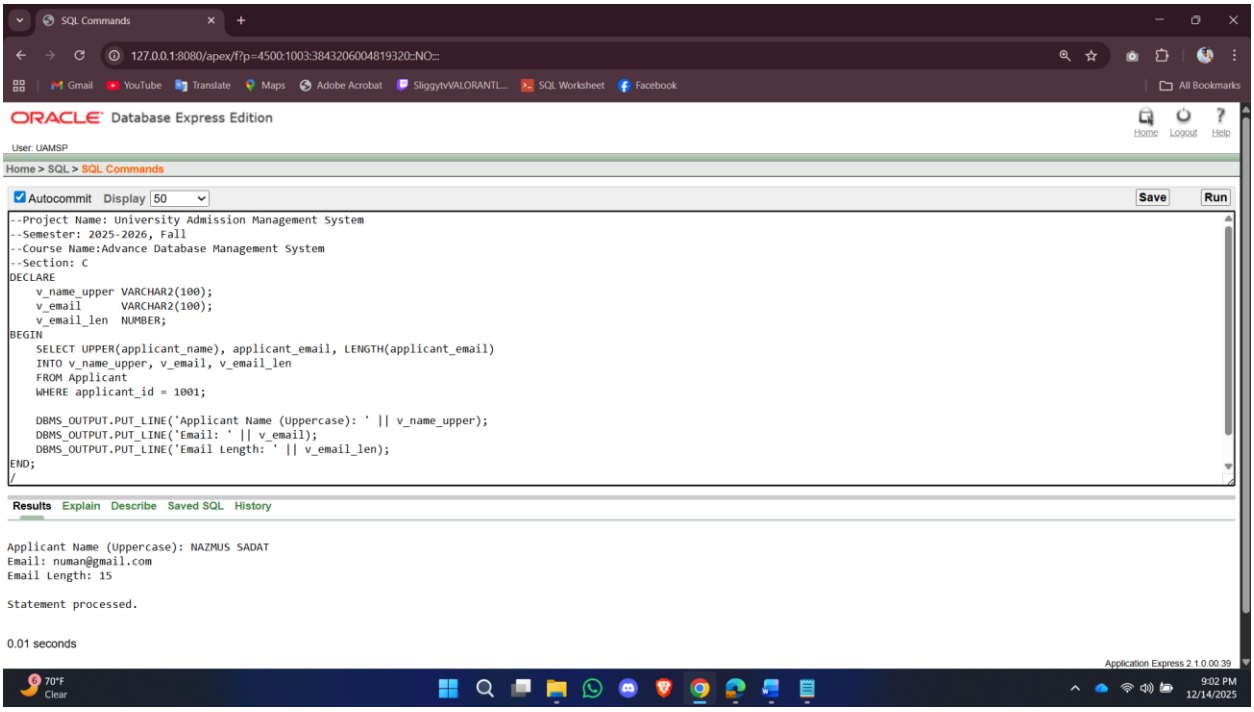
Single-row function:

5. Write a PL/SQL block using single-row functions to display applicant name in uppercase, email and length of email for applicant ID 1001.

```
DECLARE
```

```
    v_name_upper VARCHAR2(100);
    v_email      VARCHAR2(100);
    v_email_len  NUMBER;
BEGIN
    SELECT UPPER(applicant_name), applicant_email, LENGTH(applicant_email)
    INTO v_name_upper, v_email, v_email_len
    FROM Applicant
    WHERE applicant_id = 1001;

    DBMS_OUTPUT.PUT_LINE('Applicant Name (Uppercase): ' || v_name_upper);
    DBMS_OUTPUT.PUT_LINE('Email: ' || v_email);
    DBMS_OUTPUT.PUT_LINE('Email Length: ' || v_email_len);
END;
/
```

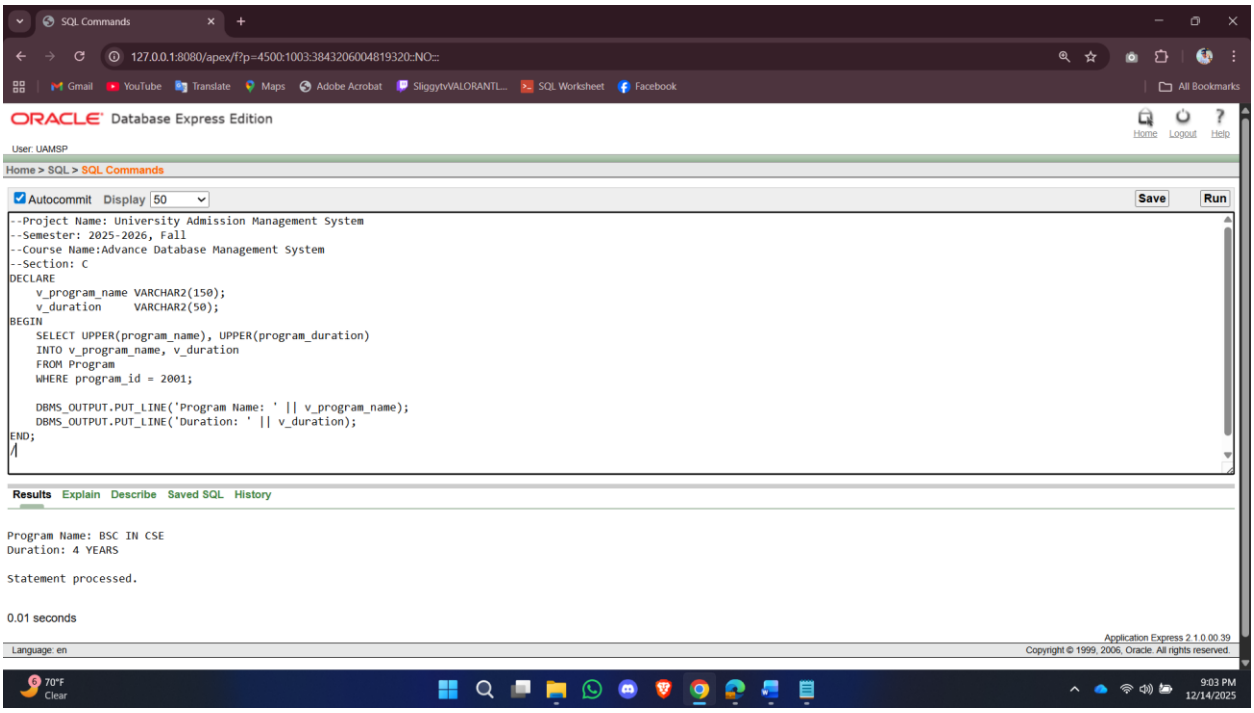


6. Write a PL/SQL block using single-row functions to display program name and duration in uppercase.

```
DECLARE
```

```
    v_program_name VARCHAR2(150);
    v_duration     VARCHAR2(50);
BEGIN
    SELECT UPPER(program_name), UPPER(program_duration)
    INTO v_program_name, v_duration
    FROM Program
    WHERE program_id = 2001;

    DBMS_OUTPUT.PUT_LINE('Program Name: ' || v_program_name);
    DBMS_OUTPUT.PUT_LINE('Duration: ' || v_duration);
END;
/
```



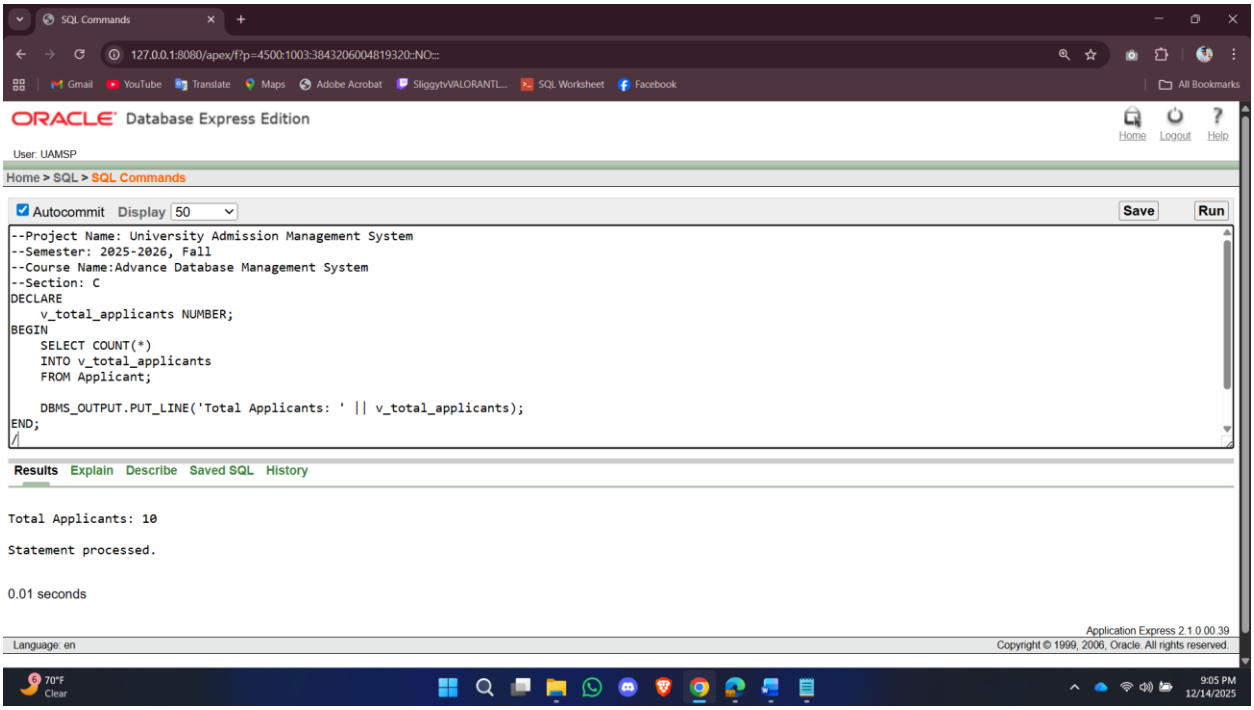
Group function:

7. Write a PL/SQL block to display total number of applicants using group function.

```
DECLARE
    v_total_applicants NUMBER;
BEGIN
```

```
    SELECT COUNT(*)
    INTO v_total_applicants
    FROM Applicant;

    DBMS_OUTPUT.PUT_LINE('Total Applicants: ' || v_total_applicants);
END;
/
```

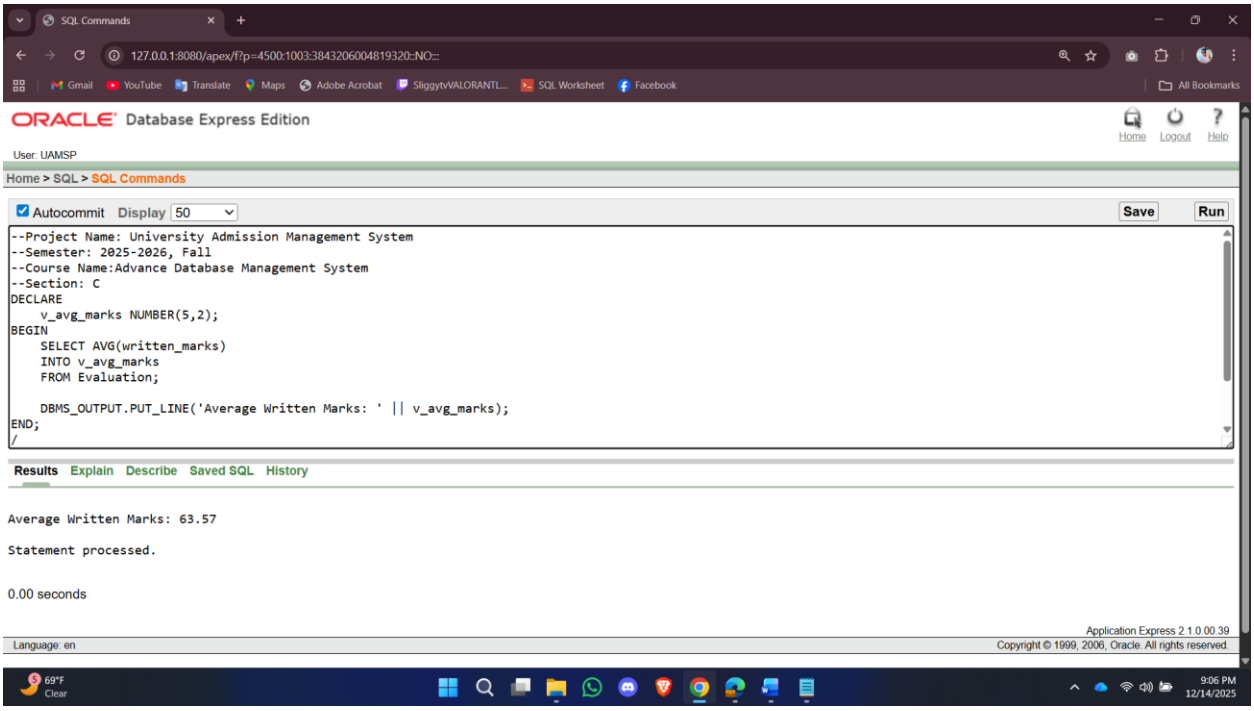


8. Write a PL/SQL block to display average written marks of all evaluations using group function.

```
DECLARE
    v_avg_marks NUMBER(5,2);
BEGIN
    SELECT AVG(written_marks)
    INTO v_avg_marks
    FROM Evaluation;

    DBMS_OUTPUT.PUT_LINE('Average Written Marks: ' || v_avg_marks);
END;
```

/



Loop:

9. Write a PL/SQL block using WHILE loop to display the first 5 applicant IDs along with their name, HSC GPA, and SSC GPA.

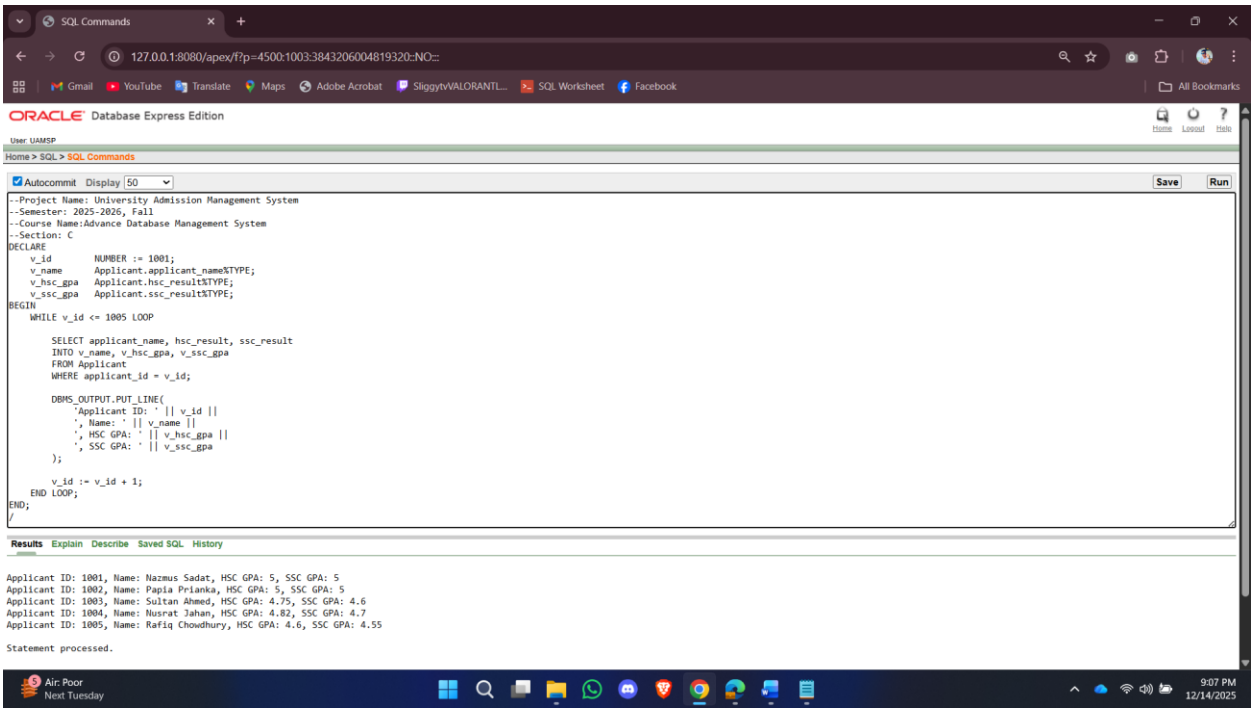
```
DECLARE
   v_id        NUMBER := 1001;
   v_name      Applicant.applicant_name%TYPE;
   v_hsc_gpa   Applicant.hsc_result%TYPE;
   v_ssc_gpa   Applicant.ssc_result%TYPE;
BEGIN
   WHILE v_id <= 1005 LOOP

      SELECT applicant_name, hsc_result, ssc_result
      INTO v_name, v_hsc_gpa, v_ssc_gpa
      FROM Applicant
      WHERE applicant_id = v_id;

      DBMS_OUTPUT.PUT_LINE(
```

```
            'Applicant ID: ' || v_id ||
            ', Name: ' || v_name ||
            ', HSC GPA: ' || v_hsc_gpa ||
            ', SSC GPA: ' || v_ssc_gpa
        );

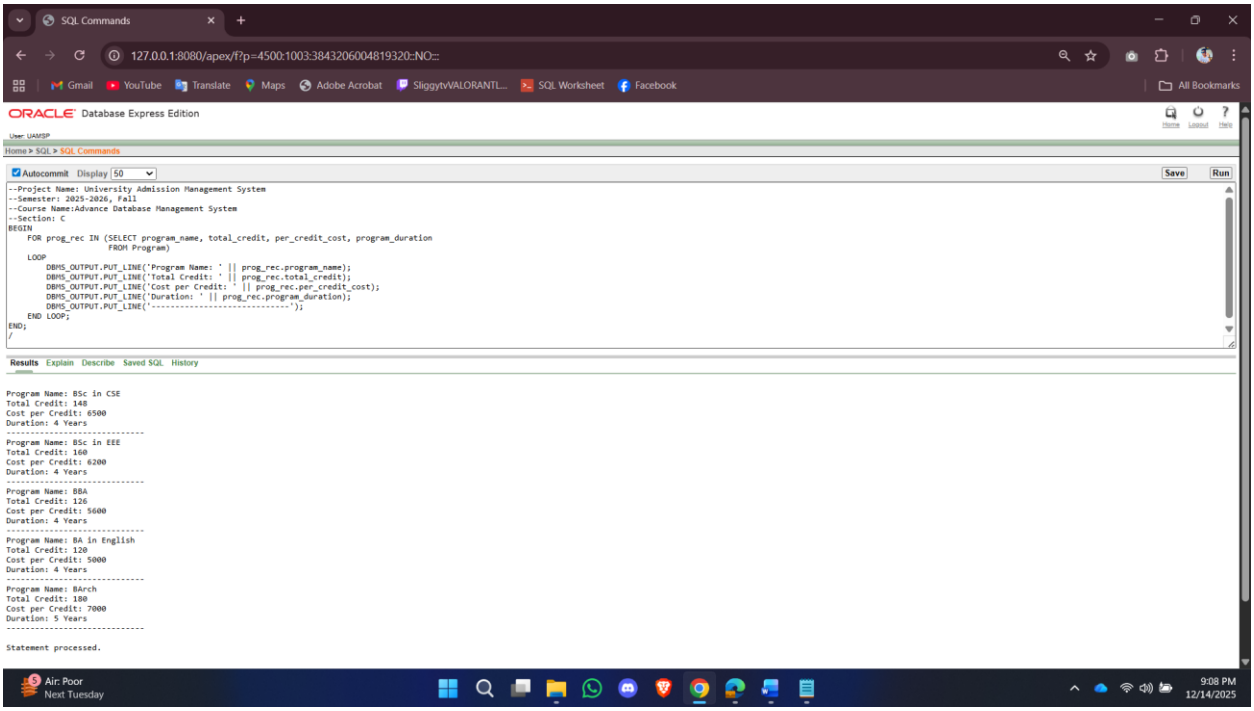
        v_id := v_id + 1;
    END LOOP;
END;
/
```



10. Write a PL/SQL block using FOR loop to display all program name, credit, per credit cost and duration.

```
BEGIN
    FOR prog_rec IN (SELECT program_name, total_credit, per_credit_cost, program_duration
                     FROM Program)
    LOOP
        DBMS_OUTPUT.PUT_LINE('Program Name: ' || prog_rec.program_name);
        DBMS_OUTPUT.PUT_LINE('Total Credit: ' || prog_rec.total_credit);
```

```
    DBMS_OUTPUT.PUT_LINE('Cost per Credit: ' || prog_rec.per_credit_cost);
    DBMS_OUTPUT.PUT_LINE('Duration: ' || prog_rec.program_duration);
    DBMS_OUTPUT.PUT_LINE('----------------------------');
  END LOOP;
END;
/
```



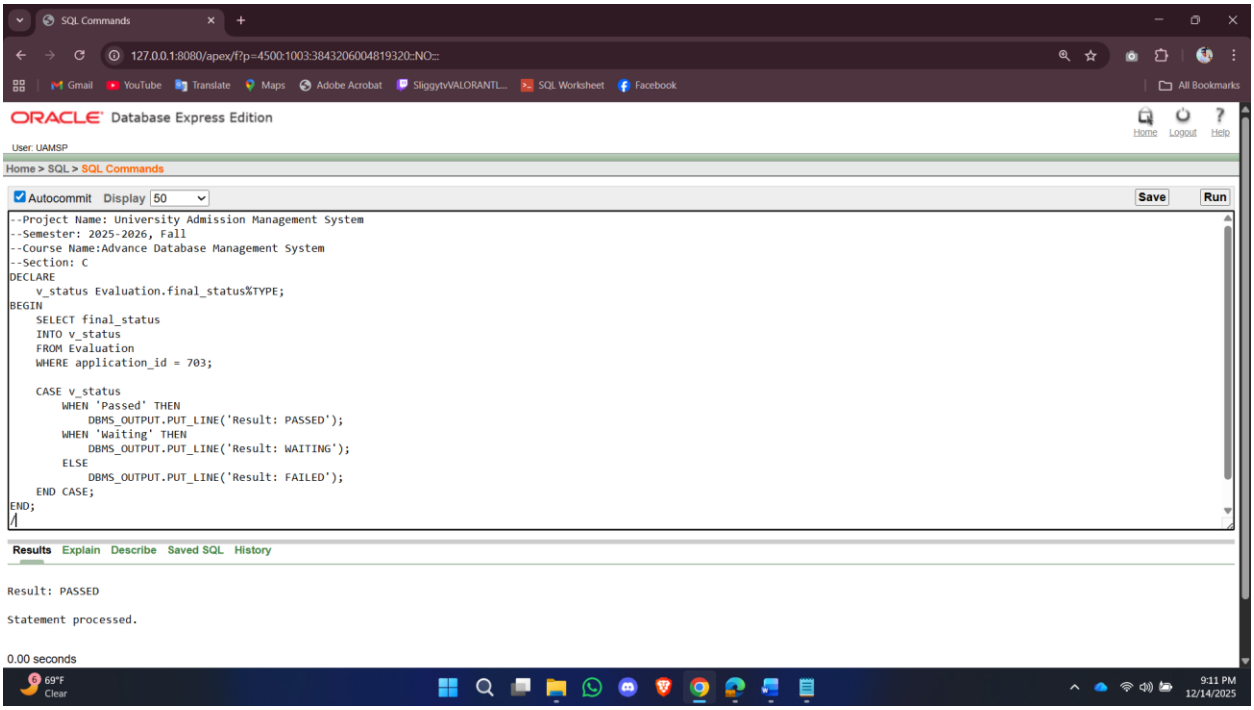
Conditional statements:

11. Write a PL/SQL block using CASE statement to display evaluation result for application ID 703.

```
DECLARE
  v_status Evaluation.final_status%TYPE;
BEGIN
  SELECT final_status
  INTO v_status
  FROM Evaluation
  WHERE application_id = 703;

  CASE v_status
```

```
        WHEN 'Passed' THEN
            DBMS_OUTPUT.PUT_LINE('Result: PASSED');
        WHEN 'Waiting' THEN
            DBMS_OUTPUT.PUT_LINE('Result: WAITING');
        ELSE
            DBMS_OUTPUT.PUT_LINE('Result: FAILED');
    END CASE;
END;
/
```



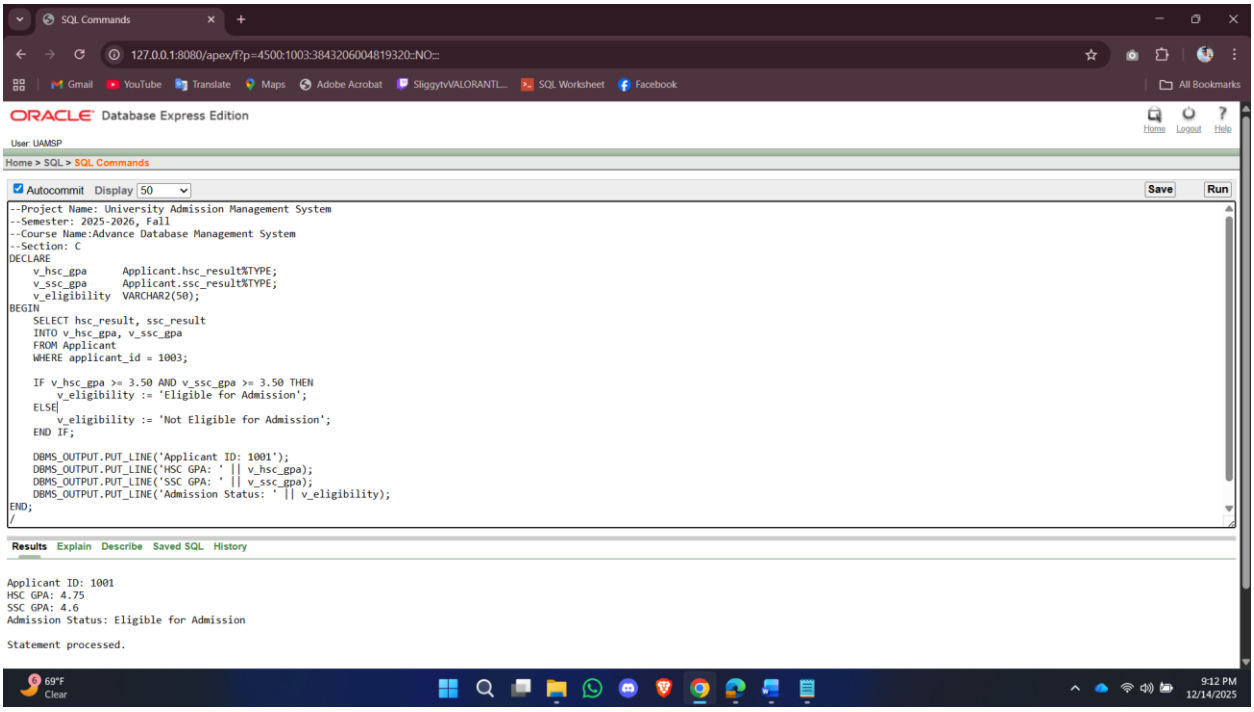
12. Write a PL/SQL block to check the admission eligibility of an applicant based on their HSC and SSC GPA which id is 1003.

```
DECLARE
    v_hsc_gpa      Applicant.hsc_result%TYPE;
    v_ssc_gpa      Applicant.ssc_result%TYPE;
    v_eligibility  VARCHAR2(50);
BEGIN
    SELECT hsc_result, ssc_result
    INTO v_hsc_gpa, v_ssc_gpa
```

```
    FROM Applicant
    WHERE applicant_id = 1003;

    IF v_hsc_gpa >= 3.50 AND v_ssc_gpa >= 3.50 THEN
        v_eligibility := 'Eligible for Admission';
    ELSE
        v_eligibility := 'Not Eligible for Admission';
    END IF;

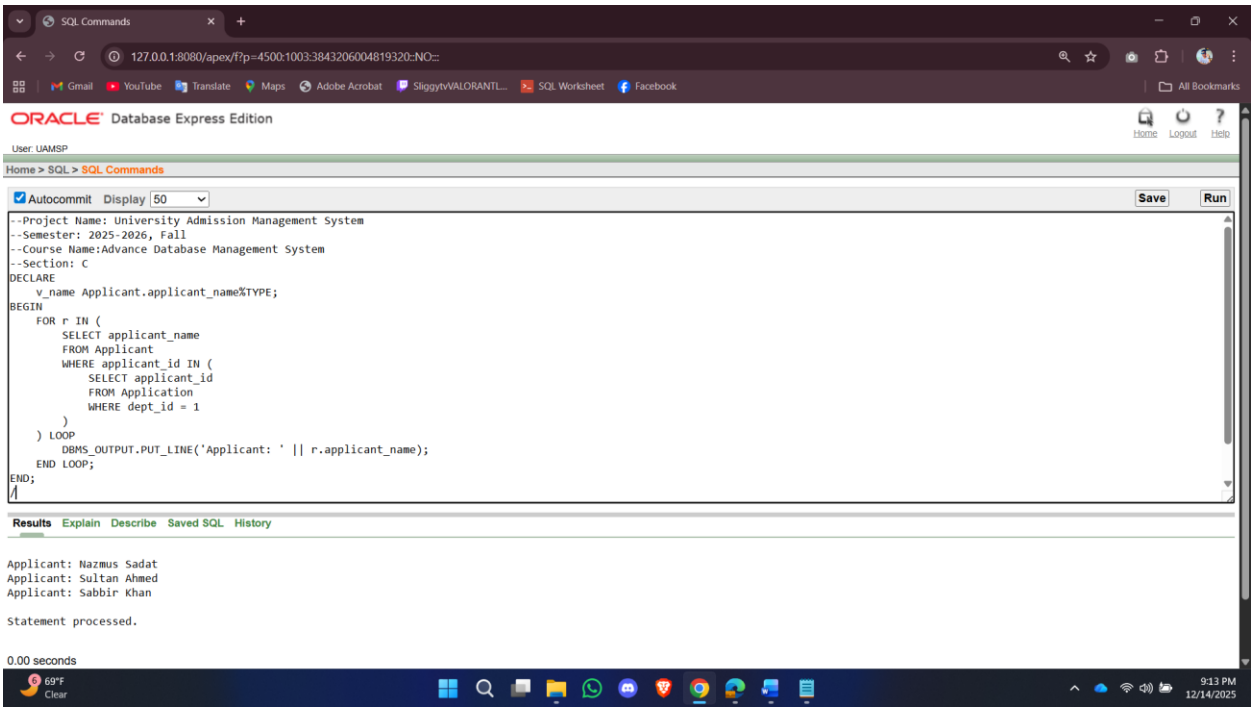
    DBMS_OUTPUT.PUT_LINE('Applicant ID: 1001');
    DBMS_OUTPUT.PUT_LINE('HSC GPA: ' || v_hsc_gpa);
    DBMS_OUTPUT.PUT_LINE('SSC GPA: ' || v_ssc_gpa);
    DBMS_OUTPUT.PUT_LINE('Admission Status: ' || v_eligibility);
END;
/
```



Subquery:

13. Write a PL/SQL block using multiple-row subquery to display applicants who applied to Computer Science department.

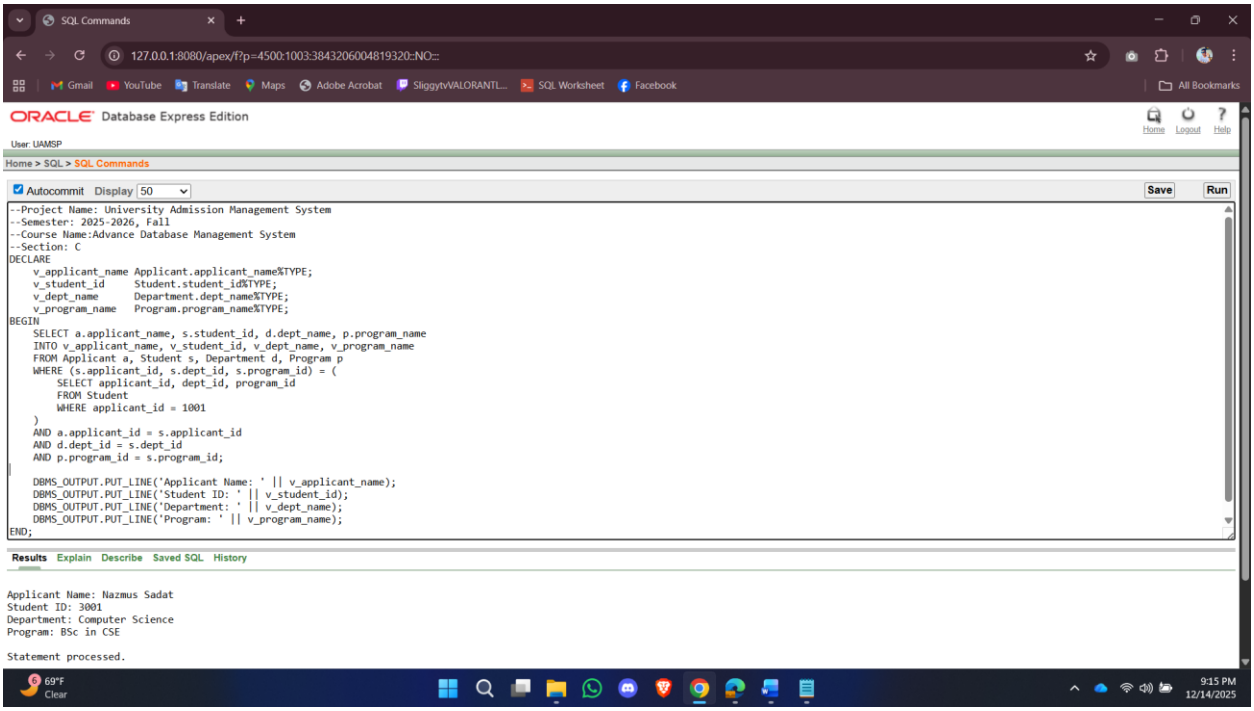
```
DECLARE
   v_name Applicant.applicant_name%TYPE;
BEGIN
   FOR r IN (
      SELECT applicant_name
      FROM Applicant
      WHERE applicant_id IN (
         SELECT applicant_id
         FROM Application
         WHERE dept_id = 1
      )
   ) LOOP
      DBMS_OUTPUT.PUT_LINE('Applicant: ' || r.applicant_name);
   END LOOP;
END;
/
```



14. Write a PL/SQL block using a multiple-column subquery to display the applicant name, student ID, department name, and program name for applicant ID 1001.

```
DECLARE
   v_applicant_name Applicant.applicant_name%TYPE;
   v_student_id     Student.student_id%TYPE;
   v_dept_name      Department.dept_name%TYPE;
   v_program_name   Program.program_name%TYPE;
BEGIN
   SELECT a.applicant_name, s.student_id, d.dept_name, p.program_name
   INTO v_applicant_name, v_student_id, v_dept_name, v_program_name
   FROM Applicant a, Student s, Department d, Program p
   WHERE (s.applicant_id, s.dept_id, s.program_id) = (
      SELECT applicant_id, dept_id, program_id
      FROM Student
      WHERE applicant_id = 1001
   )
   AND a.applicant_id = s.applicant_id
   AND d.dept_id = s.dept_id
   AND p.program_id = s.program_id;

   DBMS_OUTPUT.PUT_LINE('Applicant Name: ' || v_applicant_name);
   DBMS_OUTPUT.PUT_LINE('Student ID: ' || v_student_id);
   DBMS_OUTPUT.PUT_LINE('Department: ' || v_dept_name);
   DBMS_OUTPUT.PUT_LINE('Program: ' || v_program_name);
END;
/
```

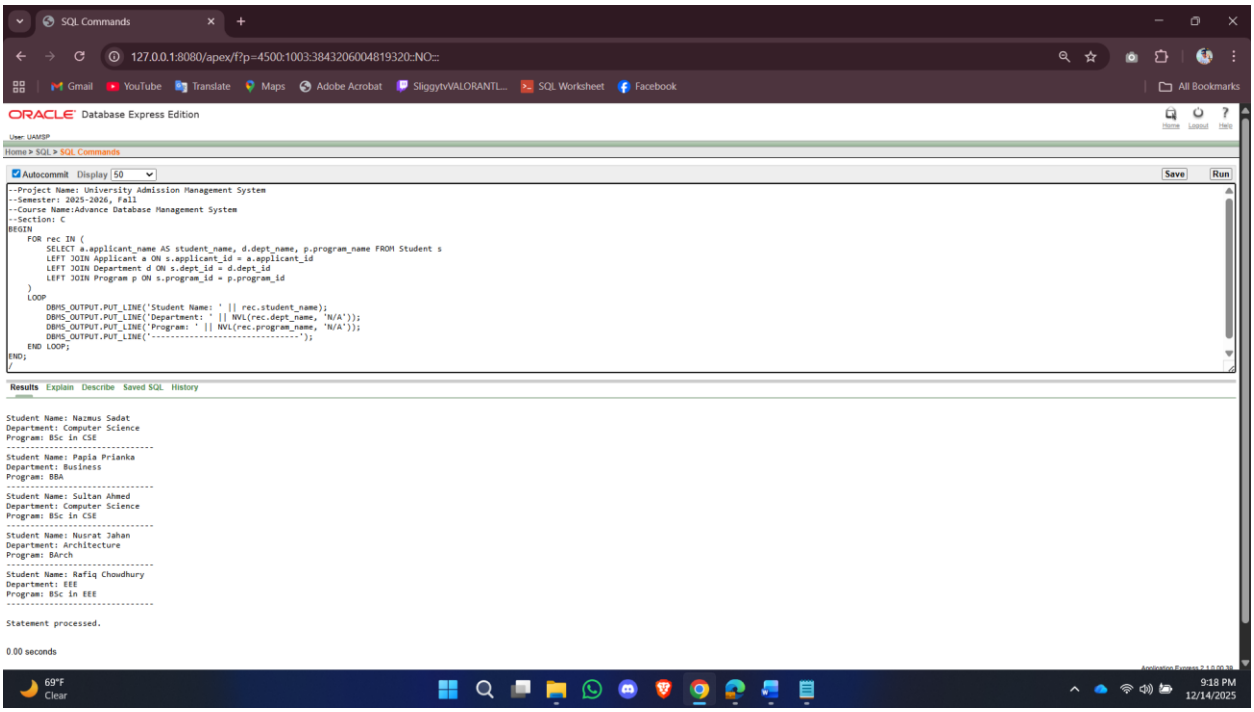


Joining:

15. Write a PL/SQL block to display the names of all students along with their corresponding department name and program name.

```
BEGIN
  FOR rec IN (
    SELECT a.applicant_name AS student_name,
           d.dept_name,
           p.program_name
    FROM Student s
    LEFT JOIN Applicant a ON s.applicant_id = a.applicant_id
    LEFT JOIN Department d ON s.dept_id = d.dept_id
    LEFT JOIN Program p ON s.program_id = p.program_id
  )
  LOOP
    DBMS_OUTPUT.PUT_LINE('Student Name: ' || rec.student_name);
    DBMS_OUTPUT.PUT_LINE('Department: ' || NVL(rec.dept_name, 'N/A'));
    DBMS_OUTPUT.PUT_LINE('Program: ' || NVL(rec.program_name, 'N/A'));
    DBMS_OUTPUT.PUT_LINE('-------------------------------');
```

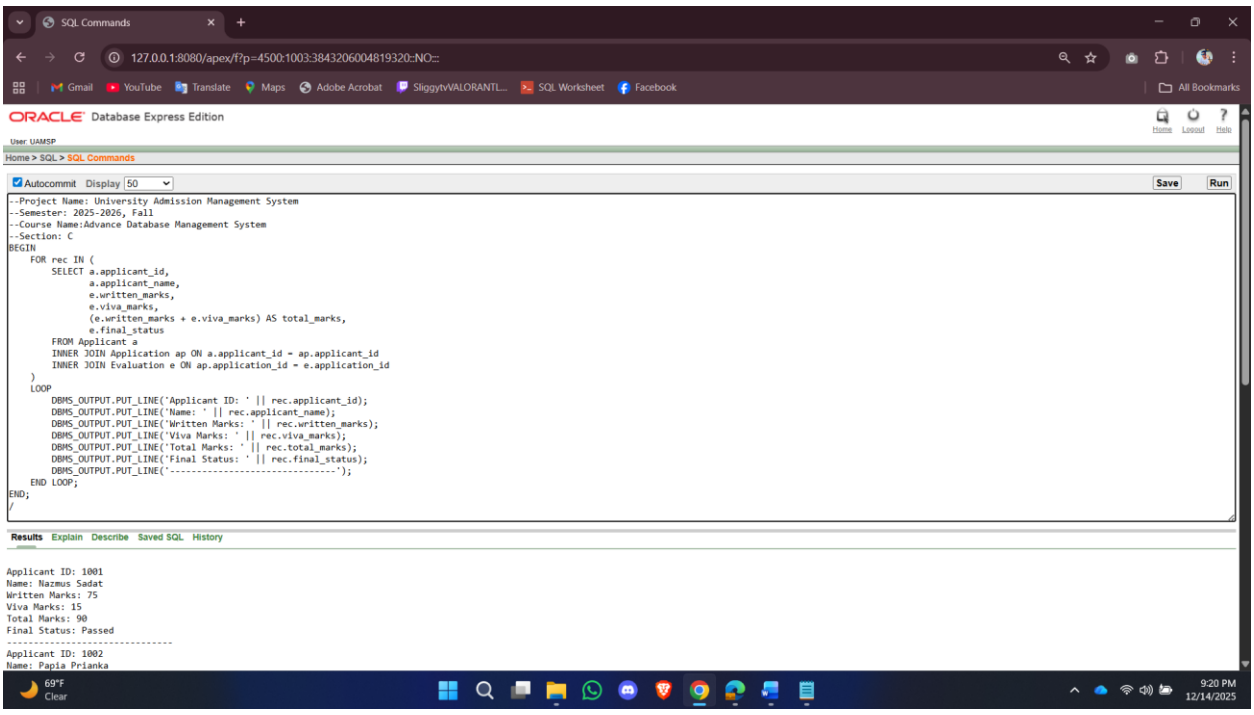
```
    END LOOP;
END;
/
```

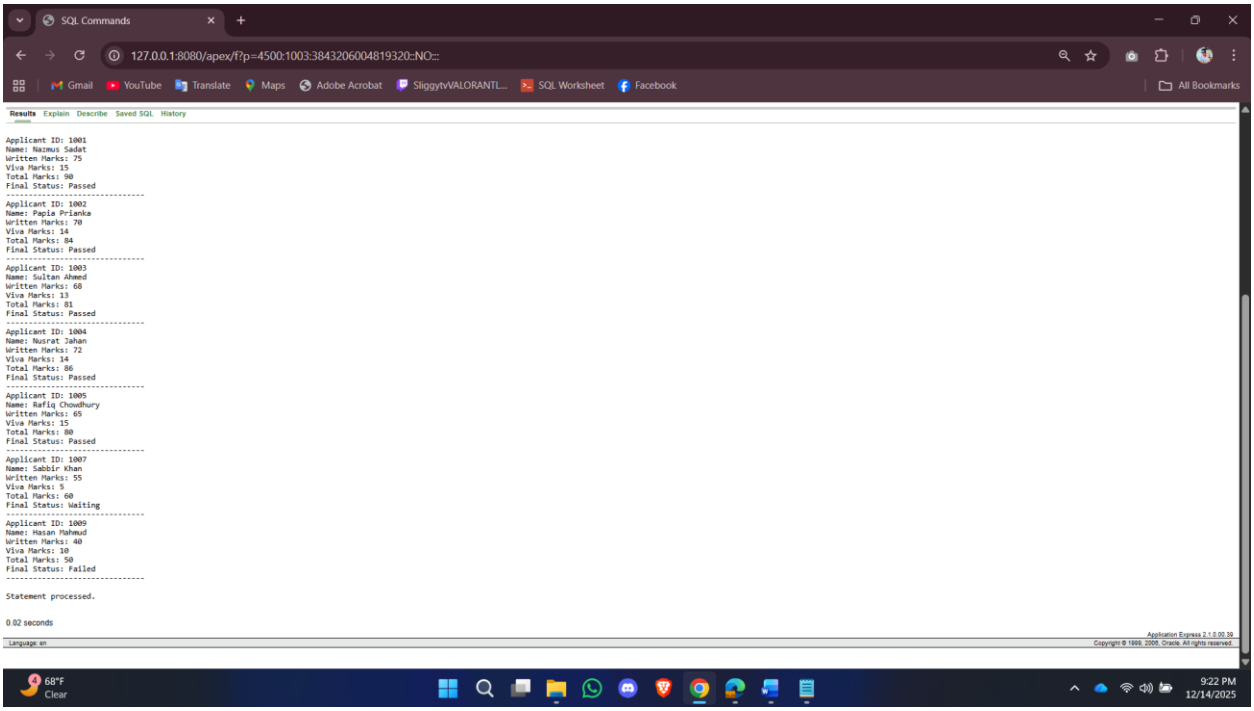


16. Write a PL/SQL block to display each applicant's ID and name along with their evaluation details: written marks, viva marks, total marks (sum of written and viva), and final status.

```
BEGIN
  FOR rec IN (
    SELECT a.applicant_id,
           a.applicant_name,
           e.written_marks,
           e.viva_marks,
           (e.written_marks + e.viva_marks) AS total_marks,
           e.final_status
    FROM Applicant a
    INNER JOIN Application ap ON a.applicant_id = ap.applicant_id
    INNER JOIN Evaluation e ON ap.application_id = e.application_id
  )
```

```
    LOOP
        DBMS_OUTPUT.PUT_LINE('Applicant ID: ' || rec.applicant_id);
        DBMS_OUTPUT.PUT_LINE('Name: ' || rec.applicant_name);
        DBMS_OUTPUT.PUT_LINE('Written Marks: ' || rec.written_marks);
        DBMS_OUTPUT.PUT_LINE('Viva Marks: ' || rec.viva_marks);
        DBMS_OUTPUT.PUT_LINE('Total Marks: ' || rec.total_marks);
        DBMS_OUTPUT.PUT_LINE('Final Status: ' || rec.final_status);
        DBMS_OUTPUT.PUT_LINE('-------------------------------');
    END LOOP;
END;
/
```





Advance PL/SQL:

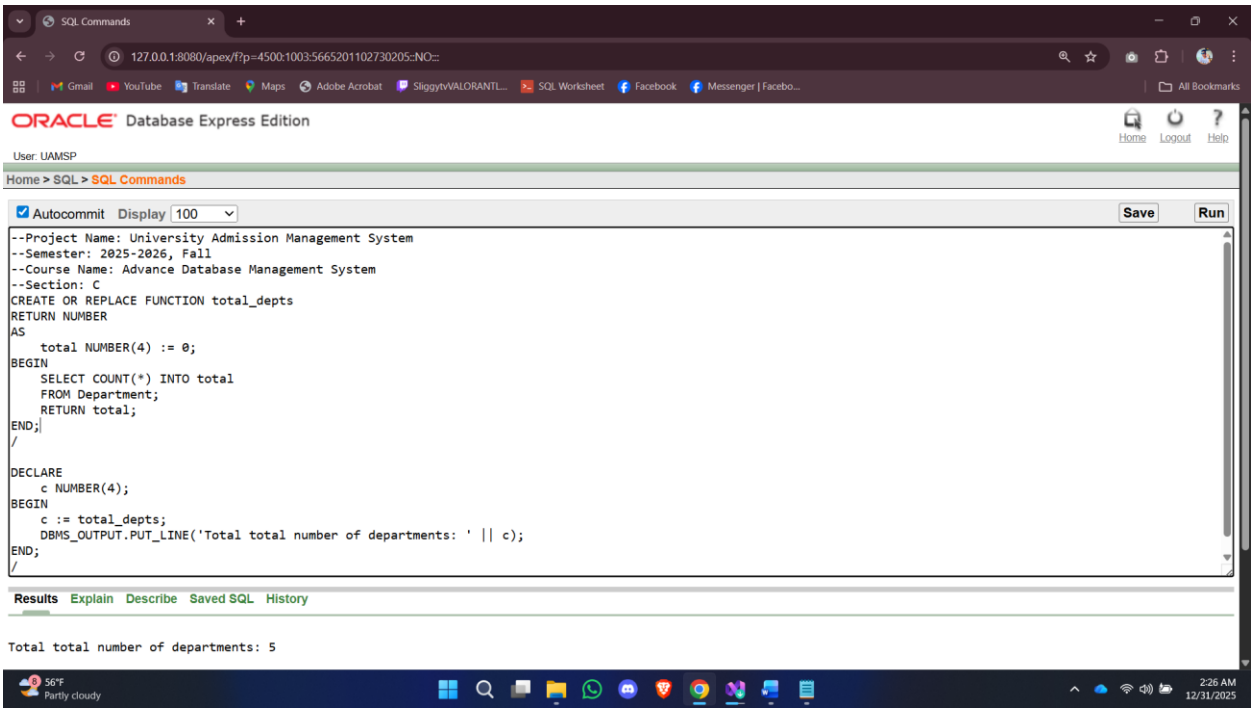
Stored function:

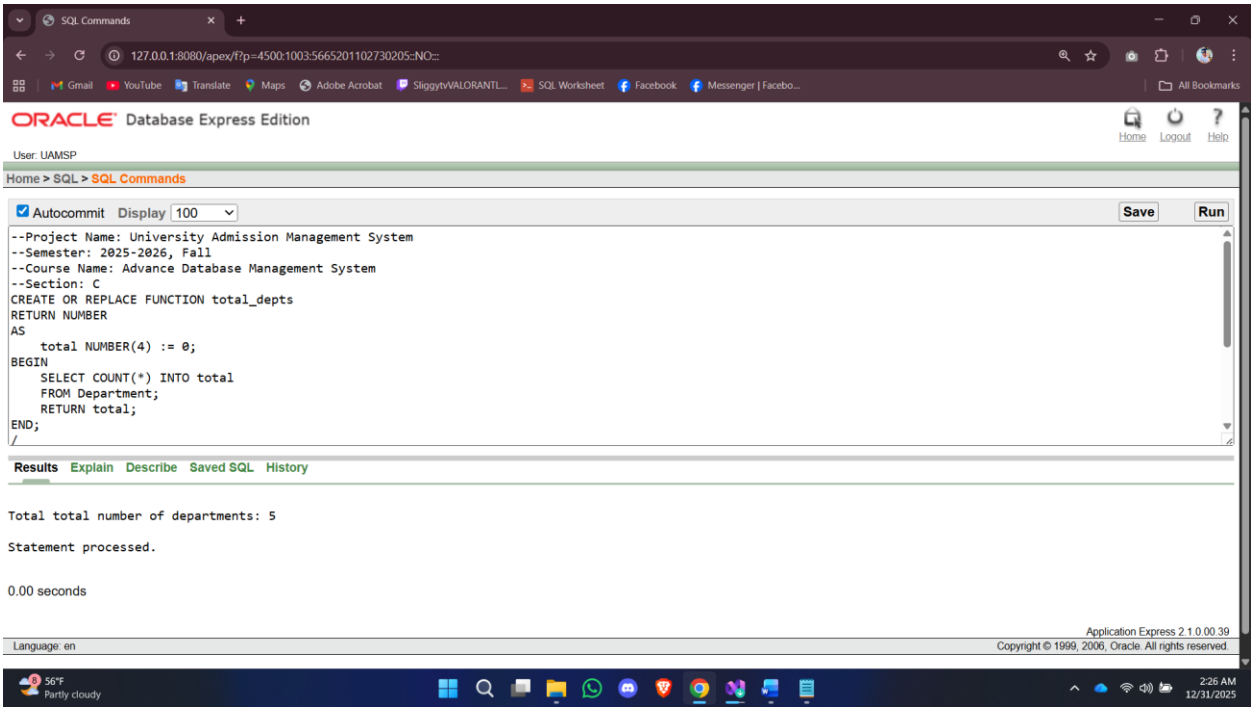
1. Create and call a PL/SQL stored function that returns the total number of departments.

```
CREATE OR REPLACE FUNCTION total_depts
RETURN NUMBER
AS
   total NUMBER(4) := 0;
BEGIN
   SELECT COUNT(*) INTO total
   FROM Department;
   RETURN total;
END;
/

DECLARE
   c NUMBER(4);
```

```
BEGIN
   c := total_depts;
   DBMS_OUTPUT.PUT_LINE('Total total number of departments: ' || c);
END;
/
```

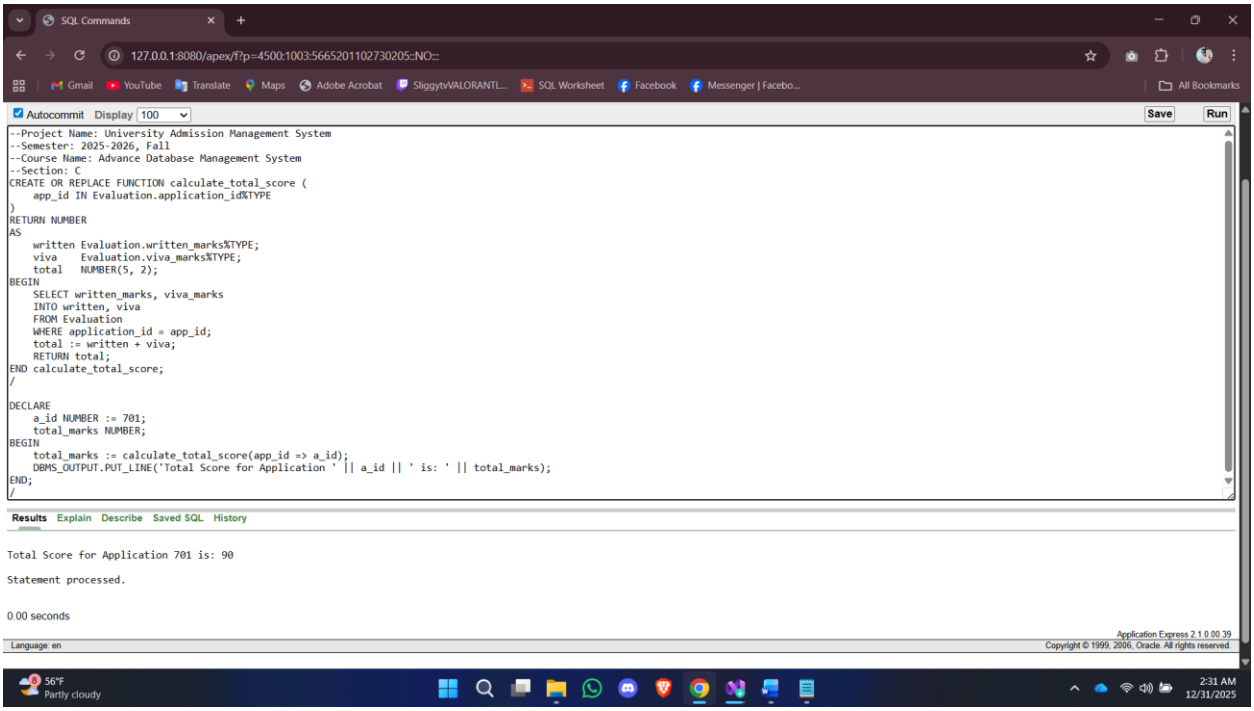




2. Create and call a PL/SQL stored function that returns the total marks of an application.

```
CREATE OR REPLACE FUNCTION calculate_total_score (
   app_id IN Evaluation.application_id%TYPE
)
RETURN NUMBER
AS
   written Evaluation.written_marks%TYPE;
   viva    Evaluation.viva_marks%TYPE;
   total   NUMBER(5, 2);
BEGIN
   SELECT written_marks, viva_marks
   INTO written, viva
   FROM Evaluation
   WHERE application_id = app_id;
   total := written + viva;
   RETURN total;
END calculate_total_score;
/

DECLARE
   a_id NUMBER := 701;
   total_marks NUMBER;
BEGIN
   total_marks := calculate_total_score(app_id => a_id);
   DBMS_OUTPUT.PUT_LINE('Total Score for Application ' || a_id || ' is: ' || total_marks);
END;
/
```

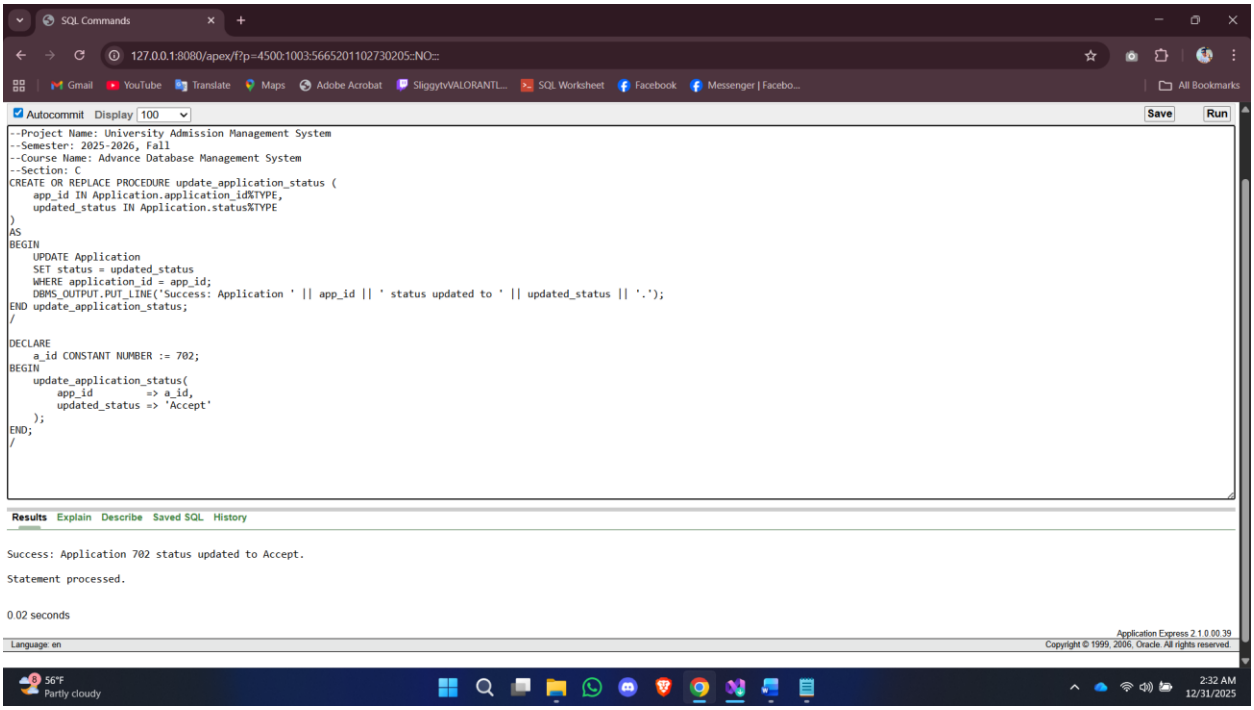


Stored procedure:

3. Create and call a PL/SQL stored procedure that takes an app_id and a updated_status as parameters and updates the status of the applicant's record in the Application table.

```
CREATE OR REPLACE PROCEDURE update_application_status (
    app_id IN Application.application_id%TYPE,
    updated_status IN Application.status%TYPE
)
AS
BEGIN
    UPDATE Application
    SET status = updated_status
    WHERE application_id = app_id;
    DBMS_OUTPUT.PUT_LINE('Success: Application ' || app_id || ' status updated to ' || updated_status || '.');
END update_application_status;
/
```

```
DECLARE
   a_id CONSTANT NUMBER := 702;
BEGIN
   update_application_status(
      app_id        => a_id,
      updated_status => 'Accept'
   );
END;
/
```



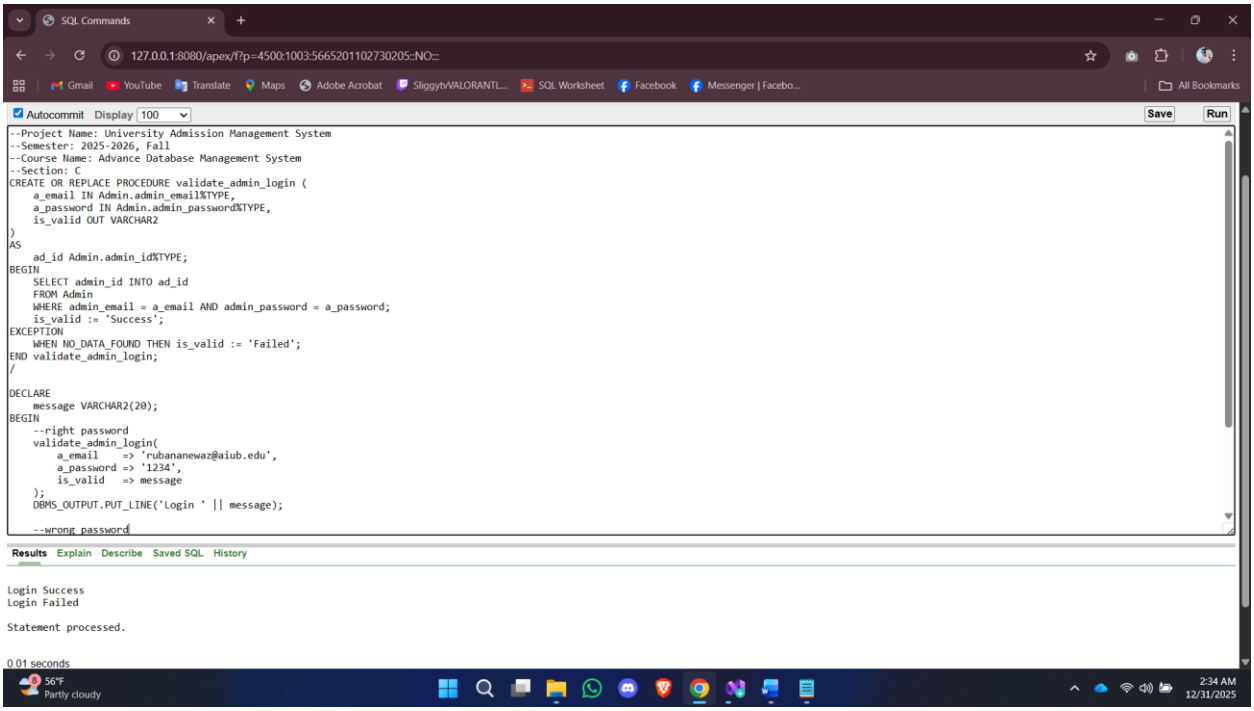
4. Create a PL/SQL stored procedure that validates the admin login and test it with both right and wrong passwords for admin Rubana Newaz.

```
CREATE OR REPLACE PROCEDURE validate_admin_login (
   a_email IN Admin.admin_email%TYPE,
   a_password IN Admin.admin_password%TYPE,
   is_valid OUT VARCHAR2
)
AS
   ad_id Admin.admin_id%TYPE;
```

```
BEGIN
   SELECT admin_id INTO ad_id
   FROM Admin
   WHERE admin_email = a_email AND admin_password = a_password;
   is_valid := 'Success';
EXCEPTION
   WHEN NO_DATA_FOUND THEN is_valid := 'Failed';
END validate_admin_login;
/

DECLARE
   message VARCHAR2(20);
BEGIN
   --right password
   validate_admin_login(
      a_email    => 'rubananewaz@aiub.edu',
      a_password => '1234',
      is_valid   => message
   );
   DBMS_OUTPUT.PUT_LINE('Login ' || message);

   --wrong password
   validate_admin_login(
      a_email    => 'rubananewaz@aiub.edu',
      a_password => '1235',
      is_valid   => message
   );
   DBMS_OUTPUT.PUT_LINE('Login ' || message);
END;
/
```



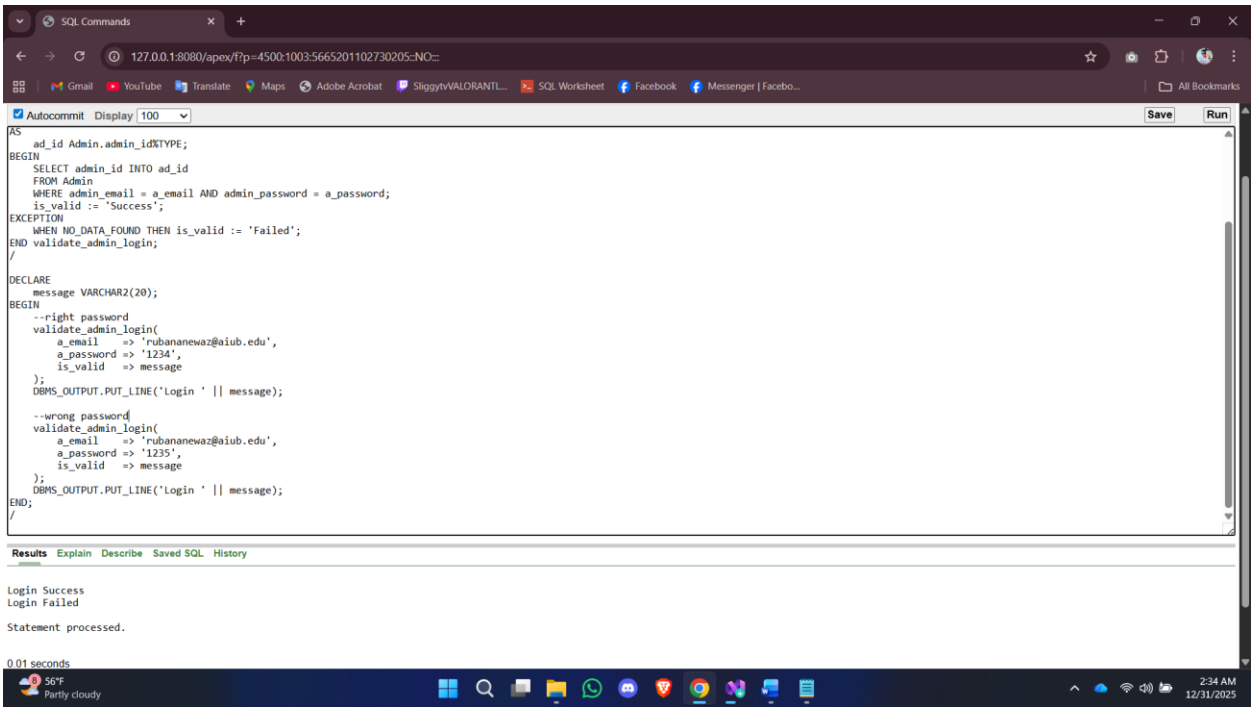


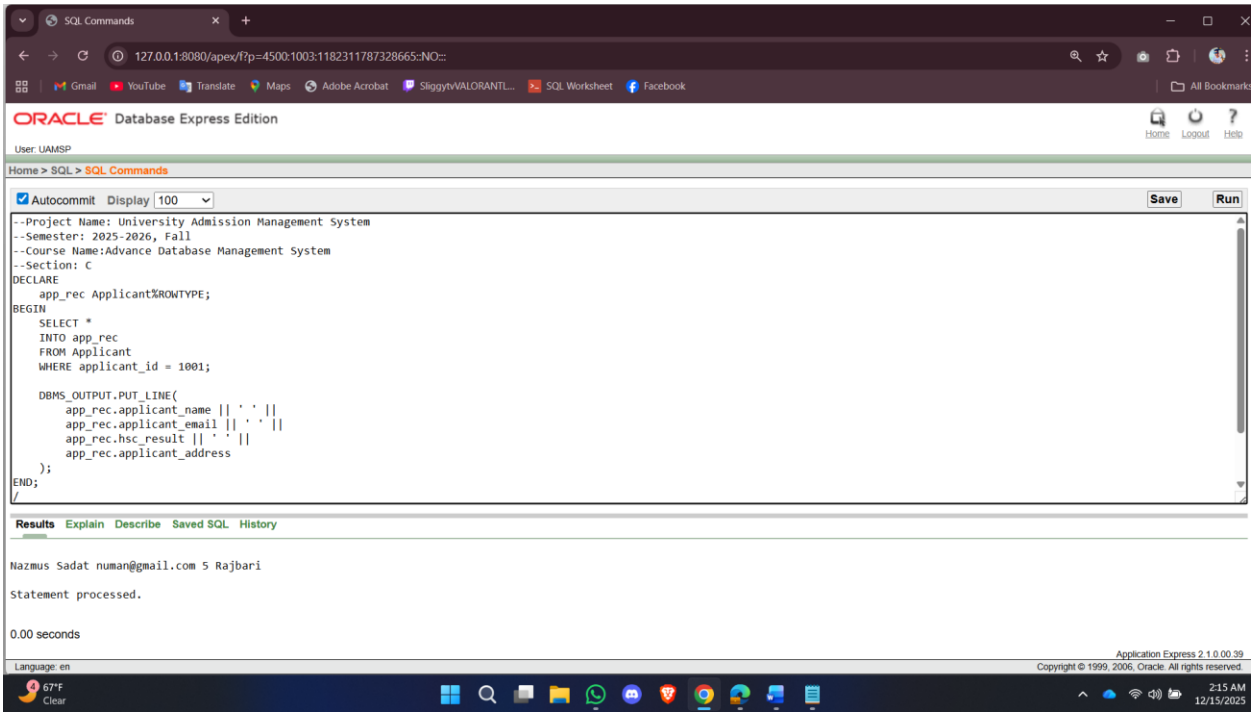
Table-based record:

5. Display full information of a specific applicant using a table-based record from the Applicant table.

```
DECLARE
  app_rec Applicant%ROWTYPE;
```

```
BEGIN
    SELECT *
    INTO app_rec
    FROM Applicant
    WHERE applicant_id = 1001;

    DBMS_OUTPUT.PUT_LINE(
        app_rec.applicant_name || ' ' ||
        app_rec.applicant_email || ' ' ||
        app_rec.hsc_result || ' ' ||
        app_rec.applicant_address
    );
END;
/
```

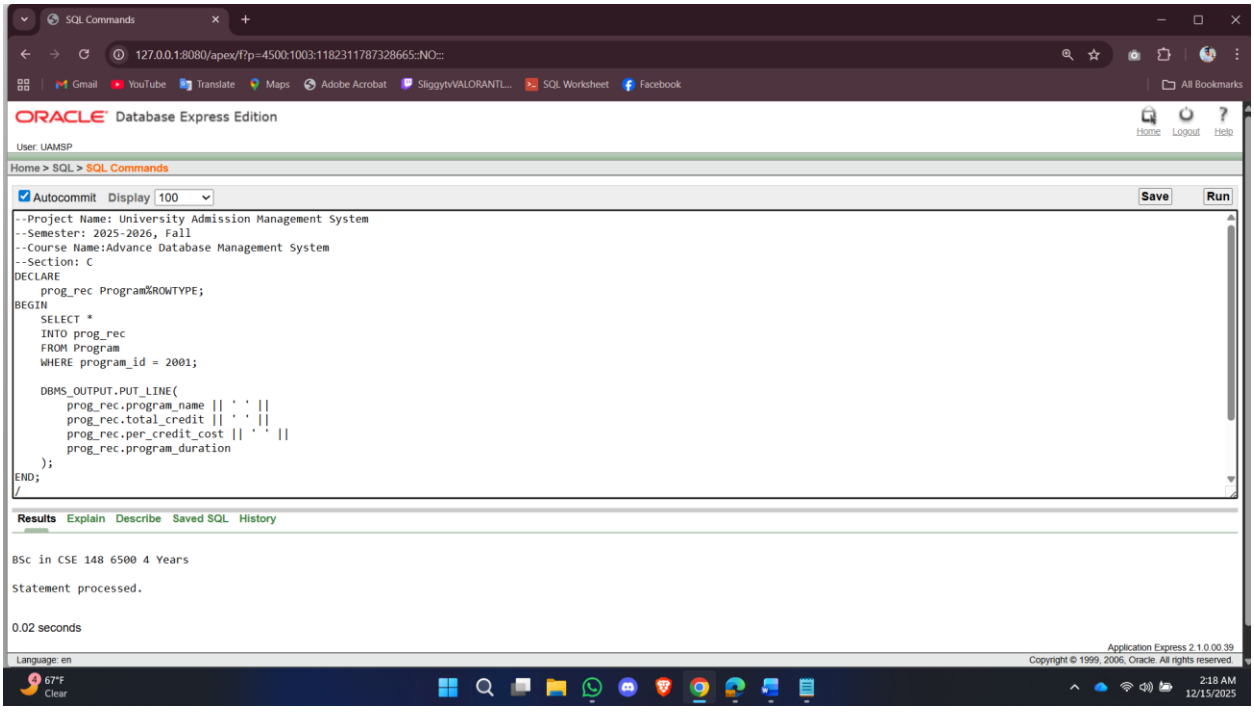


6. Retrieve and display complete program details for a given program using a table-based record from the Program table.

```
DECLARE
```

```
    prog_rec Program%ROWTYPE;
BEGIN
    SELECT *
    INTO prog_rec
    FROM Program
    WHERE program_id = 2001;

    DBMS_OUTPUT.PUT_LINE(
        prog_rec.program_name || ' ' ||
        prog_rec.total_credit || ' ' ||
        prog_rec.per_credit_cost || ' ' ||
        prog_rec.program_duration
    );
END;
/
```



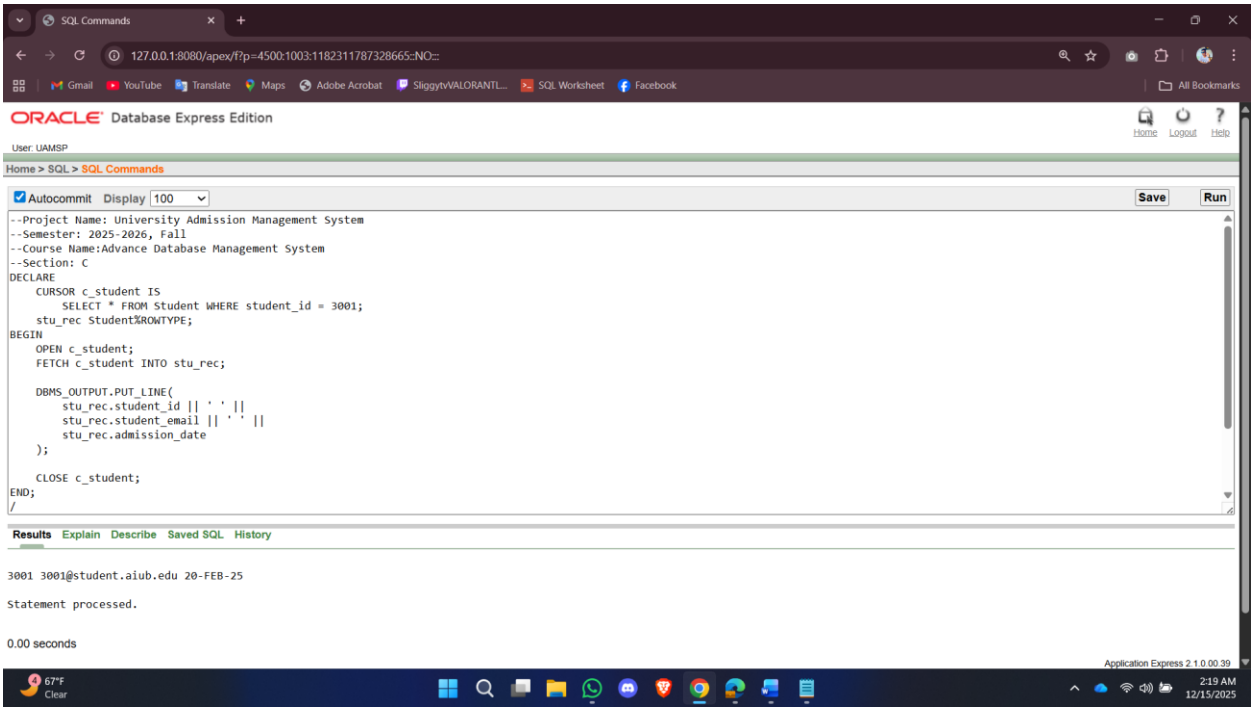
Explicit cursor:

7. Display student admission information using an explicit cursor from the Student table.

```
DECLARE
   CURSOR c_student IS
      SELECT * FROM Student WHERE student_id = 3001;
   stu_rec Student%ROWTYPE;
BEGIN
   OPEN c_student;
   FETCH c_student INTO stu_rec;

   DBMS_OUTPUT.PUT_LINE(
      stu_rec.student_id || ' ' ||
      stu_rec.student_email || ' ' ||
      stu_rec.admission_date
   );

   CLOSE c_student;
END;
/
```



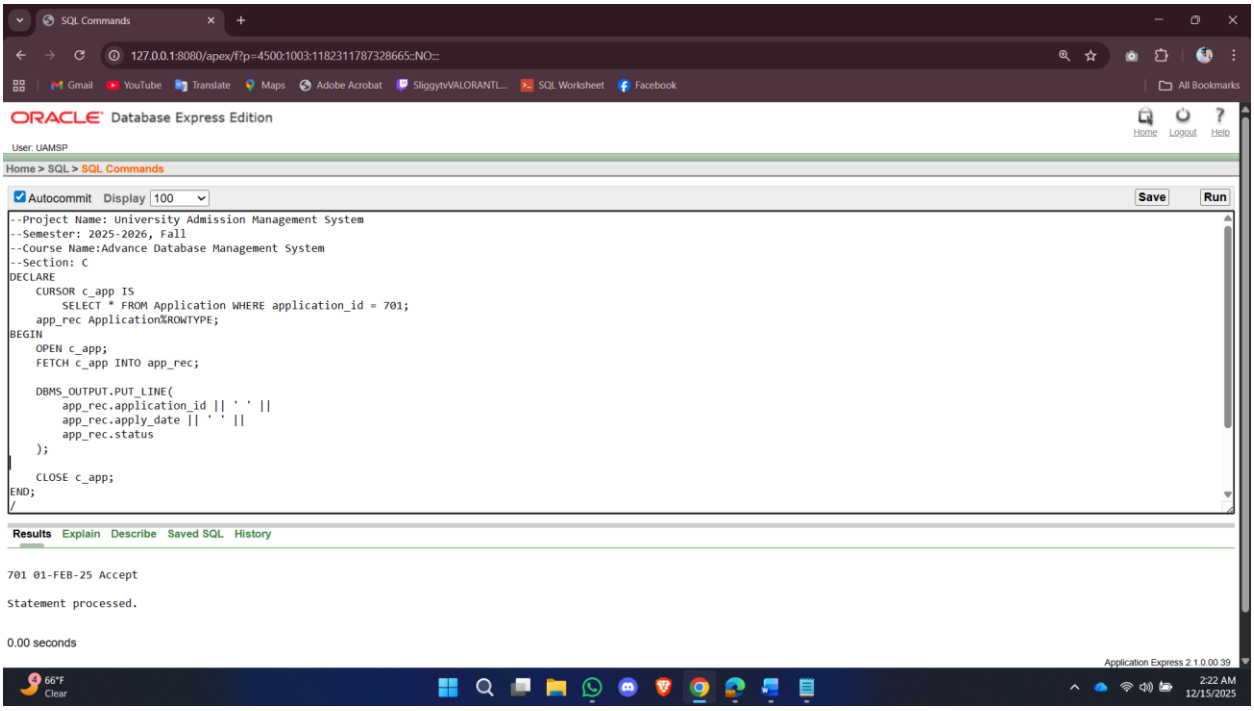
8. Retrieve application status details using an explicit cursor from the Application table.

```
DECLARE
   CURSOR c_app IS
      SELECT * FROM Application WHERE application_id = 701;
   app_rec Application%ROWTYPE;
BEGIN
   OPEN c_app;
   FETCH c_app INTO app_rec;

   DBMS_OUTPUT.PUT_LINE(
      app_rec.application_id || ' ' ||
      app_rec.apply_date || ' ' ||
      app_rec.status
   );

   CLOSE c_app;
END;
```

/



Cursor-based record:

9. Display the applicant name, written marks, viva marks, and final evaluation status given by a specific teacher using a cursor-based record.

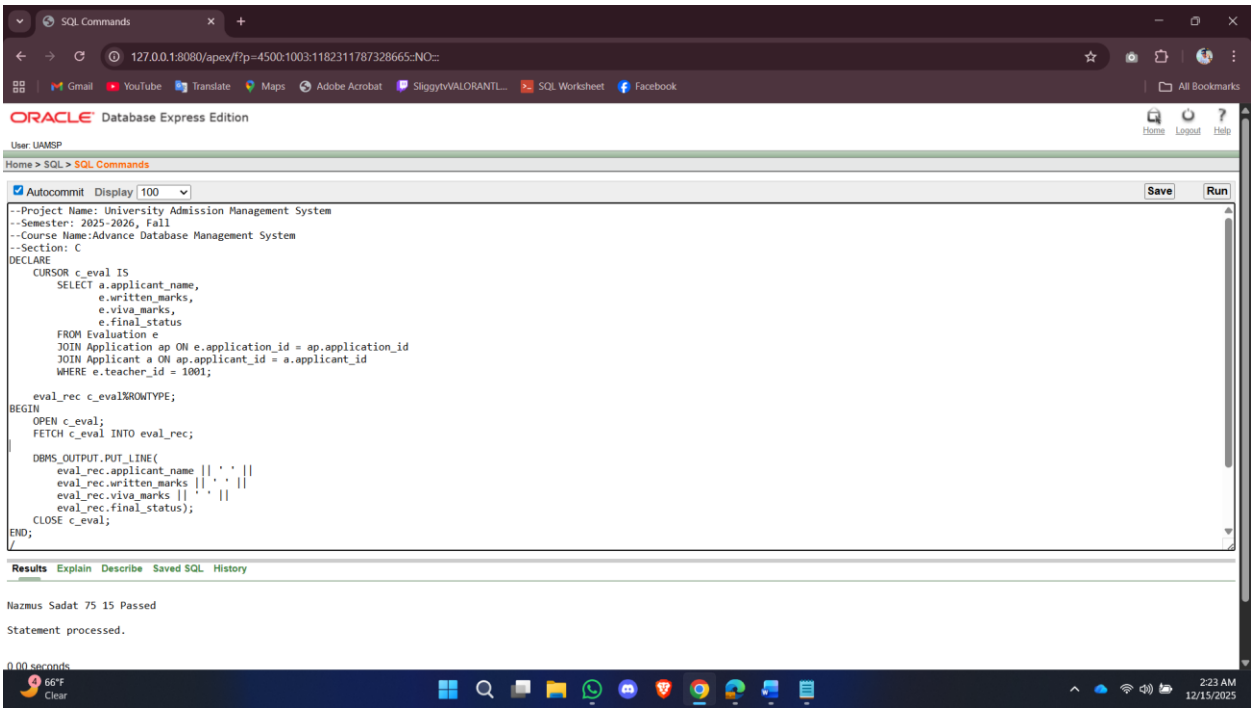
```
DECLARE
   CURSOR c_eval IS
      SELECT a.applicant_name,
           e.written_marks,
           e.viva_marks,
           e.final_status
      FROM Evaluation e
      JOIN Application ap ON e.application_id = ap.application_id
      JOIN Applicant a ON ap.applicant_id = a.applicant_id
      WHERE e.teacher_id = 1001;

   eval_rec c_eval%ROWTYPE;
BEGIN
   OPEN c_eval;
```

```
    FETCH c_eval INTO eval_rec;

    DBMS_OUTPUT.PUT_LINE(
        eval_rec.applicant_name || ' ' ||
        eval_rec.written_marks || ' ' ||
        eval_rec.viva_marks || ' ' ||
        eval_rec.final_status
    );

    CLOSE c_eval;
END;
/
```



10. Display the applicant ID and applicant name whose applications were rejected by a specific admin using a cursor-based record.

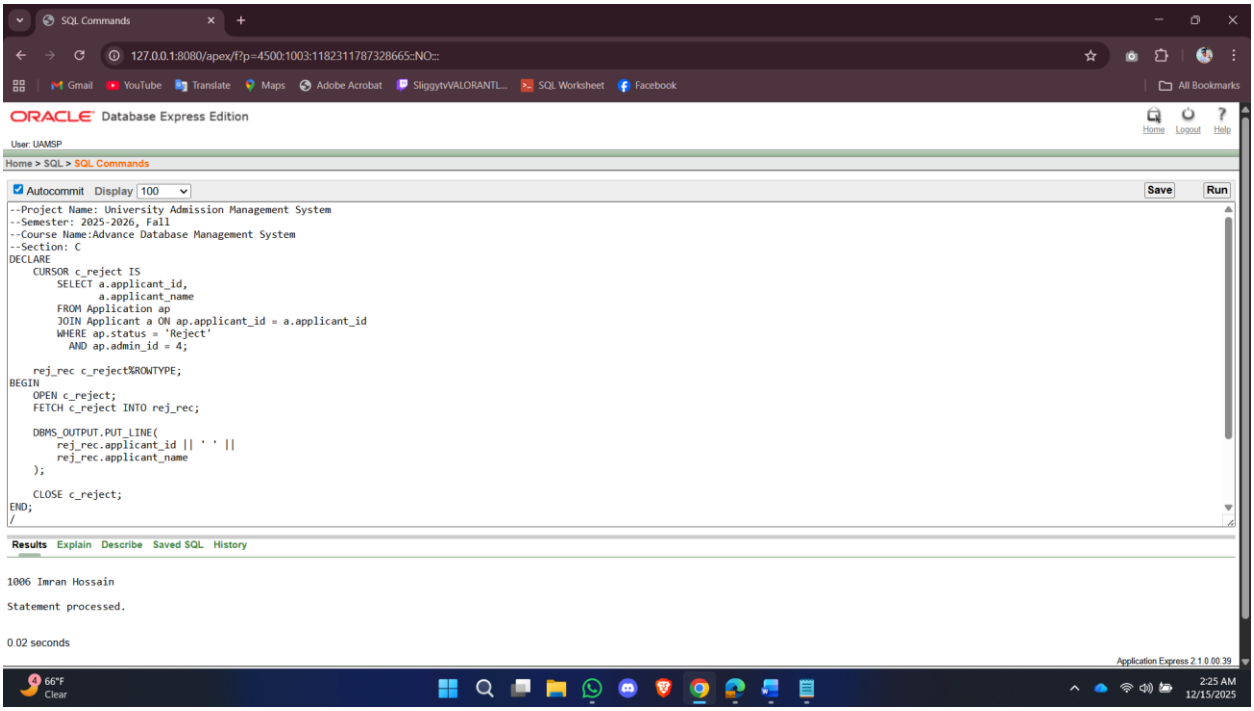
```
DECLARE
    CURSOR c_reject IS
        SELECT a.applicant_id,
               a.applicant_name
```

```
        FROM Application ap
        JOIN Applicant a ON ap.applicant_id = a.applicant_id
        WHERE ap.status = 'Reject'
          AND ap.admin_id = 4;

    rej_rec c_reject%ROWTYPE;
BEGIN
    OPEN c_reject;
    FETCH c_reject INTO rej_rec;

    DBMS_OUTPUT.PUT_LINE(
        rej_rec.applicant_id || ' ' ||
        rej_rec.applicant_name
    );

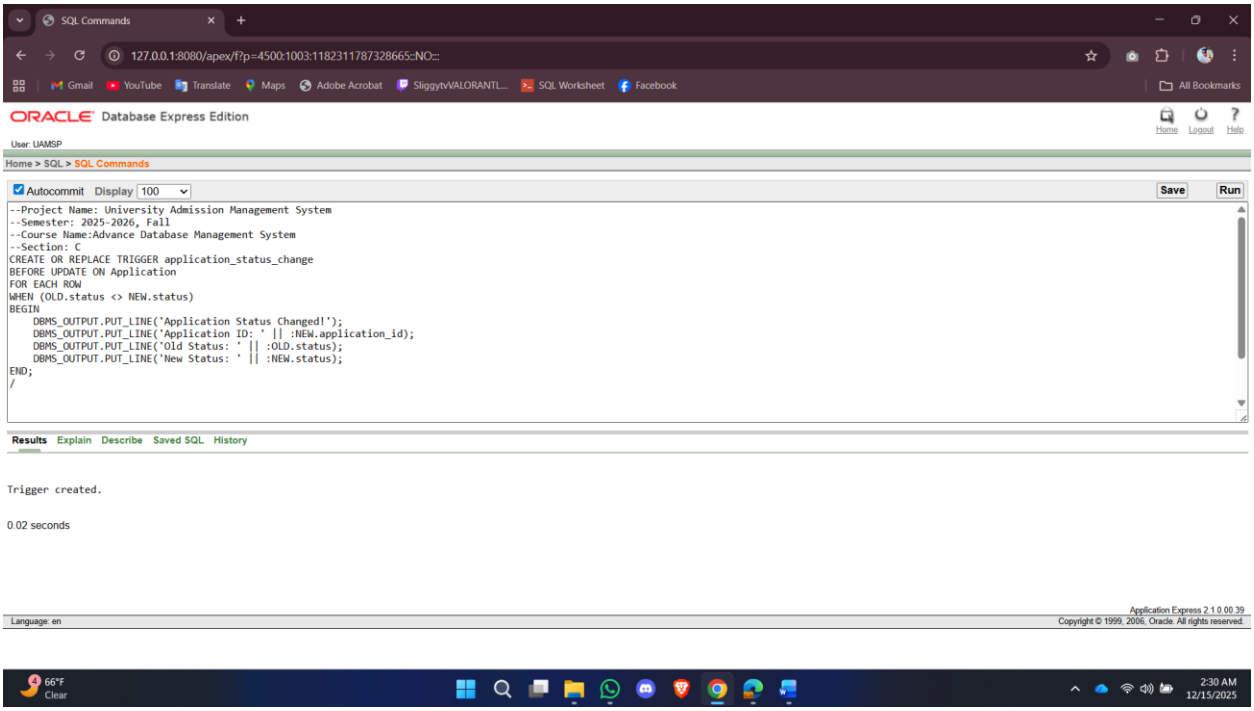
    CLOSE c_reject;
END;
/
```



Row level trigger:

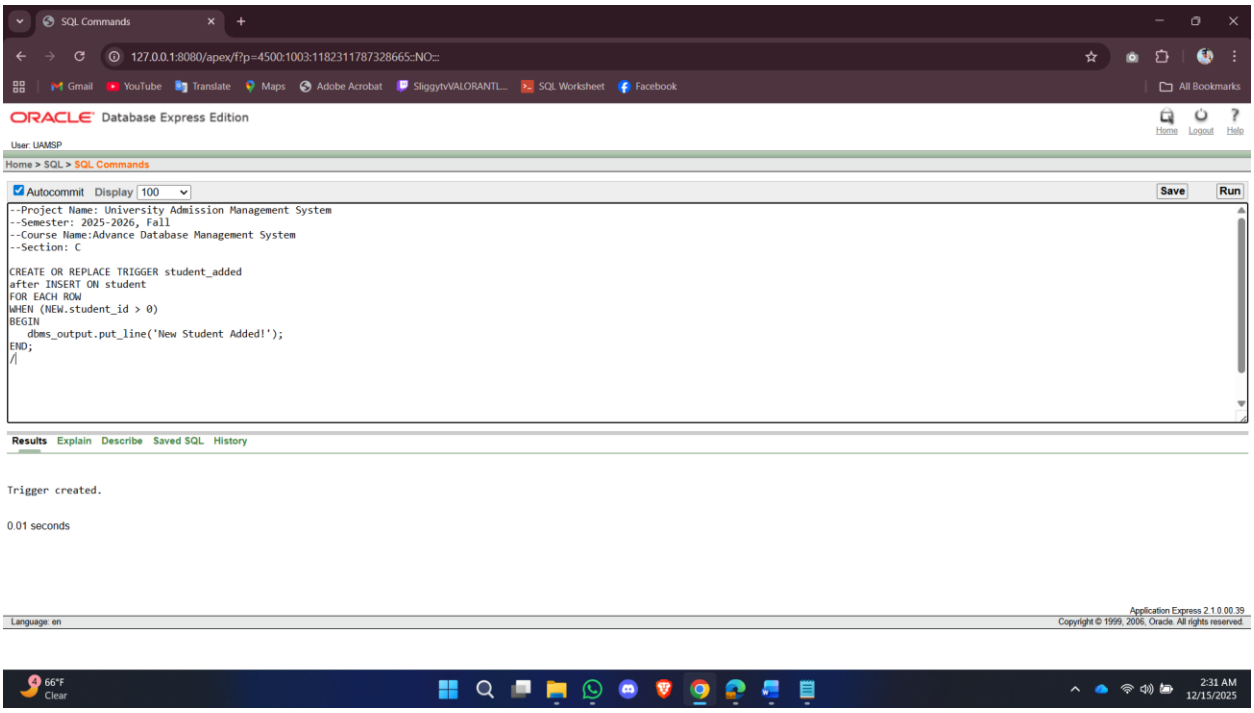
11. Create a PL/SQL row level trigger that sends a notification message containing application id, old status and new status when the status of the application is updated.

```
CREATE OR REPLACE TRIGGER application_status_change
BEFORE UPDATE ON Application
FOR EACH ROW
WHEN (OLD.status <> NEW.status)
BEGIN
   DBMS_OUTPUT.PUT_LINE('Application Status Changed!');
   DBMS_OUTPUT.PUT_LINE('Application ID: ' || :NEW.application_id);
   DBMS_OUTPUT.PUT_LINE('Old Status: ' || :OLD.status);
   DBMS_OUTPUT.PUT_LINE('New Status: ' || :NEW.status);
END;
/
```



12. Create a PL/SQL row level trigger that sends a notification message when new student is added.

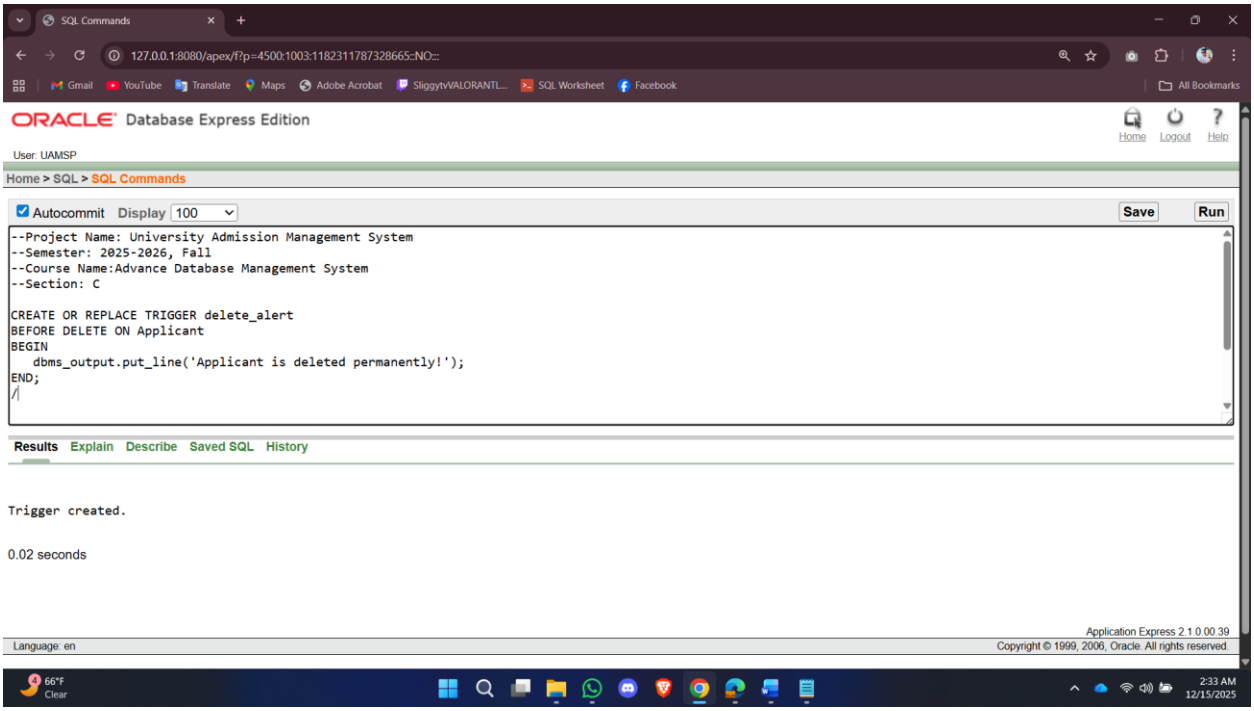
```
CREATE OR REPLACE TRIGGER student_added
after INSERT ON student
FOR EACH ROW
WHEN (NEW.student_id > 0)
BEGIN
  dbms_output.put_line('New Student Added!');
END;
/
```



Statement level trigger:

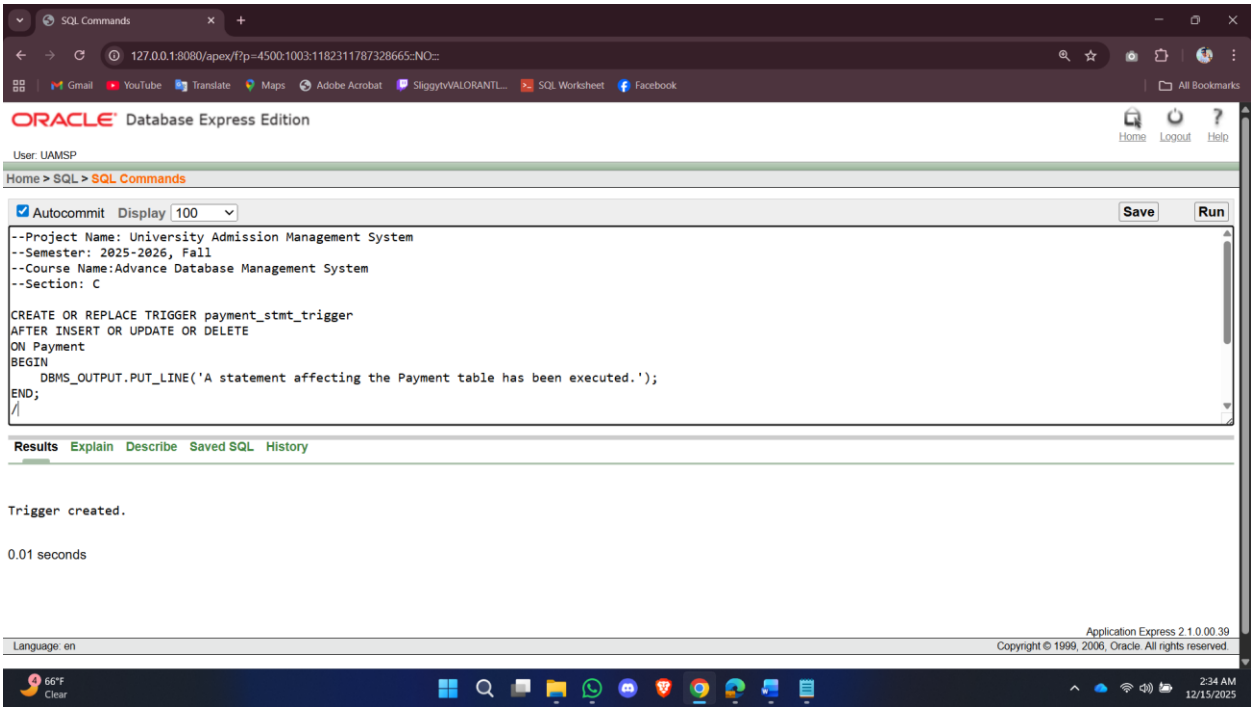
13. Create a PL/SQL statement level trigger that sends a notification message when an applicant is deleted.

```
CREATE OR REPLACE TRIGGER delete_alert
BEFORE DELETE ON Applicant
BEGIN
  dbms_output.put_line('Applicant is deleted permanently!');
END;
/
```



14. Create a statement-level trigger on the Payment table that automatically logs a message whenever any payment is inserted, updated, or deleted.

```
CREATE OR REPLACE TRIGGER payment_stmt_trigger
AFTER INSERT OR UPDATE OR DELETE
ON Payment
BEGIN
   DBMS_OUTPUT.PUT_LINE('A statement affecting the Payment table has been executed.');
END;
/
```



Package:

15. Create a PL/SQL package displays an applicant's ID, name and the corresponding payment amount and payment date of that applicant using applicant ID as input.

```
-- Create the package
CREATE OR REPLACE PACKAGE applicant_payment_pkg AS
   PROCEDURE show_applicant_info(p_applicant_id Applicant.applicant_id%TYPE);
   PROCEDURE show_payment_info(p_applicant_id Applicant.applicant_id%TYPE);
END applicant_payment_pkg;
/


-- Create the package body
CREATE OR REPLACE PACKAGE BODY applicant_payment_pkg AS

   PROCEDURE show_applicant_info(p_applicant_id Applicant.applicant_id%TYPE) IS
      a_name Applicant.applicant_name%TYPE;
   BEGIN
      SELECT applicant_name
      INTO a_name
```

```
    FROM Applicant
    WHERE applicant_id = p_applicant_id;

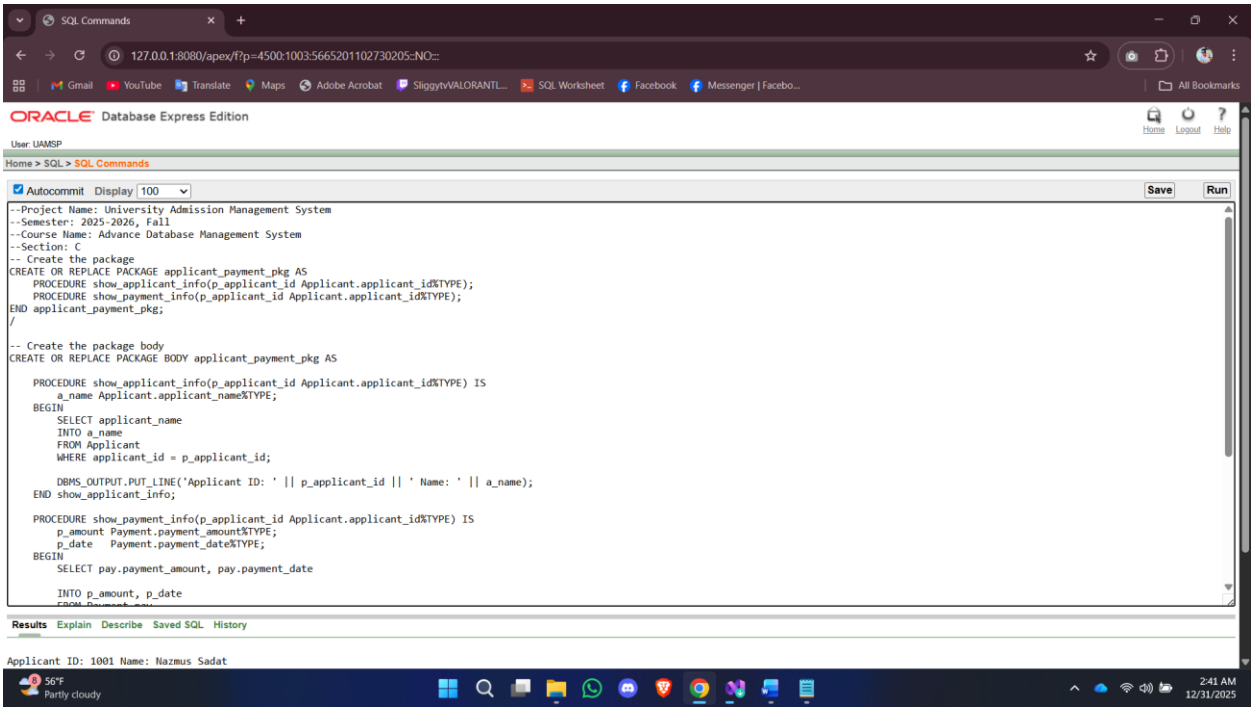
    DBMS_OUTPUT.PUT_LINE('Applicant ID: ' || p_applicant_id || ' Name: ' || a_name);
  END show_applicant_info;

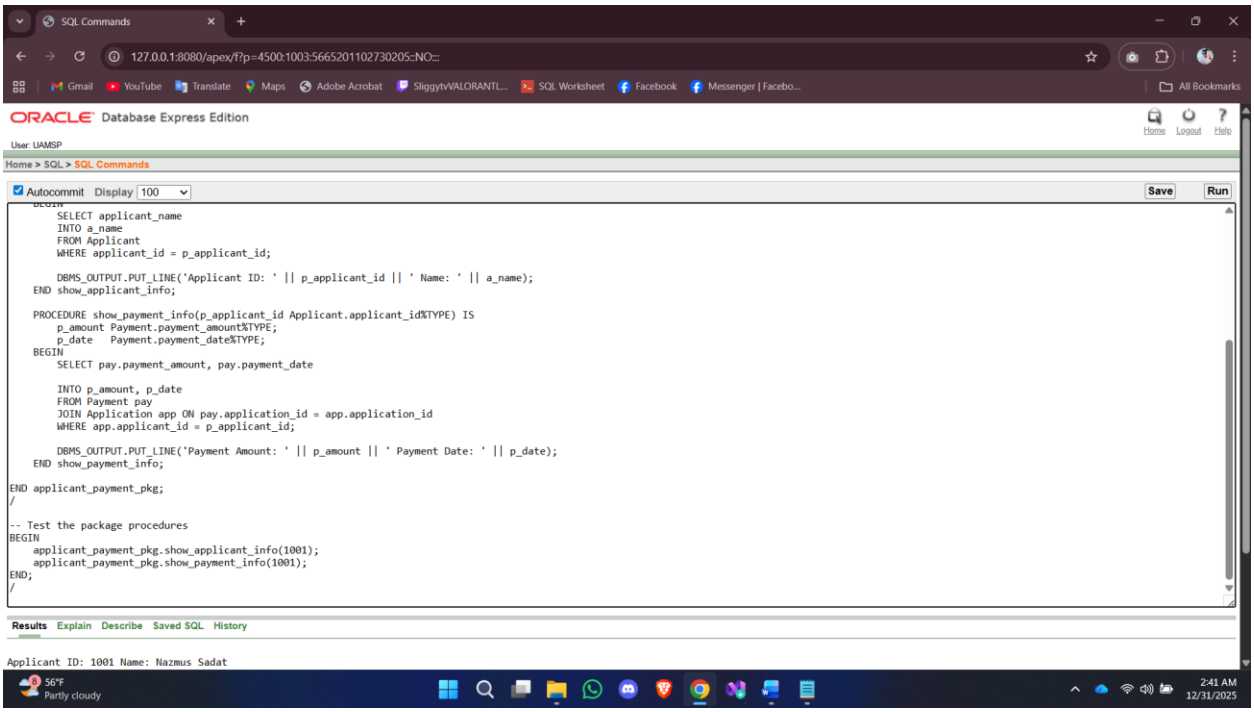
  PROCEDURE show_payment_info(p_applicant_id Applicant.applicant_id%TYPE) IS
    p_amount Payment.payment_amount%TYPE;
    p_date   Payment.payment_date%TYPE;
  BEGIN
    SELECT pay.payment_amount, pay.payment_date
    INTO p_amount, p_date
    FROM Payment pay
    JOIN Application app ON pay.application_id = app.application_id
    WHERE app.applicant_id = p_applicant_id;

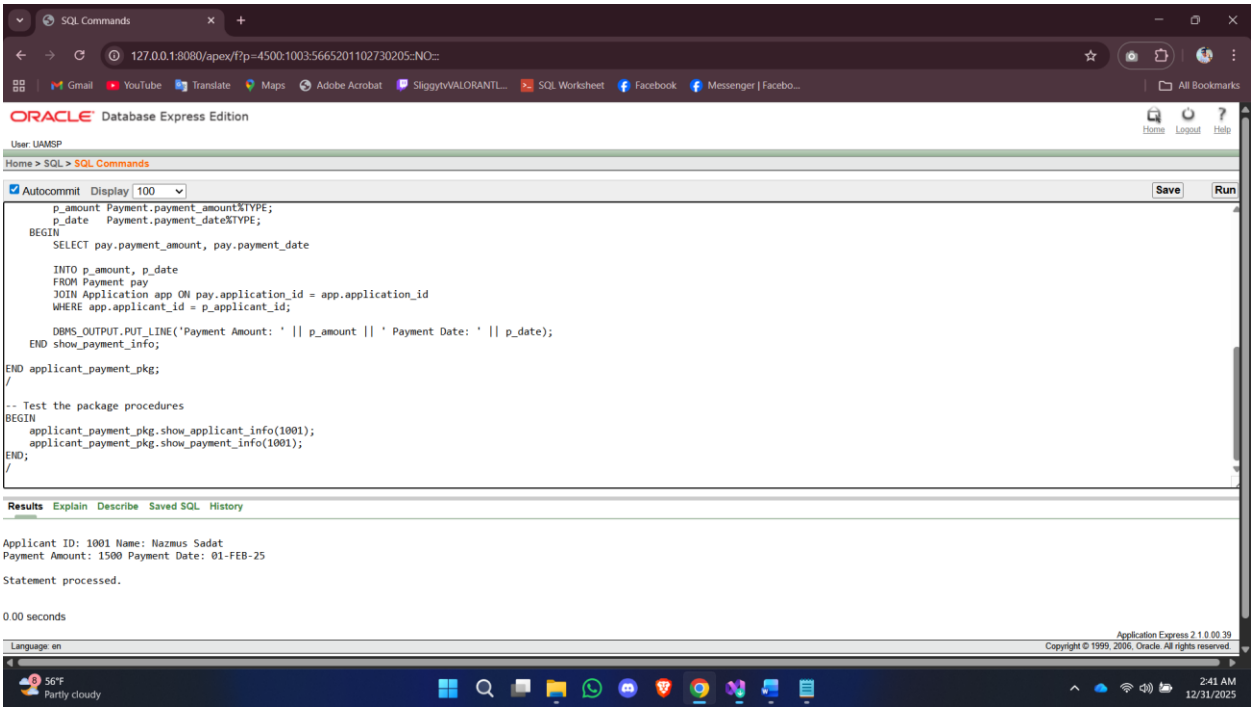
    DBMS_OUTPUT.PUT_LINE('Payment Amount: ' || p_amount || ' Payment Date: ' ||
p_date);
  END show_payment_info;

END applicant_payment_pkg;
/

-- Test the package procedures
BEGIN
  applicant_payment_pkg.show_applicant_info(1001);
  applicant_payment_pkg.show_payment_info(1001);
END;
/
```


SQL Commands
127.0.0.1:8080/apex/f?p=4500:1003:5665201102730205::NO:::
ORACLE Database Express Edition
User: UAMSP
Home > SQL > SQL Commands
Autocommit Display 100
Save
Run
--Project Name: University Admission Management System
--Semester: 2025-2026, Fall
--Course Name: Advance Database Management System
--Section: C
-- Create the package
CREATE OR REPLACE PACKAGE applicant_payment_pkg AS
PROCEDURE show_applicant_info(p_applicant_id Applicant.applicant_id%TYPE);
PROCEDURE show_payment_info(p_applicant_id Applicant.applicant_id%TYPE);
END applicant_payment_pkg;
/
-- Create the package body
CREATE OR REPLACE PACKAGE BODY applicant_payment_pkg AS
PROCEDURE show_applicant_info(p_applicant_id Applicant.applicant_id%TYPE) IS
a_name Applicant.applicant_name%TYPE;
BEGIN
SELECT applicant_name
INTO a_name
FROM Applicant
WHERE applicant_id = p_applicant_id;
DBMS_OUTPUT.PUT_LINE('Applicant ID: ' || p_applicant_id || ' Name: ' || a_name);
END show_applicant_info;
PROCEDURE show_payment_info(p_applicant_id Applicant.applicant_id%TYPE) IS
p_amount Payment.payment_amount%TYPE;
p_date Payment.payment_date%TYPE;
BEGIN
SELECT pay.payment_amount, pay.payment_date
INTO p_amount, p_date
Results Explain Describe Saved SQL History
Applicant ID: 1001 Name: Nazmus Sadat


SQL Commands
127.0.0.1:8080/apex/f?p=4500:1003:5665201102730205::NO:::
ORACLE Database Express Edition
User: UAMSP
Home > SQL > SQL Commands
Autocommit Display 100
Save
Run
SELECT applicant_name
INTO a_name
FROM Applicant
WHERE applicant_id = p_applicant_id;
DBMS_OUTPUT.PUT_LINE('Applicant ID: ' || p_applicant_id || ' Name: ' || a_name);
END show_applicant_info;
PROCEDURE show_payment_info(p_applicant_id Applicant.applicant_id%TYPE) IS
p_amount Payment.payment_amount%TYPE;
p_date Payment.payment_date%TYPE;
BEGIN
SELECT pay.payment_amount, pay.payment_date
INTO p_amount, p_date
FROM Payment pay
JOIN Application app ON pay.application_id = app.application_id
WHERE app.applicant_id = p_applicant_id;
DBMS_OUTPUT.PUT_LINE('Payment Amount: ' || p_amount || ' Payment Date: ' || p_date);
END show_payment_info;
END applicant_payment_pkg;
/
-- Test the package procedures
BEGIN
applicant_payment_pkg.show_applicant_info(1001);
applicant_payment_pkg.show_payment_info(1001);
END;
/
Results Explain Describe Saved SQL History
Applicant ID: 1001 Name: Nazmus Sadat



16. Create a PL/SQL package that take an applicant ID and displays the student's ID, student name, viva marks, written marks, department name, and program name.

```
-- Create the package specification
CREATE OR REPLACE PACKAGE student_evaluation_pkg AS
    PROCEDURE show_student_evaluation(p_applicant_id Applicant.applicant_id%TYPE);
    FUNCTION get_total_marks(p_applicant_id Applicant.applicant_id%TYPE) RETURN NUMBER;
END student_evaluation_pkg;
/

-- Create the package body
CREATE OR REPLACE PACKAGE BODY student_evaluation_pkg AS

    -- Procedure to display student evaluation info
    PROCEDURE show_student_evaluation(p_applicant_id Applicant.applicant_id%TYPE) IS
        s_id      Student.student_id%TYPE;
        s_name    Applicant.applicant_name%TYPE;
        v_mark    Evaluation.viva_marks%TYPE;
```

```
    w_mark    Evaluation.written_marks%TYPE;
    d_name    Department.dept_name%TYPE;
    p_name    Program.program_name%TYPE;
  BEGIN
    SELECT s.student_id, a.applicant_name, e.viva_marks, e.written_marks, d.dept_name,
pr.program_name
    INTO s_id, s_name, v_mark, w_mark, d_name, p_name
    FROM Student s
    JOIN Applicant a ON s.applicant_id = a.applicant_id
    JOIN Evaluation e ON e.application_id = (SELECT application_id FROM Application
WHERE applicant_id = a.applicant_id)
    JOIN Department d ON s.dept_id = d.dept_id
    JOIN Program pr ON s.program_id = pr.program_id
    WHERE a.applicant_id = p_applicant_id;

    DBMS_OUTPUT.PUT_LINE('Student ID: ' || s_id ||
                         ', Name: ' || s_name ||
                         ', Viva Marks: ' || v_mark ||
                         ', Written Marks: ' || w_mark ||
                         ', Department: ' || d_name ||
                         ', Program: ' || p_name);
  END show_student_evaluation;

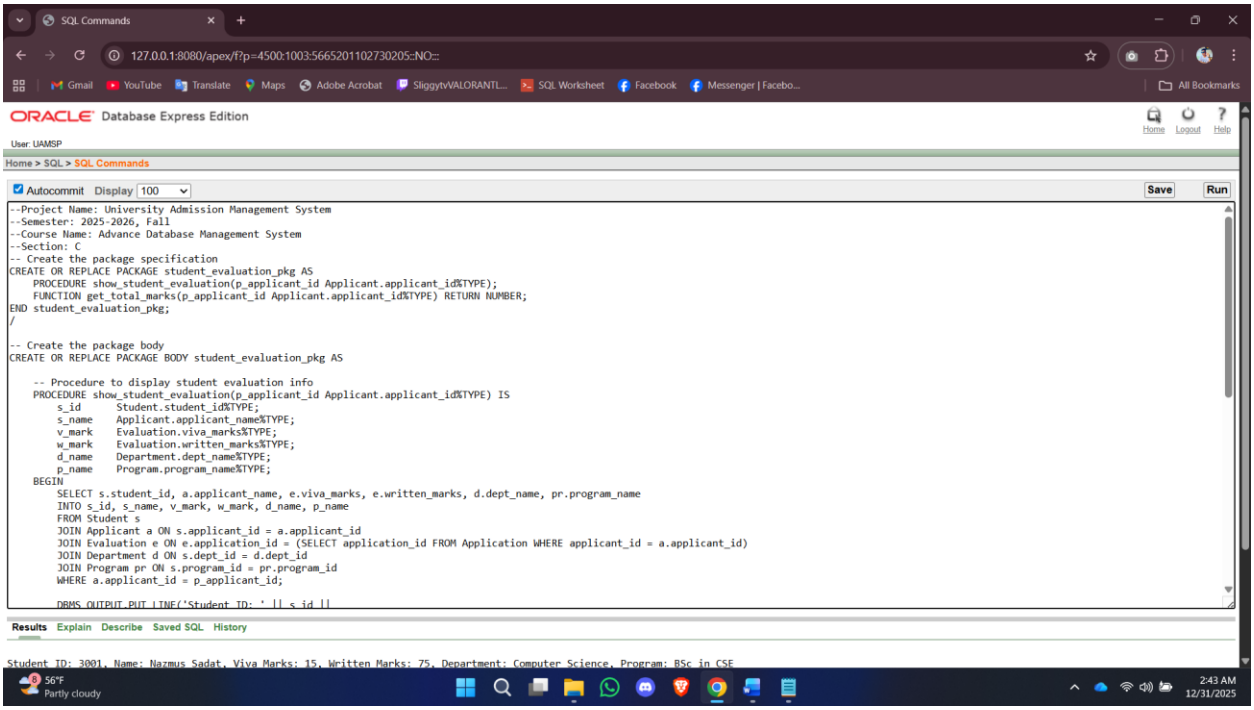
  -- Function to calculate total marks
  FUNCTION get_total_marks(p_applicant_id Applicant.applicant_id%TYPE) RETURN
NUMBER IS
    total_marks NUMBER;
  BEGIN
    SELECT (e.viva_marks + e.written_marks)
    INTO total_marks
    FROM Evaluation e
```

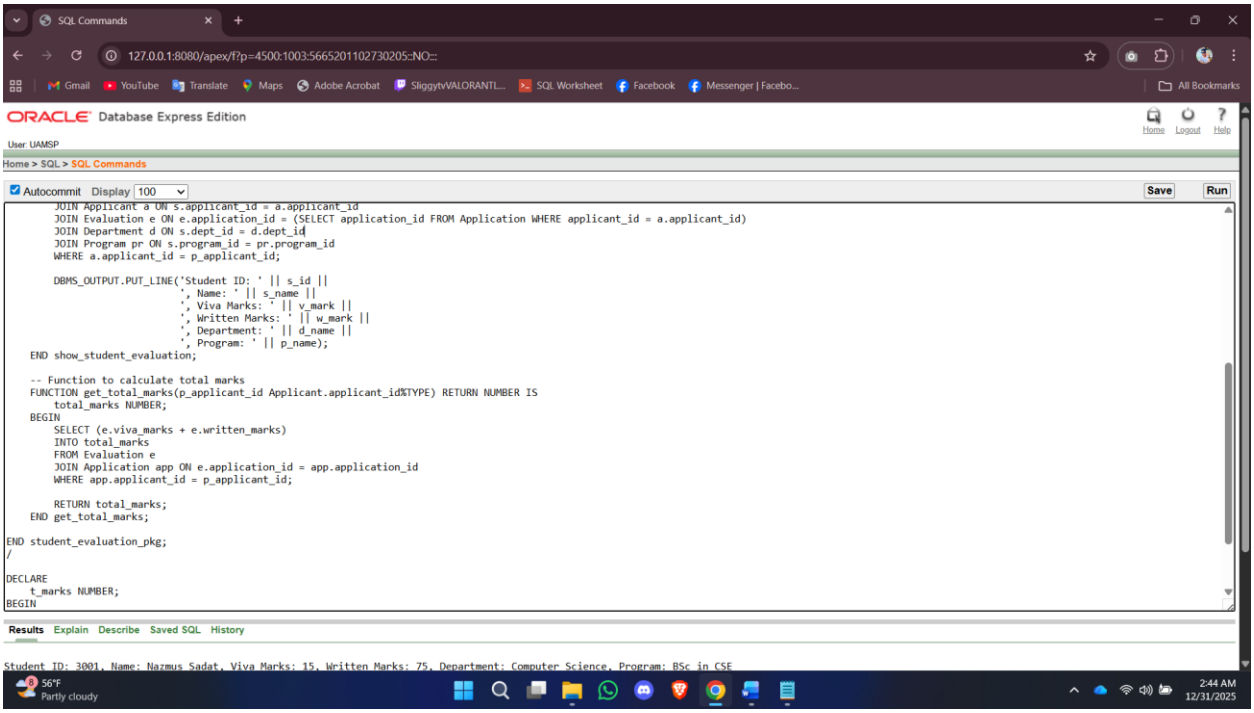
```
        JOIN Application app ON e.application_id = app.application_id
        WHERE app.applicant_id = p_applicant_id;

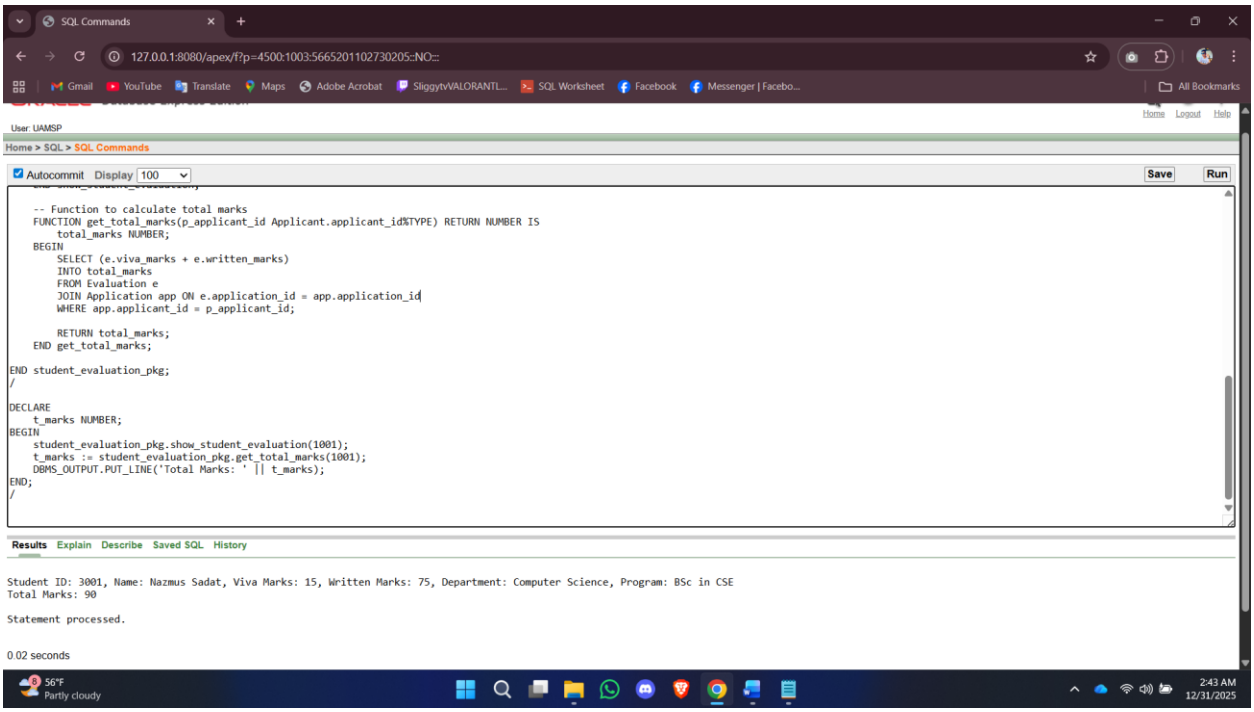
        RETURN total_marks;
    END get_total_marks;

END student_evaluation_pkg;
/

DECLARE
    t_marks NUMBER;
BEGIN
    student_evaluation_pkg.show_student_evaluation(1001);
    t_marks := student_evaluation_pkg.get_total_marks(1001);
    DBMS_OUTPUT.PUT_LINE('Total Marks: ' || t_marks);
END;
/
```




SQL Commands
127.0.0.1:8080/apex/f?p=4500:1003:5665201102730205::NO:::
ORACLE Database Express Edition
User: UAMSP
Home > SQL > SQL Commands
Autocommit Display 100
Save
Run
JOIN Applicant a ON s.applicant_id = a.applicant_id
JOIN Evaluation e ON e.application_id = (SELECT application_id FROM Application WHERE applicant_id = a.applicant_id)
JOIN Department d ON s.dept_id = d.dept_id
JOIN Program pr ON s.program_id = pr.program_id
WHERE a.applicant_id = p_applicant_id;
DBMS_OUTPUT.PUT_LINE('Student ID: ' || s_id ||
', Name: ' || s_name ||
', Viva Marks: ' || v_mark ||
', Written Marks: ' || w_mark ||
', Department: ' || d_name ||
', Program: ' || p_name);
END show_student_evaluation;
-- Function to calculate total marks
FUNCTION get_total_marks(p_applicant_id Applicant.applicant_id%TYPE) RETURN NUMBER IS
total_marks NUMBER;
BEGIN
SELECT (e.viva_marks + e.written_marks)
INTO total_marks
FROM Evaluation e
JOIN Application app ON e.application_id = app.application_id
WHERE app.applicant_id = p_applicant_id;
RETURN total_marks;
END get_total_marks;
END student_evaluation_pkg;
/
DECLARE
t_marks NUMBER;
BEGIN
Results Explain Describe Saved SQL History
Student ID: 3001, Name: Nazmus Sadat, Viva Marks: 15, Written Marks: 75, Department: Computer Science, Program: BSc in CSE
56°F
Partly cloudy
2:44 AM
12/31/2025


SQL Commands
127.0.0.1:8080/apex/f?p=4500:1003:5665201102730205::NO:::
User: UAMSP
Home > SQL > SQL Commands
Autocommit Display 100
Save
Run
-- Function to calculate total marks
FUNCTION get_total_marks(p_applicant_id Applicant.applicant_id%TYPE) RETURN NUMBER IS
total_marks NUMBER;
BEGIN
SELECT (e.viva_marks + e.written_marks)
INTO total_marks
FROM Evaluation e
JOIN Application app ON e.application_id = app.application_id
WHERE app.applicant_id = p_applicant_id;
RETURN total_marks;
END get_total_marks;
END student_evaluation_pkg;
/
DECLARE
t_marks NUMBER;
BEGIN
student_evaluation_pkg.show_student_evaluation(1001);
t_marks := student_evaluation_pkg.get_total_marks(1001);
DBMS_OUTPUT.PUT_LINE('Total Marks: ' || t_marks);
END;
/
Results Explain Describe Saved SQL History
Student ID: 3001, Name: Nazmus Sadat, Viva Marks: 15, Written Marks: 75, Department: Computer Science, Program: BSc in CSE
Total Marks: 90
Statement processed.
0.02 seconds
56°F
Partly cloudy
2:43 AM
12/31/2025

Conclusion:

The University Admission Management System has been successfully designed and implemented as a complete and well-structured relational database system that addresses the real-world challenges of managing a university admission process. Throughout this project, we transformed a complex admission workflow. The project's key achievement is the transformation of a complex admission workflow into an optimally structured and functionally effective relational database. The project backend is developed by minimal redundancy, guaranteed data consistency and robust referential integrity which are achieved through rigorous normalization and the strategic use of constraints. The insertion of realistic and comprehensive sample data confirmed the robustness of the schema constraints across various scenarios. This process validated that the database structure can correctly handle the diversity and complexity of real-world admission data. Furthermore, both basic and advanced PL/SQL programs were developed to demonstrate practical database operations. These implementations verify that the database is not only structurally correct but also functionally effective. Overall, this project demonstrates a strong application of advanced database concepts. The University Admission Management System provides a reliable backend foundation that can support real-world university admission operations efficiently.

Proposed Future Work

Although the current version of the University Admission Management System fulfills all core database requirements, several enhancements can be introduced to improve usability, scalability, and real-world applicability:

1. **Advanced Reporting and Analytics**: Dashboards can be developed to generate statistical reports such as department-wise application counts, pass/fail ratios, payment summaries, and admission trends. These insights would help university authorities make informed decisions.
2. **Automation and Notification System**: Automated email or SMS notifications can be integrated to inform applicants about application status changes, payment confirmations, and evaluation results in real time.

3. **Scalability and Integration**: The system can be expanded to integrate with other university systems where applicant can apply in multiple university from a single web-app, forming a complete university management ecosystem.